МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 9

по дисциплине «Программирование корпоративных систем»

Тема работы: БАЗОВЫЙ УРОВЕНЬ ВЕБ-АВТОМАТИЗАЦИИ: УПРАВЛЕНИЕ БРАУЗЕРОМ И ВЗАИМОДЕЙСТВИЕ С DOM

Вариант: 11

Выполнил:	ЭФБО-09-23		Калинин Н.В.
	(группа)	*(подпись)*	*(Фамилия И.О.)*
Принял:	доц.		Громов Е.В.
	(должность)	*(подпись)*	*(Фамилия И.О.)*

Москва

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

На девятой практической работе мы заложим фундамент веб-автоматизации – научимся понимать, как устроен веб изнутри, и освоим базовые инструменты управления браузером.

Мы начнем с понимания того, чем веб-автоматизация принципиально отличается от автоматизации десктопных приложений. Это важно, потому что подходы здесь совершенно разные: веб работает через браузер как единую среду исполнения, имеет клиент-серверную архитектуру и требует учета сетевых задержек и асинхронности.

Далее мы разберем, как устроен веб с точки зрения клиент-серверного взаимодействия – что происходит между браузером и сервером, чем статический кон-

тент отличается от динамического, и почему это знание критически важно для правильного выбора инструментов автоматизации.

Особое внимание мы уделим DOM – объектной модели документа. Это центральное понятие для любого, кто занимается веб-автоматизацией. Мы поймем, как HTML превращается в дерево элементов, по которому робот может навигиро-вать, находить нужные узлы и взаимодействовать с ними. Без понимания DOM невозможно эффективно искать элементы и строить надежные скрипты.

Затем мы познакомимся с инструментами веб-разработчика, которые станут нашими главными помощниками в работе. Мы научимся открывать консоль браузера, инспектировать элементы, копировать селекторы и анализировать сетевые запросы – все то, что позволяет понять структуру любой страницы и написать правильный код для автоматизации.

После теоретической базы мы перейдем к практике и изучим базовые блоки веб-автоматизации в среде Puzzle RPA. Мы разберем, как создавать сессию браузера, открывать страницы, находить элементы, кликать по ним, вводить текст, работать с ожиданиями, переключаться между вкладками и фреймами, выполнять JavaScript, а также закрывать браузер корректно. Это набор базовых действий, из которых складывается любая автоматизация.

В завершение практической работы мы подведем итоги, закрепим изученный материал и выполним практическое задание, состоящее из двух частей, которое позволит нам убедиться, что фундамент для дальнейшего освоения веб-автоматизации надежно построен.

Веб-автоматизация: отличия от десктопной

Понятие веб-автоматизации

Веб-автоматизация – это процесс программного управления веб-браузером для выполнения действий, которые обычно совершает человек: переход по ссылкам, заполнение форм, клики по кнопкам, сбор данных и т.д.

Робот не просто скачивает HTML-код страницы (как это делает обычный парсер), а эмулирует поведение реального пользователя:

- Открывает браузер
- Ждет загрузки страницы
- Кликает по элементам
- Скроллит страницу
- Вводит текст с клавиатуры

Отличие веб-автоматизации от десктопной автоматизации. Это принципиально разные подходы, хотя цель (автоматизация действий) может казаться похожей.

Характеристика	Веб-автоматизация	Десктопная автоматизация
Среда	Браузер (Chrome, Firefox, Safari)	Операционная система (Windows, macOS, Linux)
Единообразие	Высокое – браузеры следуют веб-стандартам	Низкое – разные ОС имеют разные GUI-библиотеки
Кроссплатформенность	Один код работает на всех	Часто нужна адаптация под

	ОС (через браузер)	каждую ОС
Инструменты	Selenium, Playwright, Puppeteer	AutoIt, PyAutoGUI, Win32 API, UI Automation

Веб-автоматизация проще с точки зрения кроссплатформенности – вы пишете код один раз, и он будет работать на Windows, Linux и macOS, потому что браузер везде одинаково интерпретирует HTML, CSS и JavaScript.

Десктопная автоматизация сложнее: кнопка "OK" в Windows-приложении и в macOS-приложении – это совершенно разные объекты с разными API для взаимодействия.

Веб-приложения имеют клиент-серверную архитектуру:

- Клиент (браузер) отвечает за отображение интерфейса
- Сервер хранит данные и бизнес-логику
- Между ними постоянно происходит обмен данными по сети
- Десктоп-приложения обычно работают локально:
- Весь код и данные находятся на компьютере пользователя
- Сеть может не использоваться вообще или использоваться только для отдельных операций

Последствия для автоматизации:

- В веб-автоматизации нужно учитывать задержки сети (страница может грузиться долго)
- В веб-автоматизации контент может динамически подгружаться (AJAX, WebSockets)
- В десктоп-автоматизации интерфейс обычно статичен после запуска

Технологии рендеринга:

	Веб	Десктоп
Интерфейс	HTML + CSS + JavaScript	Нативные элементы ОС (Win32, Cocoa, Qt, GTK)
Динамичность	Высокая – DOM можно менять в реальном времени	Средняя – интерфейс обычно фиксирован
Доступ к элементам	Через DOM-модель (селекторы, XPath)	Через API оконной системы (поиск по заголовкам, классам)
Скорость изменений	Интерфейс может меняться каждый день	Интерфейс меняется реже, с выходом новых версий

Веб-элементы легко находить через id, class, XPath – это стандартизировано. В десктопных приложениях часто приходится использовать "костыли": поиск по координатам экрана, распознавание изображений, что ненадежно.

Ключевые сценарии Веб-автоматизации

Веб-автоматизация становится незаменимым инструментом в тех ситуациях, когда обычные подходы – ручной труд или простые парсеры – оказываются неэффективными, невозможными или экономически нецелесообразными. Рассмотрим ключевые сценарии.

1. Парсинг динамических сайтов

Современные веб-сайты все чаще строятся как одностраничные приложения, где весь контент подгружается асинхронно через JavaScript. Это создает серьез-

ную проблему для традиционных парсеров, которые просто получают HTML-код и пытаются извлечь из него данные. Когда вы заходите на такой сайт, сервер от-дает только пустой скелет страницы с минимальной разметкой, а все содержимое – тексты, изображения, списки товаров – подтягивается позже с помощью дополнительных запросов к API, которые выполняются уже в браузере.

2. Автоматизация рутинных действий

Веб-автоматизация необходима для выполнения однообразных и повторяющихся задач, которые вручную отнимают много времени, подвержены ошибкам из-за человеческого фактора и сложно масштабируются – один физически не способен сделать тысячу однотипных операций за день, тогда как робот справляется с этим за минуты без потери качества.

3. Тестирование веб-приложений

Современные веб-приложения обновляются часто – иногда по несколько раз в день. При каждом изменении кода существует риск, что что-то сломается в уже работающем функционале. Проверять вручную все сценарии использования каждый раз, когда разработчики выкатывают новую версию, невозможно – это заняло бы дни. Именно здесь на помощь приходит автоматизированное тестирование.

4. Сбор данных для аналитики и машинного обучения

В эпоху больших данных компании нуждаются в сборе информации о конкурентах, маркетологам требуются данные о позициях в поисковой выдаче и тональности упоминаний, а разработчикам нейросетей необходимы размеченные данные для обучения. Поскольку эти данные часто доступны только через веб-интерфейс, получить их можно исключительно с помощью веб-автоматизации, которая позволяет роботам обходить сотни страниц, собирать цены, отзывы, изображения, анализировать мета-теги и ссылочную массу для SEO, превращая неструктурированную информацию в ценную аналитику.

5. Интеграция систем без API

В корпоративной среде часто возникает ситуация, когда две системы нужно интегрировать, но ни одна из них не предоставляет открытого API для программного взаимодействия. Это может быть старая учетная система, которая работает только через веб-интерфейс, или внешний сервис поставщика, который не заинтересован в автоматизации. В таких случаях веб-автоматизация становится мостом между системами.

Робот может выступать в роли "клея": он забирает данные из одной системы, заходит в другую, вводит эти данные в форму, нажимает кнопки отправки и забирает результат обратно. Это позволяет автоматизировать процессы, которые раньше требовали участия человека-оператора, переносящего данные из одной системы в другую вручную.

6. Мониторинг доступности и восстановление

Критически важные сервисы требуют постоянного контроля доступности. Если сайт перестает работать, важно узнать об этом как можно быстрее, чтобы принять меры. Веб-автоматизация позволяет настроить мониторинг, который регулярно заходит на сайт, выполняет ключевые сценарии – авторизацию, поиск,

оформление заказа – и проверяет, что все работает корректно. Если на каком-то этапе происходит ошибка, система оповещает администраторов.

7. Создание скриншотов и визуального контроля

В некоторых задачах важно не просто собрать данные, но и зафиксировать визуальное состояние страницы. Это может быть необходимо для контроля дизайна – проверки, что верстка не сломалась после обновлений, или для создания архивных копий важных веб-страниц. Веб-автоматизация позволяет делать скриншоты страниц в автоматическом режиме, в том числе прокручивая страницу и склеивая скриншот целиком, если она длиннее одного экрана.

Веб-автоматизация необходима в ситуациях, когда обычные методы сбора данных или взаимодействия с веб-приложениями оказываются недостаточными. Ключевые признаки того, что вам нужна веб-автоматизация – это наличие динамического контента, который загружается через JavaScript, необходимость выполнения однотипных рутинных действий в больших объемах, потребность в регулярном мониторинге, который человек не может выполнять непрерывно, отсутствие API для интеграции систем, а также задачи тестирования, требующие повторяемых и масштабируемых проверок.

Веб-автоматизация превращает браузер из инструмента для человека в инструмент для программ, позволяя переносить на плечи роботов те задачи, которые раньше требовали утомительного и подверженного ошибкам ручного труда.

Как устроен веб: клиент-серверная архитектура

В мире веб-технологий клиентом выступает браузер. Это программа, которая живет на вашем компьютере, планшете или телефоне. Браузер умеет делать несколько важных вещей:

- Отправлять HTTP-запросы – он знает, как правильно попросить сервер прислать страницу
- Принимать и интерпретировать ответы – он понимает HTML, CSS, JavaScript
- Отображать интерфейс – он рисует страницу такой, какой ее задумал разработчик
- Хранить данные – браузер помнит ваши пароли, cookies, историю посещений
- Выполнять код – он запускает JavaScript-программы, которые оживляют страницу

Важно понимать: браузер – это не просто "окно в интернет". Это сложная среда исполнения, которая сама является программой со своими возможностями и ограничениями. Когда мы занимаемся веб-автоматизацией, мы фактически управляем браузером, заставляя его делать то, что обычно делает человек.

Сервер – это удаленный компьютер, который постоянно работает и ждет запросов. Он может находиться где угодно – в соседней комнате, в другом городе или на другом континенте. Сервер умеет:

- Принимать HTTP-запросы – он слушает определенный порт (обычно 80 для HTTP и 443 для HTTPS)

- Обрабатывать их – он запускает программу, которая решает, что делать с запросом
- Отдавать ответы – он формирует HTML, JSON, картинки или другие данные и отправляет обратно
- Хранить данные – у сервера есть доступ к базам данных, файлам, другим сервисам

Сервер может быть простым – например, отдавать статические HTML-файлы, которые лежат в папке. А может быть сложным – запускать бизнес-логику, обращаться к базам данных, вызывать другие сервисы, генерировать ответы на лету.

Главное отличие: клиент всегда инициирует общение. Он первым отправляет запрос. Сервер никогда не начинает диалог сам – он только отвечает. Это ключевое свойство веб-архитектуры, которое определяет все остальное. Браузер не может сказать "элемент появился на странице" – он может только спросить "дай мне страницу", а потом разбираться с тем, что пришло.

Чтобы клиент и сервер могли понимать друг друга, им нужен общий язык. В вебе этот язык называется HTTP – протокол передачи гипертекста.

Когда вы вводите адрес сайта и нажимаете Enter, ваш браузер отправляет серверу запрос, в котором указывает, что он хочет получить главную страницу, сообщает, какой браузер у него установлен, какие форматы он понимает и на каком языке хочет получить ответ.

Сервер, получив этот запрос, отправляет ответ. В ответе он сообщает статус – все ли прошло успешно или произошла ошибка, указывает тип содержимого – это HTML, картинка или JSON, передает служебные заголовки с дополнительной информацией и, самое главное, отправляет тело ответа – тот самый HTML-код, который браузер превратит в видимую страницу.

Сервер всегда отвечает с кодом статуса, который говорит, как прошла обработка запроса. Коды, начинающиеся на двойку, означают успех – страница найдена и отправлена. Коды на тройку говорят о перенаправлении – например, страница перемещена на новый адрес. Коды на четверку сигнализируют об ошибке клиента – страница не найдена, доступ запрещен или запросов отправлено слишком много. Коды на пятерку говорят об ошибке сервера – что-то сломалось на той стороне, и нужно повторить попытку позже.

Для веб-автоматизации эти коды очень важны. Если робот получает ошибку, что страница не найдена, значит, ссылка ведет в никуда. Если приходит сообщение о слишком большом количестве запросов, значит, нас заблокировали за активность, и нужно делать паузу. Если сервер упал с внутренней ошибкой, нужно подождать и попробовать снова.

Вся сложность веб-автоматизации заключается в том, что сайты бывают двух принципиально разных типов. И то, что работает для одного, может полностью провалиться на другом.

Статический сайт – это самый простой вид веб-приложений. Сервер хранит на своем диске готовые HTML-файлы. Когда приходит запрос, сервер просто

находит нужный файл и отправляет его браузеру. Никакой дополнительной обработки не происходит. Файл отправляется таким, какой он есть.

Такие сайты загружаются быстро, их легко хостить, они надежны, потому что не требуют сложной логики на сервере. Но у них есть серьезный недостаток – чтобы изменить хоть одну букву на сайте, нужно редактировать HTML-файл и перезагружать его на сервере.

Для автоматизации статические сайты – это самое простое, что может быть. Все данные уже есть в HTML, который приходит с сервера. Роботу достаточно получить этот HTML и извлечь нужную информацию. Никаких ожиданий, никакой магии.

Динамические сайты работают иначе. На сервере нет готовых HTML-файлов. Вместо этого есть данные в базе данных и шаблоны, по которым эти данные нужно отобразить. Когда приходит запрос, сервер запускает программу, которая достает из базы данных актуальную информацию, вставляет ее в шаблон, формиру-ет HTML-код и только потом отправляет его браузеру.

Каждый раз, когда вы открываете страницу интернет-магазина, сервер проверяет актуальные цены, наличие товаров, персональные скидки для вашего аккаунта и только потом собирает страницу. Поэтому разные люди видят разные версии одной и той же страницы.

Для автоматизации динамические сайты уже сложнее, но все еще предсказуемы. HTML формируется на сервере и приходит целиком, сразу со всеми данными. Робот по-прежнему может получить его и извлечь информацию. Разница лишь в том, что этот HTML каждый раз может немного отличаться.

Самый сложный случай: одностраничные приложения. А теперь представьте себе сайт, который вообще не отдает HTML с данными. Вместо этого он отдает пустую оболочку, а все содержимое подгружает отдельно, уже после загрузки страницы, с помощью JavaScript. Это и есть одностраничные приложения – современный подход к созданию веб-сайтов.

Когда вы заходите на такой сайт, сервер отправляет вам минимальный HTML – буквально пустой div, в который потом будет вставлен контент. Вместе с этим HTML приходит огромный JavaScript-файл – программа, которая будет управлять страницей.

Браузер загружает эту программу, запускает ее. JavaScript-код, работающий уже в вашем браузере, начинает запрашивать с сервера данные – список товаров, цены, изображения. Получив эти данные, он вставляет их в те самые пустые div, создавая страницу прямо на вашем компьютере.

Когда вы кликаете по ссылке, страница не перезагружается. Вместо этого JavaScript-код запрашивает новые данные, чистит экран и рисует заново. Все происходит без участия сервера в рендеринге страниц.

Если вы попытаетесь спарсить такой сайт с помощью обычного HTTP-запроса, вы получите только пустую оболочку. Никаких данных там не будет, по-тому что данные подгружаются JavaScript-кодом уже после загрузки. Чтобы по-лучить данные, вам нужно не просто отправить запрос, а запустить браузер, до-

ждаться выполнения JavaScript-кода, дождаться, пока он сходит за данными на сервер, и только потом смотреть на то, что получилось.

Это ключевое отличие, которое определяет, когда нужна веб-автоматизация с реальным браузером, а когда можно обойтись простыми запросами.

Понимание клиент-серверной архитектуры дает роботу суперсилу. Он начинает видеть за кулисами то, что обычный пользователь не замечает.

Когда робот знает, что сайт динамический, он понимает, что нужно ждать, пока сервер соберет страницу, прежде чем искать на ней данные. Когда робот понимает, что перед ним одностраничное приложение, он знает, что нужно запус-кать полноценный браузер, а не просто отправлять HTTP-запросы.

Когда робот видит код статуса 429, он понимает, что его слишком быстро стучатся в дверь, и нужно замедлиться. Когда он получает 500, он знает, что проблема не в нем, а на сервере, и нужно просто повторить попытку позже.

Более того, понимание архитектуры позволяет роботу быть умным. Вместо того чтобы проосто открывать страницу и ждать фиксированное время, он может следить за сетевыми запросами в инструментах разработчика и ждать именно тот запрос, который приносит нужные данные. Это делает автоматизацию быстрее и надежнее.

Клиент-серверная архитектура – это фундамент, на котором строится весь веб. Браузер-клиент всегда спрашивает, сервер всегда отвечает. Они общаются на языке HTTP, обмениваясь запросами и ответами с кодами статуса.

Сайты бывают статическими, где страницы готовы заранее, динамическими, где они собираются в момент запроса, и одностраничными приложениями, где вся логика переехала в браузер, а сервер отдает только данные.

Понимание этих механизмов позволяет выбирать правильные инструменты, настраивать корректные ожидания и писать скрипты, которые работают стабильно в самых разных условиях. Без этого фундамента любая автоматизация превращается в угадывание, когда и какие данные появятся на странице.

DOM (Document Object Model) – модель документа

Основные понятия DOM

DOM – это программный интерфейс к веб-странице. Он представляет документ в виде дерева узлов, где каждый узел – это часть страницы: элемент, атри-бут, текст, комментарий.

Когда браузер загружает HTML-код, он делает примерно следующее. Сначала он читает текст HTML от начала до конца. Встречая открывающий тег, он создает новый узел в дереве. Встречая закрывающий тег, он понимает, что узел закончился, и возвращается на уровень выше. Текст между тегами становится текстовым узлом, вложенным в элемент. В результате получается иерархическая структура, где у каждого узла есть родитель, могут быть дети и соседи.

Почему это так важно? Потому что DOM – это единственный способ взаимодействия с веб-страницей. Когда мы говорим "найти элемент с таким идентификатором", мы обращаемся к DOM. Когда мы говорим "прочитать текст кнопки", мы читаем свойство узла в DOM. Когда мы говорим "кликнуть по элементу", мы пе-

редаем команду браузеру, который находит соответствующий узел в DOM и генерирует событие.

Без DOM веб-страница была бы просто статичным документом, который нельзя изменить после загрузки. Благодаря DOM, мы можем добавлять новые элементы, удалять старые, менять стили, реагировать на действия пользователя. И именно благодаря DOM мы можем управлять страницей программно.

Главная метафора для понимания DOM – это дерево. Как у дерева есть ко-рень, от которого отходят ветви, которые ветвятся на более мелкие веточки, так и у HTML-документа есть корневой узел, внутри которого вложены все остальные.

Самым верхним узлом в DOM всегда является узел `document`. Это точка входа в документ. От него уже идет ветвь к корневому HTML-элементу. Внутри HTML-элемента находятся два основных узла – `head` и `body`. `Head` содержит метаинформацию о странице – заголовок, подключенные стили, скрипты. `Body` содержит все, что пользователь видит на экране – заголовки, абзацы, картинки, кнопки, формы.

Каждый элемент может содержать внутри себя другие элементы. Например, на странице интернет-магазина есть блок с товарами. Внутри этого блока – карточки товаров. Внутри каждой карточки – название, цена, кнопка "Купить". Это и есть иерархия. В терминах DOM, блок товаров – это родительский узел для карточек. Карточки – дочерние узлы для блока и одновременно родительские для названия, цены и кнопки.

Такая иерархия позволяет нам искать элементы не только по всей странице целиком, но и в контексте. Мы можем сначала найти блок товаров, а потом уже внутри него искать карточки. Это делает автоматизацию более надежной – если на странице есть несколько похожих блоков, мы не перепутаем их.

В DOM существуют разные типы узлов, хотя чаще всего мы работаем с узлами-элементами. Узел-элемент соответствует HTML-тегу – `div`, `p`, `button`, `a`. Узел-текст содержит текст внутри элемента. Узел-атрибут представляет собой атрибут элемента – `id`, `class`, `href`. Узел-комментарий – это HTML-комментарии, которые не видны на странице, но присутствуют в коде.

Важно понимать, что DOM – это не то же самое, что HTML, который пришел с сервера. Между ними есть принципиальная разница.

HTML – это текст. Это исходный код, который разработчик написал или который сервер сгенерировал. DOM – это объектная структура, которая создается браузером на основе этого текста.

Когда браузер получает HTML, он начинает его парсить – читать символ за символом. Встречая открывающий тег, он создает объект-элемент. Встречая текст, он создает текстовый узел. Встречая закрывающий тег, он закрывает теку-щий элемент и возвращается к родителю. В результате получается структура, ко-торая идеально отражает вложенность, описанную в HTML.

Но на этом процесс не заканчивается. После того как базовое дерево построено, браузер начинает выполнять JavaScript-код, который может менять это дерево. Скрипты могут добавлять новые элементы, удалять существующие, менять

атрибуты, перемещать узлы. DOM начинает жить своей жизнью, отличаясь от исходного HTML.

Именно поэтому мы говорим, что HTML – это то, что сервер отправил, а DOM – это то, что браузер видит в данный момент. Для веб-автоматизации важно второе. Робот взаимодействует не с исходным кодом, а с актуальной DOM-структурой, которая может быть существенно изменена JavaScript-кодом.

Навигация по DOM: как найти нужный элемент

Представьте, что вы стоите перед огромным деревом с тысячами веток и листьев, и вам нужно найти один конкретный лист. Как это сделать? Можно искать по уникальному признаку – например, листок с биркой, на которой написано его имя. Можно идти от ствола, сворачивая на нужные ветки. Можно описать путь: третья ветка от ствола, на ней вторая веточка, на ней пятый листок.

В DOM навигация работает по тем же принципам.

- Уникальные идентификаторы

Самый надежный способ найти элемент – это использовать его уникальный идентификатор. Разработчики часто добавляют элементам атрибут `id`, который должен быть уникальным в пределах всей страницы. Если у элемента есть id, найти его можно мгновенно и гарантированно. Это как найти листок с биркой.

- Поиск по классам и тегам

Если у элемента нет id, можно искать по классу или тегу. Классы обычно объединяют группы похожих элементов – все кнопки на странице могут иметь класс "`button`", все карточки товаров – класс "`product-card`". Поиск по классу вернет все элементы с таким классом. Из них уже можно выбрать нужный по порядку или по дополнительным признакам.

- Относительная навигация

Можно искать элемент не по всей странице, а в контексте уже найденного. Например, сначала найти блок товаров, а потом внутри него искать карточки. Это называется относительным поиском. Он делает автоматизацию более устойчивой – если на странице появится другой похожий блок, робот не перепутает их, пото-му что ищет строго внутри уже известного контейнера.

- Путь к элементу

Самый гибкий, но и самый хрупкий способ – это описывать путь к элементу от корня дерева. Например, спуститься к `body`, затем к блоку с определенным классом, затем к его первому дочернему элементу, затем к кнопке внутри. Такой путь называется `XPath`. Он очень мощный – позволяет искать по тексту, по части атрибута, по положению среди соседей. Но он же и самый ненадежный – малейшее изменение структуры страницы сломает такой путь.

DOM – это не статичная структура. Она постоянно меняется в течение жизни страницы. Понимание этого критически важно для веб-автоматизации.

Когда страница только загружается, DOM постепенно строится. Сначала появляются одни элементы, потом, когда браузер доходит до соответствующих мест

в HTML, появляются другие. Это значит, что в момент, когда робот начинает искать элемент, его может еще не быть в DOM.

После загрузки страницы в дело вступает JavaScript. Скрипты могут добавлять новые элементы, удалять старые, менять атрибуты. Например, при скролле страницы могут подгружаться новые карточки товаров. При нажатии на кнопку может появляться всплывающее окно. При успешной отправке формы может исчезать форма и появляться сообщение об успехе.

Элементы могут быть временно скрыты и потом показаны. Могут менять свое состояние – становиться доступными для клика или, наоборот, блокировать-ся.

Для веб-автоматизации это означает, что нельзя просто написать "найти элемент и кликнуть". Нужно учитывать, что элемент может появиться не сразу, мо-жет стать кликабельным не сразу, может исчезнуть после выполнения действия. Нужно уметь ждать нужного состояния, проверять, что элемент готов к взаимо-действию, и корректно обрабатывать ситуации, когда элемент исчезает.

DOM дает роботу суперспособности. Понимая структуру дерева и умея с ней работать, мы можем делать очень многое.

- Поиск и извлечение данных

Самое базовое – это находить элементы и читать их содержимое. Можно прочитать текст кнопки, чтобы убедиться, что она говорит "Отправить", а не "Загружается". Можно взять значение поля ввода, чтобы проверить, правильно ли заполнилась форма. Можно извлечь ссылку из атрибута href, чтобы перейти по ней дальше. Можно собрать все цены на странице, взяв текст из всех элементов с классом "price". DOM позволяет проверять, в каком состоянии находится элемент. Виден ли он на странице или скрыт? Доступен ли для взаимодействия или заблокирован? Выбран ли чекбокс или нет? Эта информация критически важна для принятия решений в автоматизации.

- Взаимодействие

DOM – это интерфейс для взаимодействия. Клик по элементу – это команда, которая отправляется через DOM. Ввод текста в поле – тоже. Наведение мыши, скроллинг к элементу, перетаскивание – все это операции над DOM-узлами.

- Ожидание изменений

Самый важный навык в веб-автоматизации – это умение ждать нужных изменений в DOM. Не просто ждать фиксированное время, а ждать именно того события, которое говорит, что страница готова. Появился нужный элемент – можно продолжать. Исчез лоадер – значит, данные загрузились. Текст на кнопке изменился с "Отправить" на "Отправлено" – значит, действие выполнено.

Для работы с DOM у нас есть мощный инструмент – встроенные средства разработчика в браузере. Они позволяют увидеть DOM таким, какой он есть в данный момент, и исследовать его.

Открыв инструменты разработчика и перейдя на вкладку Elements, мы видим дерево DOM в том виде, в каком его видит браузер. Мы можем наводить на элементы и видеть, как они подсвечиваются на странице. Можем искать по тексту, по

селекторам. Можем копировать точный путь к элементу, чтобы использовать его в скрипте.

Это наша рабочая лаборатория. Любой скрипт начинается с того, что мы открываем сайт, исследуем его DOM с помощью инструментов разработчика, находим надежные селекторы и только потом пишем код. Без этого навыка веб-автоматизация превращается в угадывание.

DOM – это объектная модель документа, то представление веб-страницы, с которым работает браузер и с которым работаем мы, когда пишем скрипты автоматизации. Это дерево узлов, где каждый узел соответствует элементу страницы, атрибуту или тексту.

Понимание DOM дает нам возможность находить элементы разными способами – по идентификаторам, классам, через относительный поиск или по слож-ным путям. Мы можем читать свойства элементов, проверять их состояние и взаимодействовать с ними.

Знание жизненного цикла DOM позволяет нам правильно синхронизироваться со страницей – ждать появления элементов, исчезновения лоадеров, изменения состояний. А инструменты разработчика дают нам возможность исследовать DOM и подбирать надежные селекторы.

Для веб-автоматизации DOM – это не просто теория. Это основной инструмент работы. Каждый клик, каждое прочитанное значение, каждое ожидание – это операция над DOM. Без глубокого понимания того, как устроена эта модель, невозможно писать надежные и эффективные скрипты.

Инструменты веб-разработчика для робота

Инструменты разработчика – это ваше окно во внутреннее устройство страницы. Они показывают DOM таким, какой он есть на самом деле, позволяют экспериментировать с селекторами, анализировать сетевые запросы и понимать, как страница живет.

Открываются эти инструменты одной клавишей – F12. И для специалиста по веб-автоматизации эта клавиша становится такой же важной, как для программиста – клавиша запуска кода.

- Вкладка *Elements* – это ваша главная рабочая лошадка. Она показывает DOM-дерево страницы в том виде, в каком его видит браузер в данный мо-мент. Это не исходный HTML, который пришел с сервера, а актуальная структура, возможно, уже измененная JavaScript-кодом.
- Когда вы открываете вкладку *Elements*, вы видите иерархический список всех элементов на странице. Вы можете раскрывать и сворачивать узлы, видеть, какие элементы вложены в какие. При наведении на элемент в дереве он подсвечивается на странице – это помогает понять, какой именно элемент вы сейчас смотрите. При наведении на элемент на странице он подсвечивается в дереве – так можно быстро найти, где в структуре находится интересующий вас блок.

- Самая полезная функция вкладки *Elements* для веб-автоматизации – это возможность скопировать точный путь к элементу. Когда вы нашли нужный элемент в дереве, вы можете кликнуть по нему правой кнопкой мыши и увидеть меню с опциями копирования. Опция "*Copy selector*" скопирует CSS-селектор, который ведет к этому элементу. Это строка, которую вы можете использовать в своем коде для поиска элемента. Опция "*Copy XPath*" скопирует XPath-выражение – более гибкий, но и более сложный способ описания пути к элементу.
- Важно понимать, что вкладка *Elements* показывает DOM в реальном времени. Если JavaScript добавит новый элемент на страницу, вы увидите, как он появляется в дереве. Если скрипт изменит атрибут, вы увидите, как изменилось значение. Это позволяет отслеживать, что происходит со страницей во время работы, и понимать, в какой момент появляются нужные данные.
- Вкладка *Console* – это ваша песочница. Здесь вы можете писать и выполнять JavaScript-код прямо в браузере, экспериментировать с селекторами, проверять, находятся ли нужные элементы, и смотреть, какие данные можно извлечь.
- Прежде чем писать код автоматизации, вы можете проверить, правильно ли работает ваш селектор, прямо в консоли. Введите команду, которая ищет элемент по тому же селектору, который вы собираетесь использовать. Если команда вернула элемент – значит, селектор работает. Если вернула null или пустой список – значит, селектор составлен неверно или элемента еще нет на странице.
- Консоль позволяет экспериментировать мгновенно. Вы можете попробовать разные селекторы, посмотреть, какой из них находит нужный элемент, какой находит слишком много лишнего, какой вообще ничего не находит. Это быстрее, чем перезапускать скрипт каждый раз, когда нужно проверить изменение.
- Консоль также позволяет заглянуть в данные, которые хранятся в JavaScript-объектах на странице. Многие современные сайты держат данные в глобальных переменных. Введя имя такой переменной в консоли, вы можете увидеть весь массив данных, который сайт загрузил для отображения. Иногда это позволяет извлекать данные проще и быстрее, чем парсить DOM.
- Вы можете использовать консоль, чтобы имитировать действия, которые будет выполнять ваш робот. Найти элемент и кликнуть по нему, заполнить поле ввода, прочитать текст – все это можно сделать вручную через консоль, чтобы убедиться, что сценарий работает, прежде чем писать код.
- Вкладка *Network* – это окно в коммуникации между браузером и сервером. Она показывает все HTTP-запросы, которые браузер отправляет, и все отве-ты, которые получает. Это бесценный инструмент для понимания того, как на самом деле работает сайт.

- Когда вы открываете вкладку *Network* и перезагружаете страницу, вы видите список всех запросов. Каждый запрос – это одна операция: загрузка *HTML*-документа, получение *CSS*-файла, загрузка картинки, отправка данных формы, запрос к *API* за данными.
- Для каждого запроса вы видите его тип – *HTML, CSS, JavaScript, JSON*, изображение. Видите код ответа – успешно ли прошел запрос или произошла ошибка. Видите время выполнения – сколько заняла загрузка. Видите размер – сколько данных было передано.
- Одна из самых важных возможностей вкладки *Network* – это поиск запросов, которые возвращают данные. На многих современных сайтах данные не вшиты в *HTML*, а подгружаются отдельно через *API*-запросы к серверу. Эти запросы часто возвращают данные в формате *JSON* – структурированном виде, который гораздо проще парсить, чем *HTML*.

Надежность селектора

Найти селектор, который будет работать сегодня, – просто. Найти селектор, который будет работать через месяц, когда разработчики изменят верстку, – вот настоящее искусство. Надежность селектора определяется тем, насколько он устойчив к изменениям страницы.

Самый надежный способ найти элемент – это использовать его уникальный идентификатор, атрибут id. Разработчики обычно дают элементам `id`, когда хотят иметь к ним быстрый и гарантированный доступ. `id` должен быть уникальным на всей странице. Если у элемента есть `id`, используйте его. Это самый стабильный селектор, потому что `id` редко меняют без необходимости.

Если `id` нет, но есть атрибут `name`, он тоже подходит. `name` часто используется для полей форм и тоже достаточно стабилен. Он может повторяться на странице, но обычно в контексте формы этого достаточно.

Следующий по надежности – это специализированные атрибуты, которые разработчики добавляют специально для тестирования. Часто можно встретить атрибуты вроде `data-testid`, `data-automation-id` или подобные. Они создаются именно для того, чтобы автоматизация могла надежно находить элементы, и никогда не меняются при изменении дизайна.

CSS-классы уже менее надежны. Классы в первую очередь предназначены для стилизации, и разработчики могут менять их, когда переделывают дизайн. То, что сегодня называется `button-primary`, завтра может стать `btn-primary` или просто `btn`. Использовать классы можно, но нужно понимать риски.

XPath по тексту – это гибкий, но хрупкий подход. Он позволяет искать кнопку по надписи на ней: "найти кнопку, на которой написано Отправить". Это удобно, потому что текст кнопки обычно не меняется. Но если на странице есть две кнопки с одинаковым текстом, *XPath* найдет обе, и нужно будет уточнять поиск. Кроме того, текст может измениться – например, при локализации сайта на дру-гой язык.

Самое опасное в автоматизации – это автоматически сгенерированные идентификаторы. Многие фреймворки генерируют `id` динамически, например, `id="j_1726354"` или `id="ember123"`. При перезагрузке страницы такой `id` может измениться. Если вы используете такой `id` в селекторе, ваш скрипт сломается при следующем запуске. Увидели такой `id` – ищите другой способ найти элемент.

Также стоит избегать слишком длинных и сложных путей, которые копируются из инструментов разработчика. Путь вида `"html body div.container div.content div.sidebar div.widget div.widget-body ul li:nth-child(3) a"` – это бомба замедленного действия. Любое изменение структуры – добавление нового блока, изменение порядка элементов – сломает такой селектор. Лучше найти что-то более уникальное на этом пути: уникальный класс, атрибут, или подняться выше и искать в контексте.

Когда вы приступаете к автоматизации нового сайта, у вас есть четкая последовательность действий.

Сначала вы открываете сайт и нажимаете F12. Переходите на вкладку *Elements* и находите нужный элемент. Смотрите, есть ли у него `id`. Если есть и `id` не выглядит как сгенерированный – используйте его.

Если `id` нет, смотрите на атрибуты. Есть ли `data-testid`? Есть ли `name`? Есть ли что-то уникальное среди классов? Один из классов может быть специфичным именно для этого типа элементов.

Если ничего уникального нет, вы смотрите на контекст. Может быть, у родительского элемента есть уникальный `id` или класс. Тогда вы можете найти родителя, а потом внутри него искать нужный элемент по типу или по классу. Это сужает область поиска и делает селектор более надежным.

Если и это не помогает, вы смотрите на текст элемента. Искать кнопку по надписи на ней – вполне приемлемый вариант, если текст уникален и стабилен.

И только в самом крайнем случае вы используете сложный *XPath* с порядковыми номерами или полный путь от корня.

После того как вы подобрали селектор, вы обязательно проверяете его в консоли, введя команду поиска. Если элемент нашелся – селектор работает. Если нашелся не тот элемент или нашелся лишний – нужно уточнить.

Инструменты веб-разработчика – это главный инструмент специалиста по веб-автоматизации. Вкладка *Elements* позволяет видеть *DOM* и копировать селек-торы. Вкладка *Console* дает возможность экспериментировать с поиском и прове-рять селекторы в реальном времени. Вкладка *Network* раскрывает закулисную жизнь веб-приложения – какие запросы отправляются, какие данные передаются, откуда берется информация.

Выбор надежного селектора – это баланс между точностью и устойчивостью к изменениям. Приоритет отдается стабильным идентификаторам – `id`, `name`, `data`-атрибутам для тестирования. CSS-классы можно использовать с осторожностью. Сложные пути и автоматически сгенерированные `id` лучше избегать.

Умение работать с этими инструментами и выбирать правильные селекторы превращает веб-автоматизацию из угадывания в точную инженерную работу. Вы перестаете гадать, какой селектор сработает, и начинаете видеть структуру страницы, понимать, как она построена, и точно знать, где и как искать нужные элементы.

Базовые блоки веб-автоматизации в Puzzle RPA

Мы уже изучили теорию: узнали, как устроен веб, что такое DOM, как работают инструменты разработчика. Теперь эти знания помогут нам правильно настраивать блоки. Когда вы будете выбирать селектор для блока «Найти элемент», вы будете опираться на понимание структуры DOM. Когда будете настраивать ожидание, вы будете помнить о динамической загрузке страниц.

В этой части практической работы мы познакомимся с основными группами базовых блоков, которые используются в веб-автоматизации. Базовые блоки – это фундамент, с которого начинается любой сценарий взаимодействия с веб-страницей. Они отвечают за:

Навигацию – открытие страниц, управление вкладками, переходы между страницами

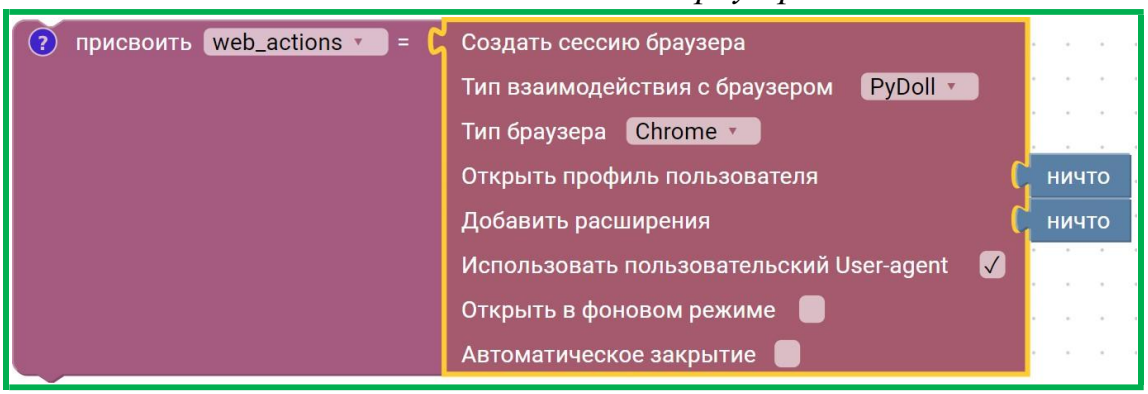
Поиск элементов – идентификация элементов на странице с помощью различных селекторов

Взаимодействие – клики, ввод текста, наведение мыши, работа с формами

Мы разберем, за что отвечает каждая группа блоков, какие блоки в нее входят и в каких сценариях они применяются. А затем, в практическом задании, вы соберете из этих блоков полноценного робота, который сможет самостоятельно собирать данные с реального сайта.

Рассмотрим систематически все группы базовых блоков, доступные в Puzzle RPA для веб-автоматизации, их функциональное назначение, параметры настройки и типовые сценарии применения. Это знание станет фундаментом, на котором будет строиться любой ваш проект по автоматизации веб-интерфейсов.

Блок «Создать сессию браузера»



Блок позволяет создать новый экземпляр браузера. Это отправная точка любого сценария веб-автоматизации – без активной сессии браузера невозможно взаимодействие с веб-страницами. Блок возвращает объект браузера, который затем используется во всех последующих блоках для управления браузером.

Параметры блока

- Тип взаимодействия с браузером – определяет, какое программное средство будет использоваться для автоматизации:

PyDoll – взаимодействие через протокол *DevTools* без использования WebDriver

Selenium – взаимодействие через *WebDriver*, обеспечивающий полную эмуляцию действий пользователя

- Тип браузера – определяет, какой веб-браузер будет запущен:
  - ✓ Chrome
  - ✓ Edge
  - ✓ Firefox (только при типе взаимодействия Selenium)
  - ✓ Yandex
  - ✓ Chromium-Gost
- Профиль пользователя – это папка, в которой браузер хранит все свои настройки, кэш, историю, авторизации и расширения. В параметре необходимо передать абсолютный путь к папке ресурс проекта. Робот автоматически создаст в этой папке все необходимые файлы для профиля пользователя браузера. Какой Принцип работы: Робот самостоятельно создает структуру профиля (каталоги и файлы User Data) по указанному пути; Повторное использование того же пути позволит сохранить состояние профиля (cookie, кэш, данные авторизации) между сеансами.

Ограничения использования профиля пользователя:

1. Функция доступна только для браузеров Chrome;
2. Перед запуском робота экземпляр браузера с выбранным профилем должен быть закрыт;

Добавить расширение – файл с расширением для браузера в формате .crx.

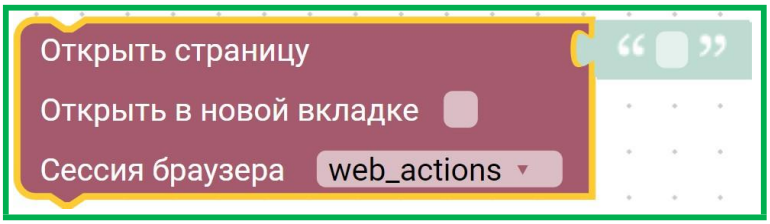
Пример: `C:\Users\Documents\expansion.crx`

Важно: начиная с версии 137, Google отключил автоматическую установку расширений из .crx. Рекомендуется использовать профиль пользователя и добавлять расширения вручную.

- Использовать пользовательский User-agent – запуск браузера с подменой идентификатора браузера.При включении снижается вероятность обнаружения автоматизированного сценария на сайтах с защитой от ботов.
- Открыть в фоновом режиме – запуск браузера в невидимом режиме.
- Автоматическое закрытие – закрытие браузера по завершении работы робота.
- Драйвер браузера – способ загрузки WebDriver (актуально для Selenium): Версия драйвера должна соответствовать версии установленного браузера.

Блок «Создать сессию браузера» является отправной точкой для любого сценария веб-автоматизации. Он предоставляет гибкие настройки для работы с различными браузерами и способами взаимодействия (PyDoll или Selenium). Возможность использования профиля пользователя позволяет сохранять состояние между сессиями, что особенно важно для сценариев с авторизацией или длительным взаимодействием с сайтом. Правильная настройка этого блока закладывает основу для стабильной и надежной работы всего робота.

Блок «Открыть страницу»



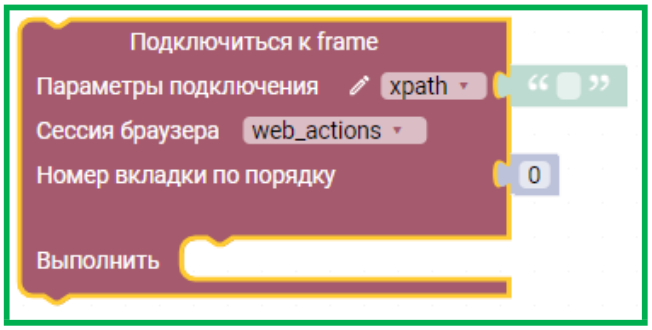
Блок открывает новую страницу в выбранной сессии браузера. Это один из ключевых блоков навигации, который позволяет переходить к нужному веб-ресурсу для дальнейшего взаимодействия.

Параметры блока

- Открыть страницу – адрес (URL) страницы, которую необходимо открыть в браузере.
- Открыть страницу в новой вкладке – определяет, следует ли открывать страницу в новой вкладке браузера.При включении страница открывается в новой вкладке, сохраняя текущую открытой. При выключении страница загружается в текущей активной вкладке.
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».

Блок «Открыть страницу» является базовым элементом навигации в веб-автоматизации. Он позволяет программно переходить на нужные веб-страницы, задавая целевой URL и управляя способом открытия (в текущей или новой вкладке). В сочетании с блоком «Создать сессию браузера» он формирует основу для любого сценария, требующего взаимодействия с веб-интерфейсами. Без этого блока невозможно приступить к работе с содержимым страницы – поиску элементов, извлечению данных или выполнению действий.

Блок «Подключиться к frame»



Блок позволяет подключиться к Frame (фрейму или iframe) на странице браузера. Фреймы представляют собой встроенные HTML-документы внутри основной страницы. Для взаимодействия с элементами, находящимися внутри фрейма, необходимо предварительно переключить контекст на этот фрейм. Блок выполняет переключение и позволяет выполнить вложенные действия внутри фрейма.

Параметры блока:

- Параметры подключения – выбор способа идентификации и поиска фрейма на странице. Доступны следующие опции:
  xpath – поиск по XPATH
  css-selector – поиск по CSS-селектору

tag_name – поиск по тегу элемента
name – поиск по атрибуту name
id – поиск по атрибуту id
class – поиск по атрибуту class

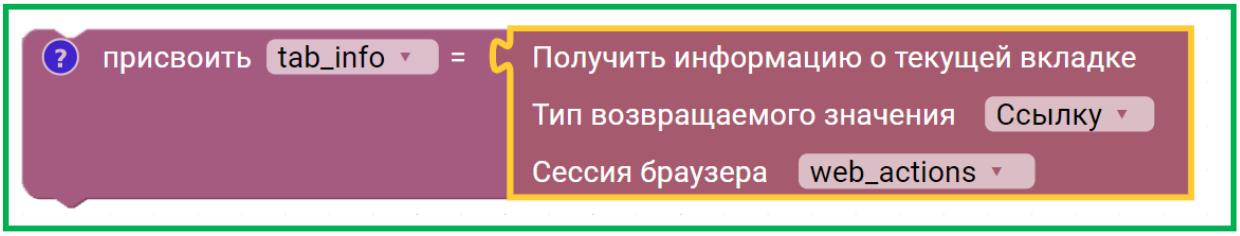
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Выполнить – разъем, внутри которого размещаются блоки, выполняемые после подключения к фрейму. Все действия внутри этого разъема будут происхо-дить в контексте выбранного фрейма.

Важно отметить, после выполнения блока «Подключиться к frame» все последующие действия (поиск элементов, клики, ввод текста и т.д.) будут выполняться внутри указанного фрейма.

Блок «Подключиться к frame» является необходимым инструментом для работы с веб-страницами, использующими iframe или фреймы. Без переключения контекста поиск элементов и взаимодействие с ними внутри фрейма будут невозможны. Блок позволяет гибко настраивать способ идентификации фрейма с помощью различных типов селекторов и обеспечивает выполнение всех вложенных действий в правильном контексте. Это особенно важно при автоматизации сложных веб-приложений, где фреймы активно используются для встраивания стороннего контента, виджетов, редакторов и форм.

Блок «Получить информацию о текущей вкладке»



Блок позволяет получить информацию о текущей вкладке браузера. Это полезно для отслеживания навигации, проверки корректности перехода по ссылке или получения заголовка страницы для дальнейшего использования в сценарии.

Параметры блока

- Тип возвращаемого значения – выбор информации, которую необходимо получить о текущей вкладке. Выпадающий список предоставляет следующие опции:
  Ссылку – получение URL-адреса текущей вкладки
  Заголовок – получение заголовка (title) текущей страницы
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».

Блок возвращает запрошенную информацию:

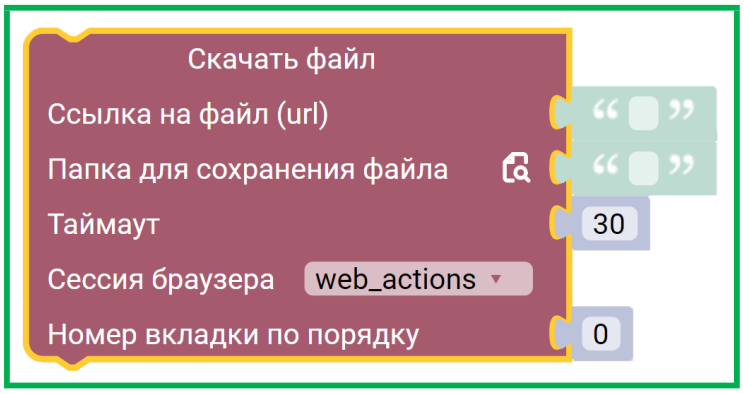
- При выборе «Ссылку» – строку с URL текущей вкладки
- При выборе «Заголовок» – строку с заголовком текущей страницы

Блок «Получить информацию о текущей вкладке» является важным элементом навигации, позволяющим отслеживать состояние браузера в процессе выполнения сценария. Он предоставляет возможность получать актуальный URL или заголовок страницы, что необходимо для:

- Проверки успешности перехода по ссылке
- Условной логики (например, если заголовок страницы не соответствует ожидаемому, выполнить альтернативные действия)
- Сбора данных о посещенных страницах для логирования или отчетности
- Динамической навигации, когда URL формируется в процессе работы робота

Блок прост в использовании и возвращает понятные строковые значения, легко интегрируемые в любые сценарии автоматизации.

Блок «Скачать файл»



Блок скачивает файл по заданной ссылке из браузера и сохраняет его на компьютере по указанному пути. Это позволяет автоматизировать процесс загрузки документов, изображений, отчетов и других файлов с веб-страниц.

Параметры блока:

- Ссылка на файл (url) – прямая ссылка (URL), по которой файл доступен для скачивания на сервере.
- Папка для сохранения файла – путь к директории, в которую необходимо сохранить скачанный файл.
- Таймаут – максимальное время ожидания завершения скачивания файла в секундах. Если файл не загрузится за указанное время, выполнение блока завершится с ошибкой.
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Блок возвращает путь к сохраненному файлу, включая имя файла. Это значение можно сохранить в переменную и использовать в дальнейших шагах сценария (например, для обработки скачанного документа).

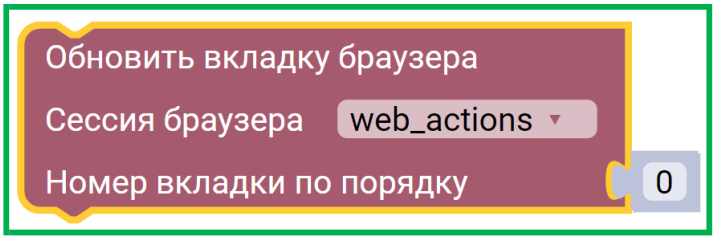
Важные особенности:

- Блок использует сессию браузера для скачивания, что позволяет учитывать авторизацию, куки и другие параметры сессии

- При скачивании файла учитываются настройки браузера (папка загрузок, автоматическое сохранение)
- Рекомендуется предварительно убедиться, что папка для сохранения существует, или предусмотреть ее создание в сценарии
- Имя файла обычно сохраняется таким, каким оно приходит с сервера (из заголовка Content-Disposition или из URL)

Блок «Скачать файл» расширяет возможности веб-автоматизации, добавляя функционал загрузки файлов. Он позволяет программно сохранять на компьютер любые файлы, доступные по ссылке, с учетом контекста браузерной сессии. Блок поддерживает настройку таймаута, что позволяет корректно обрабатывать длительные загрузки, и возвращает путь к сохраненному файлу для удобной интеграции в последующие шаги сценария.

Блок «Обновить вкладку браузера»



Блок позволяет обновить (перезагрузить) текущую или указанную вкладку браузера. Это необходимо в сценариях, где требуется сбросить состояние страницы, загрузить актуальные данные или повторить попытку загрузки при возникновении ошибок.

Параметры блока:

- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, которую необходимо обновить.

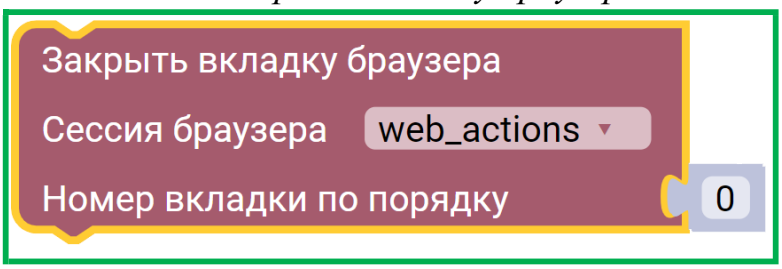
Типовые сценарии применения:

- Обновление динамического контента – когда данные на странице обновляются не в реальном времени и требуется ручное обновление
- Восстановление после ошибки – при возникновении ошибок загрузки или таймаутов можно обновить страницу и повторить попытку
- Сброс состояния формы – после заполнения формы или выполнения действий требуется вернуть страницу в исходное состояние
- Проверка обновлений – в сценариях мониторинга для периодической проверки актуальности данных
- Работа с несколькими вкладками – возможность обновить конкретную вкладку, даже если активна другая

Блок «Обновить вкладку браузера» является простым, но важным элементом навигации в веб-автоматизации. Он позволяет программно выполнять перезагрузку страницы, что расширяет возможности управления браузером и повышает устойчивость сценариев. Блок прост в настройке и эффективно дополняет набор

инструментов для управления браузером, обеспечивая полный контроль над жизненным циклом веб-страницы в процессе автоматизации.

Блок «Закрыть вкладку браузера»



Блок позволяет закрыть указанную вкладку браузера по её порядковому номеру. Это необходимо в сценариях, где требуется освобождать ресурсы, завер-шать работу с временными страницами или поддерживать порядок в интерфейсе браузера при работе с несколькими вкладками одновременно.

Параметры блока:

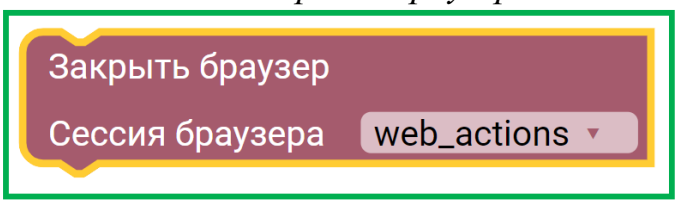
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, которую необходимо закрыть. Нумерация начинается с 0 (ноль – это первая открытая вкладка).

Важные особенности:

- После закрытия вкладки фокус автоматически переключается на следующую доступную вкладку (обычно ту, что была активна до закрытия или остаётся слева).
- Если закрыть последнюю открытую вкладку, сессия браузера остаётся активной, но браузер может отображать пустое окно.
- Нумерация вкладок динамическая: после закрытия одной вкладки порядковые номера оставшихся сдвигаются. Учитывайте это при построении сценариев с последовательным закрытием нескольких вкладок.

Блок «Закрыть вкладку браузера» является важным элементом управления ресурсами в веб-автоматизации. Он позволяет поддерживать чистоту рабочего пространства браузера, избегать накопления лишних вкладок и делает сценарии более структурированными и устойчивыми к сбоям. Блок прост в настройке и эффективно дополняет набор инструментов для управления браузером, обеспечивая полный контроль над жизненным циклом веб-страниц в процессе автоматизации.

Блок «Закрыть браузер»



Блок позволяет завершить сессию браузера, полностью закрыв окно браузера и освободив все связанные с ним ресурсы. Это необходимо в сценариях, где требуется корректно завершить работу автоматизации, избежать утечек памяти или подготовить окружение для следующего запуска.

Параметры блока:

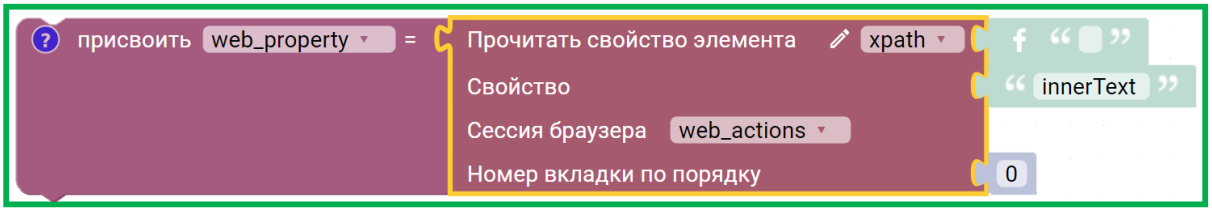
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».

Важные особенности:

- После выполнения блока объект сессии браузера становится недействительным. Дальнейшие попытки использовать эту переменную в других блоках приведут к ошибке.
- Блок закрывает все окна и вкладки, связанные с данной сессией, а не только активную вкладку.
- Закрытие браузера рекомендуется выполнять в конце любого сценария автоматизации, даже если сценарий завершается досрочно. Это считается хорошей практикой разработки.

Блок «Закрыть браузер» является обязательным элементом завершения сценариев веб-автоматизации. Он обеспечивает корректное освобождение ресурсов, предотвращает накопление «висящих» процессов в системе и позволяет выстраивать надёжные, циклические или многократно запускаемые сценарии. Блок прост в использовании, не требует дополнительных настроек и служит важной финаль-ной точкой в управлении жизненным циклом браузера.

Блок «Прочитать свойство элемента»



Блок позволяет считать значение указанного атрибута веб-элемента на странице. Это необходимо в сценариях, где требуется получить данные из интерфейса (текст, ссылки, значения полей, атрибуты) для дальнейшей обработки, проверки или сохранения в переменные.

Параметры блока:

- Параметры подключения – способ идентификации и поиска веб-элемента на странице. Доступны следующие варианты:

  `xpath` – поиск элемента по `XPath`-выражению;

  `css-selector` – поиск элемента по `CSS`-селектору;

  `tag_name` – поиск элемента по названию тега;

  `name` – поиск элемента по атрибуту `name`;

  `id` – поиск элемента по атрибуту `id`;

  `class` – поиск элемента по атрибуту `class`.
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие. Нумерация начинается с 0 (ноль – это первая открытая вкладка).

- Мастер UI для веб-автоматизации – при нажатии на соответствующую кнопку открывается визуальный помощник, который позволяет выбрать элемент на странице и сгенерировать параметры для его поиска.

Блок «Прочитать свойство элемента» является ключевым инструментом для извлечения данных из веб-интерфейсов в процессе автоматизации. Он позволяет получать любую информацию со страницы, что делает возможным построение гибких сценариев с ветвлениями, проверками и сбором данных. Блок интегрируется с Мастером UI для упрощения настройки и поддерживает все основные способы поиска элементов, обеспечивая высокую надёжность и точность работы.

Мастер UI

Мастер UI – это встроенный визуальный инструмент платформы Puzzle RPA, который упрощает создание и настройку XPath-запросов для поиска элементов на веб-странице. Вместо того чтобы писать сложные XPath-выражения вручную, вы можете наглядно выбрать элемент на странице, а редактор автоматически сформирует корректный запрос.

XPath (XML Path Language) – это язык запросов, который помогает роботу найти нужный элемент на веб-странице. Представьте, что HTML-код страницы – это дерево с ветками (родительскими элементами) и листьями (дочерними элементами). XPath – это «навигатор», который показывает роботу точный путь к элементу: например, «пройди в блок с классом menu, найди в нём список, а в списке – третью кнопку».

Кнопка  для открытия Мастера UI находится на всех блоках веб-автоматизации, где требуется указать элемент на странице (например, «Кликнуть на элемент», «Прочитать свойство элемента») Достаточно нажать на эту кнопку, и откроется окно редактора.

Перед началом работы необходимо установить Puzzle RPA Plugin – специальное расширение для браузера. Оно позволяет редактору взаимодействовать с открытой страницей: подсвечивать элементы, получать их XPath и выполнять проверки. Важно: плагин поддерживает только браузеры на основе Chromium.

Инструкция по установке:

1. Нажмите иконку Google в левом нижнем углу окна редактора XPath.

2. На открывшейся странице в магазине расширений нажмите кнопку «Установить».

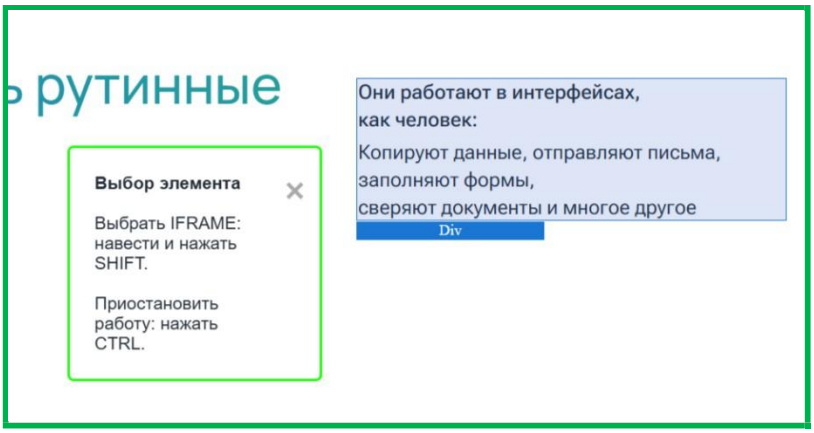
3. Перезагрузите браузер.

После открытия редактора необходимо указать, какой именно элемент на странице нужно использовать:

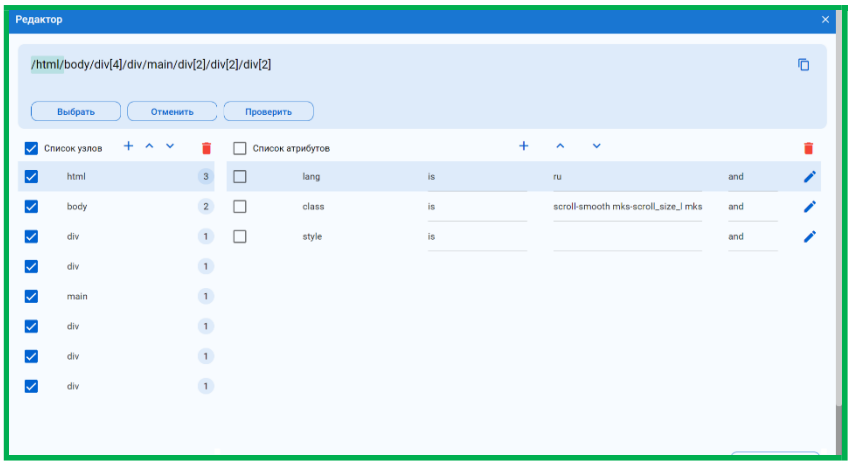
Нажмите кнопку «Выбрать» в редакторе.

Дождитесь появления подсказки о готовности инструмента.

Перейдите в окно браузера и кликните по нужному элементу на странице.



XPath-запрос к этому элементу автоматически отобразится в верхней панели редактора.



Интерфейс редактора

Редактор разделён на несколько функциональных зон:

Верхняя панель:

- Строка XPath – здесь отображается текущий XPath-запрос.
- Кнопка «Выбрать» – запускает процесс выбора элемента на странице.
- Кнопка «Проверить» – проверяет, сколько элементов на странице соответствует текущему XPath-запросу. При нажатии показывается количество найденных элементов, а кнопка «Посмотреть в браузере» позволяет подсветить их на странице.
- Кнопка «Скопировать» – копирует полученный XPath-запрос в буфер обме-на.

Левая панель: Список узлов

Здесь отображается древовидная структура XPath-запроса. Каждый уровень вложенности – это отдельный узел (элемент). С помощью чекбоксов можно включать или исключать узлы из запроса.

Кнопки управления узлами:

+ – добавить новый пользовательский узел.

∧ / ∨ – переместить выбранный узел выше или ниже (изменить порядок в запросе).

Удалить – удалить выбранный узел.

Правая панель: Список атрибутов

Когда вы выбираете узел в левой панели, справа отображаются все его атрибуты (например, `id`, `class`, `name`, `href`). С помощью чекбоксов можно добавлять или удалять атрибуты из XPath-запроса.

Для каждого атрибута можно выбрать функцию сравнения:

- `is` – значение точно равно указанному тексту.
- `contains` – значение содержит указанный текст (частичное совпадение).
- `starts-with` – значение начинается с указанного текста.
- `not is` – значение не равно указанному тексту.
- `contains-text` – текст внутри элемента содержит указанный текст.
- `starts-with-text` – текст внутри элемента начинается с указанного текста.
- `text-is` – текст внутри элемента точно равен указанному тексту.

Кнопка «Редактировать» (карандаш) у атрибута

Позволяет открыть окно дополнительных настроек атрибута:

- Использовать переменную – вместо фиксированного значения можно подставить значение из переменной.
- Добавить в скобки – группирует условие в скобки для сложных логических выражений.
- Изменить название атрибута – позволяет переименовать атрибут (например, использовать кастомный атрибут `data-id`).

Расширенные возможности настройки узлов

При нажатии на кнопку «Редактировать» у узла (или на количество его атрибутов) открывается окно «Настройки узла», где доступны дополнительные функции:

1. Переменная в качестве индекса

Вместо фиксированного номера элемента в списке (например, [3]) можно указать переменную. Это полезно, когда номер нужного элемента заранее неизвестен и вычисляется в процессе выполнения сценария.

2. Переменная в качестве узла

Позволяет подставить целый узел из переменной. Важно: при использовании этой опции все атрибуты узла должны быть отключены.

3. Функция перемещения по оси дерева

XPath позволяет искать элементы не только по иерархии «родитель → потомок», но и относительно текущего узла. Доступны следующие оси:

- `ancestor` – все предки (родитель, дед и выше).
- `ancestor-or-self` – предки и сам текущий узел.
- `preceding` – все узлы, расположенные перед текущим.
- `preceding-sibling` – предыдущие «соседи» (элементы на том же уровне).
- `following` – все узлы, расположенные после текущего.
- `following-sibling` – следующие «соседи» (элементы на том же уровне).
- `descendant` – все потомки (вложенные элементы).

- `descendant-or-self` – потомки и сам текущий узел.
- `self` – только текущий узел.
- `child` – прямые дочерние элементы.
- `parent` – родительский элемент.

Для построения сложных условий можно группировать атрибуты с помощью скобок. Например, условие `( @id = 'menu' or @class = 'main' ) and @visible = 'true'` задаётся путём добавления скобок в настройках соответствующих атрибутов. Важно: скобки добавляются парами. Открывающая скобка добавляется в настройках первого атрибута группы, закрывающая – в настройках последнего атрибута группы.

Если значение атрибута должно определяться во время выполнения сценария (например, текст поискового запроса, который меняется), его можно заменить переменной. Для этого:

1. Откройте настройки атрибута (кнопка «Редактировать»).
2. Установите чекбокс «Переменная атрибута».
3. Выберите нужную переменную из списка.
4. Нажмите «Сохранить».

Если переменная должна быть частью значения (например, user_123), укажите в поле значения символы ** в том месте, где должна подставляться перемен-ная.

Для удобства работы с деревом узлов предусмотрены горячие клавиши:

↑ / ↓ – перемещение между узлами.

Enter – включение или отключение выбранного узла (изменение чекбокса).

Сохранить – после настройки XPath-запроса нажмите эту кнопку, чтобы применить его в блоке автоматизации.

Скопировать – если нужно скопировать полученный XPath-запрос для использования в другом месте, нажмите кнопку копирования в верхней правой ча-сти редактора.

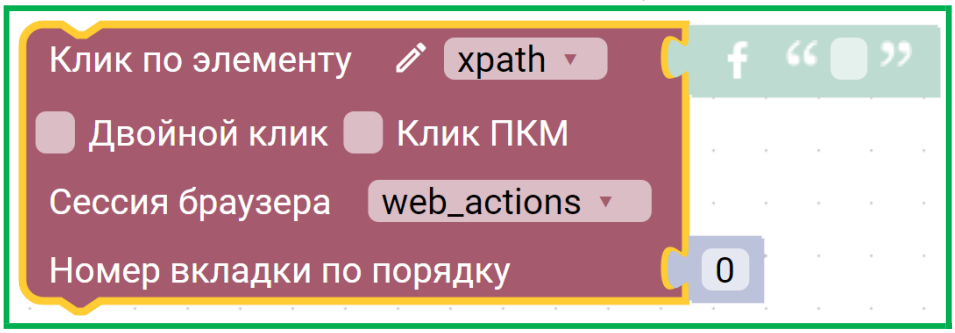
Важные особенности:

- Проверка XPath работает только если в запросе не используются перемен-ные. Для проверки подставьте временные фиксированные значения.
- При использовании переменной в качестве узла все атрибуты этого узла должны быть отключены, так как они будут подставлены вместе с переменной.
- Скобки для группировки всегда добавляются парами. Не забывайте настро-ить закрывающую скобку на последнем атрибуте группы.
- Мастер UI не заменяет знание XPath, но значительно ускоряет и упрощает процесс создания надёжных запросов, особенно для начинающих пользователей.

Мастер UI для веб-автоматизации является ключевым инструментом, снижающим порог входа в автоматизацию веб-приложений. Он позволяет создавать сложные XPath-запросы без ручного написания кода, проверять их работоспособ-

ность в реальном времени и использовать динамические переменные для построения гибких сценариев. Благодаря визуальному интерфейсу и поддержке клавиатурной навигации работа с элементами страницы становится быстрой, наглядной и надёжной.

Блок «Клик по элементу»



Блок позволяет выполнить клик (нажатие) по указанному элементу на веб-странице. Это одно из самых востребованных действий в веб-автоматизации, которое используется для навигации, взаимодействия с интерфейсом и запуска различных сценариев на сайте.

Параметры блока:

- Параметры подключения – способ идентификации и поиска веб-элемента на странице.
- Двойной клик – если этот чекбокс установлен, вместо одинарного клика выполняется двойной клик по элементу. Используется в сценариях, где требуется открыть файл, выделить текст или активировать действие, требующее двойного нажатия.
- Клик ПКМ – если этот чекбокс установлен, выполняется клик правой кнопкой мыши (открытие контекстного меню). Полезно для сценариев, где требуется вызвать дополнительное меню или скопировать данные из контекстного меню.
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие. Нумерация начинается с 0 (ноль – это первая открытая вкладка).
- Мастер UI для веб-автоматизации – при нажатии на соответствующую кнопку открывается визуальный редактор XPath, который позволяет выбрать элемент на странице и автоматически сформировать параметры для его поиска.

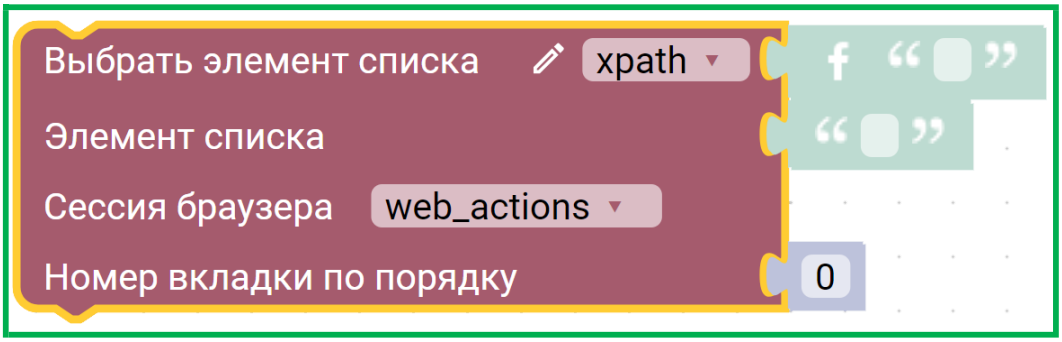
Рекомендации по использованию:

- Для кнопок и ссылок с уникальными идентификаторами рекомендуется использовать поиск по id – это самый быстрый и надёжный способ.
- Для элементов с динамически изменяющимися атрибутами (например, id генерируется случайным образом) лучше использовать XPath с относительными путями или поиск по стабильным атрибутам, таким как class или data-*.

- Перед использованием в сложных сценариях рекомендуется проверить XPath-запрос с помощью кнопки «Проверить» в Мастере UI, чтобы убедить-ся, что он находит ровно один нужный элемент.

Блок «Клик по элементу» является фундаментальным инструментом веб-автоматизации. Он обеспечивает программное взаимодействие с интерфейсом, имитируя действия реального пользователя. Благодаря гибким настройкам (одинарный, двойной клик, клик правой кнопкой) и интеграции с Мастером UI, блок позволяет реализовать широкий спектр сценариев – от простой навигации до сложных многошаговых взаимодействий с веб-приложениями. Простота настройки и надёжность выполнения делают его незаменимым при построении автоматизированных решений.

Блок «Выбрать элемент списка»



Блок позволяет выбрать один из вариантов в выпадающем списке (элементе <select>) на веб-странице. Это необходимо для автоматизации заполнения форм, выбора параметров, настройки фильтров и любых других сценариев, где пользователь взаимодействует с раскрывающимися списками.

Параметры блока:

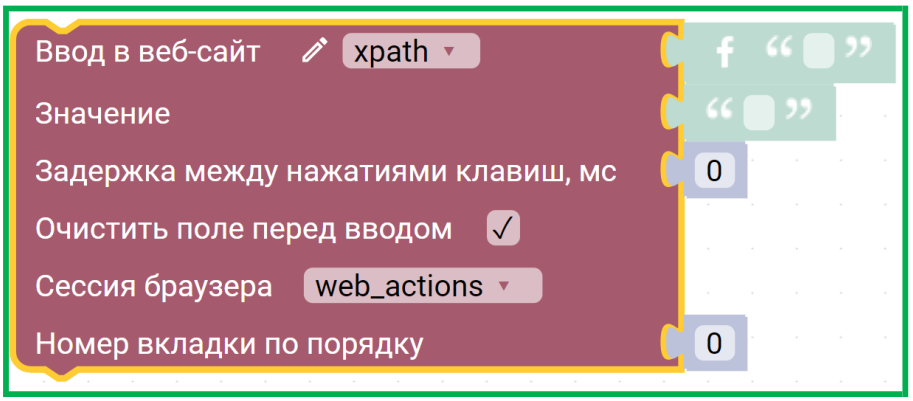
- Параметры подключения – способ идентификации и поиска веб-элемента на странице. Важно: указанный элемент должен быть тегом <select> (выпадающий список) или другим элементом, поддерживающим выбор вариантов.
- Элемент списка – значение, которое необходимо выбрать в выпадающем списке. Обычно это значение атрибута value у нужного варианта (<option>). Например, если в списке языков есть вариант <option value="ru">Русский</option>, то в параметре указывается ru. Также может использоваться текст, отображаемый в списке (например, «Русский»), в зависимости от реализации блока.
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие. Нумерация начинается с 0 (ноль – это первая открытая вкладка).
- Мастер UI для веб-автоматизации – при нажатии на соответствующую кнопку открывается визуальный редактор XPath, который позволяет выбрать элемент на странице и автоматически сформировать параметры для его поиска.

Рекомендации по использованию:

- Используйте атрибут value для выбора элемента, если это возможно. Значения value обычно более стабильны, чем видимый текст, который может меняться в зависимости от языка интерфейса или обновлений сайта.
- Проверяйте структуру списка через инструменты разработчика браузера (F12), чтобы точно определить, какой элемент и как правильно выбрать.
- Если список не открывается автоматически, перед использованием блока «Выбрать элемент списка» может потребоваться блок «Клик по элементу», чтобы раскрыть выпадающий список (для некоторых кастомных реализаций).
- Для вложенных списков (например, выбор страны → загрузка регионов → выбор региона) добавляйте ожидание (блок «Пауза» или «Ожидание элемента») после каждого выбора, чтобы дать странице время на подгрузку данных.
- При работе с динамическими списками рекомендуется использовать XPath с ожиданием конкретного значения, чтобы убедиться, что нужный вариант появился перед выбором.

Блок «Выбрать элемент списка» является специализированным инструментом для работы с выпадающими списками, который значительно упрощает автоматизацию форм по сравнению с ручной имитацией кликов и навигации. Он обеспечивает надёжное и быстрое выполнение выбора варианта, минимизирует количество шагов в сценарии и делает код автоматизации более читаемым и поддерживаемым. При работе с современными веб-приложениями, где выпадающие списки являются одним из основных элементов ввода, этот блок становится незаменимым помощником.

Блок «Ввод в веб-сайт»



Блок позволяет программно ввести текст в поле ввода на веб-странице. Это ключевое действие для автоматизации заполнения форм, поисковых запросов, авторизации и любых других сценариев, где требуется передача текстовой информации в интерфейс сайта.

Параметры блока:

- Параметры подключения – способ идентификации и поиска веб-элемента на странице.
- Значение – строка текста, которую робот вводит в указанный элемент. Это может быть как фиксированный текст (например, «Иванов»), так и значение переменной, содержащей динамические данные (например, результат предыдущих действий).

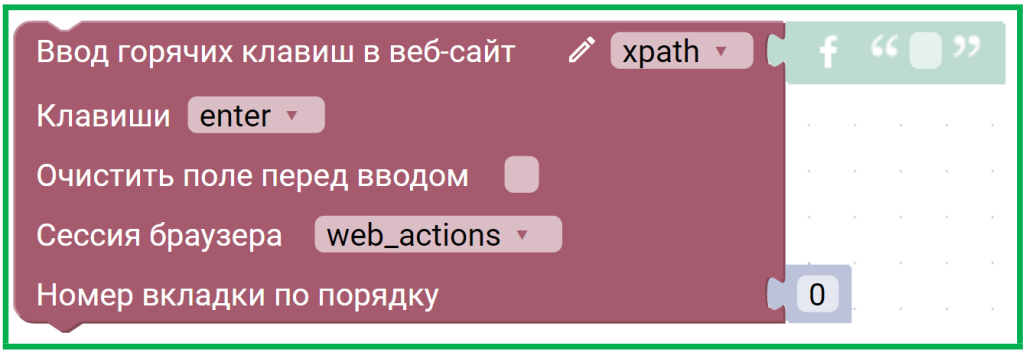
- Задержка между нажатиями клавиш, мс – интервал в миллисекундах между нажатиями отдельных клавиш при вводе текста. Имитирует скорость печати реального пользователя
- Очистить поле перед вводом – если этот чекбокс установлен, перед вводом нового текста поле будет очищено. Если опция не включена, новый текст добавится к уже существующему содержимому поля.
- Сессия браузера – переменная, содержащая объект созданной сессии браузера, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие. Нумерация начинается с 0 (ноль – это первая открытая вкладка).
- Мастер UI для веб-автоматизации – при нажатии на соответствующую кнопку открывается визуальный редактор XPath, который позволяет выбрать элемент на странице и автоматически сформировать параметры для его поиска.

Важные особенности

- Фокус ввода: перед вводом текста блок автоматически кликает по указанному полю, устанавливая фокус. Отдельно вызывать блок «Клик по элементу» не требуется.
- Очистка поля: при включённом чекбоксе «Очистить поле перед вводом» используется сочетание клавиш (обычно Ctrl+A + Delete или Backspace), которое удаляет всё содержимое поля. Если поле не очистить, новый текст добавится в конец существующего.
- Задержка между нажатиями: рекомендуется устанавливать небольшую задержку (например, 10–50 мс) для имитации реальной печати. Значение 0 означает максимально быстрый ввод, что может быть обнаружено системами защиты.
- Типы полей: блок работает с полями ввода различных типов: text, password, email, number, textarea и другими элементами, поддерживающими ввод тек-ста.

Блок «Ввод в веб-сайт» является одним из ключевых инструментов веб-автоматизации, обеспечивающим передачу данных от робота к веб-интерфейсу. В сочетании с блоками «Клик по элементу» и «Прочитать свойство элемента» он позволяет реализовать полный цикл взаимодействия с формами: переход к полю, ввод данных, отправка и проверка результата. Гибкие настройки задержки и очистки поля обеспечивают естественное поведение робота, а интеграция с Мастером UI упрощает настройку поиска полей ввода даже на самых сложных страницах.

Блок «Ввод горячих клавиш в веб-сайт»



Блок позволяет программно отправлять сочетания горячих клавиш на веб-страницу. Это необходимо для выполнения действий, которые не имеют явных кнопок в интерфейсе, или для ускорения навигации и работы с содержимым страницы. Блок имитирует нажатие комбинаций клавиш, аналогично тому, как это делает реальный пользователь.

Параметры блока

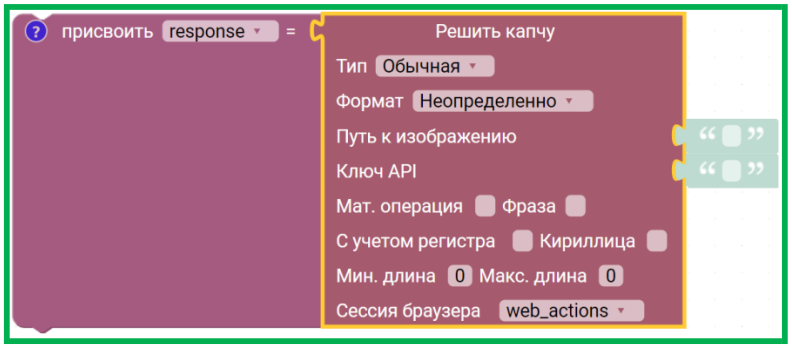
- Параметры подключения – способ идентификации и поиска веб-элемента на странице. Примечание: если необходимо отправить горячие клавиши на всю страницу (без привязки к конкретному элементу), достаточно указать любой элемент, который существует на странице, или использовать элемент body.
- Клавиши – выбор предустановленного сочетания клавиш из выпадающего списка. Доступны следующие опции:

Сочетание	Действие
`ctrl + a`	Выделить всё содержимое страницы или текущего поля
`ctrl + c`	Скопировать выделенный текст или элемент
`ctrl + v`	Вставить скопированный текст или элемент
`ctrl + x`	Вырезать выделенный текст или элемент
`ctrl + z`	Отменить последнее действие
`ctrl + r`	Обновить страницу
`ctrl + F5`	Обновить страницу с очисткой кэша (полное обновление)
`ctrl + s`	Сохранить текущую страницу
`ctrl + f`	Открыть окно поиска на странице
`ctrl + t`	Открыть новую вкладку
`ctrl + w`	Закрыть текущую вкладку
`F5`	Обновить текущую страницу
`esc`	Закрыть модальное окно, выйти из полноэкранного режима или отменить действие
`tab`	Перейти к следующему интерактивному элементу на странице
`shift + tab`	Перейти к предыдущему интерактивному элементу
`enter`	Подтвердить ввод, отправить форму или активировать выбранный элемент
`backspace`	Удалить символ слева от курсора (возврат на предыдущую страницу в браузере)
`delete`	Удалить выделенный текст или элемент
`alt + left`	Вернуться на предыдущую страницу (назад в истории браузера)

- Очистить поле перед вводом – если этот чекбокс установлен, перед отправ-кой горячих клавиш указанное поле будет очищено. Актуально для сценари-ев, где сочетание клавиш применяется к конкретному полю ввода (например, ctrl + a для выделения всего текста в поле перед его заменой).
- Сессия браузера – переменная, содержащая объект созданной сессии браузе-ра, полученный от блока «Создать сессию браузера».
- Номер вкладки по порядку – порядковый номер вкладки, в которой необхо-димо выполнить действие. Нумерация начинается с 0 (ноль – это первая от-крытая вкладка).
- Мастер UI для веб-автоматизации – при нажатии на соответствующую кноп-ку открывается визуальный редактор XPath, который позволяет выбрать эле-мент на странице и автоматически сформировать параметры для его поиска.

Блок «Ввод горячих клавиш в веб-сайт» расширяет возможности веб-автоматизации, позволяя выполнять действия, которые сложно или невозможно реализовать через обычные клики и ввод текста. Он особенно полезен в сценари-ях, требующих ускоренной навигации, работы с системными функциями браузера (вкладки, обновление, история) или имитации сложных пользовательских дей-ствий. В сочетании с блоками «Ввод в веб-сайт» и «Клик по элементу» он обеспе-чивает полный спектр взаимодействия с веб-интерфейсом на уровне клавиатурно-го ввода.

Блок «Решить капчу»



Блок позволяет автоматически решить капчу различных типов с использова-нием сервиса ruCaptcha. Это необходимо в сценариях автоматизации, где веб-страница требует подтверждения, что действия выполняются реальным пользова-телем, а не роботом. Блок интегрируется с внешним сервисом распознавания, от-правляя капчу на решение и получая результат для дальнейшего использования.

Капча (CAPTCHA – Completely Automated Public Turing test to tell Computers and Humans Apart) – это тест, который используется веб-сайтами для отличия ре-альных пользователей от автоматических программ. При автоматизации веб-действий робот сталкивается с капчей как с препятствием, которое необходимо преодолеть, чтобы продолжить выполнение сценария.

Блок «Решить капчу» передаёт изображение или параметры капчи в сервис ruCaptcha, где она решается реальными операторами или автоматическими алго-ритмами, после чего полученный ответ возвращается в сценарий для подстановки в соответствующее поле или отправки формы.

Важные особенности:

- Требуется API-ключ: для работы блока необходимо зарегистрироваться в сервисе ruCaptcha и получить API-ключ. Сервис является платным, стои-мость зависит от типа и сложности капчи.
- Ожидание решения: процесс решения капчи занимает некоторое время (обычно от нескольких секунд до минуты). Блок автоматически ожидает получения ответа от сервиса.
- Сохраняйте результат: после успешного решения капчи блок возвращает ответ (текст для обычной капчи или токен для reCAPTCHA), который необходимо сохранить в переменную и использовать в последующих блоках (например, ввести в поле или отправить с формой).
- Для reCAPTCHA v2/v3: требуется корректно настроить Sitekey и URL. Sitekey можно найти в коде страницы, выполнив поиск по фрагменту data-sitekey или g-recaptcha.
- Невидимая reCAPTCHA v2: требует дополнительной настройки, так как не имеет видимого элемента для взаимодействия. В этом случае капча срабатывает автоматически при определённых действиях на странице (например, при отправке формы).

Блок «Решить капчу» является важным инструментом для создания полноценных сценариев автоматизации, работающих с сайтами, имеющими защиту от ботов. Он интегрируется с популярным сервисом ruCaptcha, поддерживает три основных типа капч и позволяет гибко настраивать параметры распознавания в зависимости от конкретной реализации капчи на сайте. Использование этого блока значительно расширяет возможности автоматизации, позволяя обходить один из наиболее распространённых барьеров на пути робота.

Рассмотренные блоки образуют единый набор инструментов для автоматизации взаимодействия с веб-интерфейсами. Они охватывают все ключевые этапы: управление сессией браузера, навигацию по страницам и вкладкам, поиск элементов, различные виды кликов, ввод текста и горячих клавиш, работу с выпадающими списками, извлечение данных и решение капч через сервис ruCaptcha.

Все блоки унифицированы – используют единые способы поиска элементов (XPath, CSS-селектор, id, class и другие) и интегрированы с Мастером UI, который позволяет визуально выбирать элементы на странице и автоматически формировать корректные запросы без ручного написания кода.

Такая архитектура обеспечивает низкий порог входа для начинающих, высокую скорость разработки сценариев и достаточную гибкость для реализации сложных многошаговых процессов. Базовые блоки веб-автоматизации Puzzle RPA представляют собой сбалансированный набор, позволяющий решать подавляющее большинство прикладных задач по роботизации веб-систем.

Подведение итогов практической работы

Мы завершили изучение практической работы № 9 по веб-автоматизации в Puzzle RPA. За время освоения этого раздела мы рассмотрели ключевые темы, ко-

торые формируют фундамент для создания эффективных роботов, взаимодействующих с веб-интерфейсами.

Мы выяснили, что веб-автоматизация принципиально отличается от десктопной. В отличие от статичных настольных приложений, веб-интерфейсы работают по модели клиент-сервер, где страница загружается с удалённого сервера, а её структура может меняться динамически. Это накладывает особые требования к надёжности сценариев: необходимо учитывать время загрузки элементов, использовать гибкие способы поиска (XPath, CSS-селекторы) и предусматривать обработку ситуаций, когда сайт «обновляется» без нашего участия.

Мы разобрались, что в основе работы любого веб-приложения лежит клиент-серверная архитектура. Браузер (клиент) отправляет HTTP-запросы на сервер, сервер обрабатывает их и возвращает HTML-страницы, стили, скрипты и данные. Понимание этого принципа помогает роботу корректно взаимодействовать с сайтами: ожидать ответа сервера перед выполнением следующего действия, обрабатывать редиректы и понимать, почему некоторые элементы появляются на странице не мгновенно.

Мы изучили, что HTML-страница – это не просто текст, а древовидная структура, которая называется DOM. Каждый элемент страницы является узлом этого дерева, имеющим свои атрибуты, содержимое и отношения с другими узлами. Именно с DOM работает робот, когда ищет элемент по XPath или CSS-селектору. Понимание DOM позволяет нам строить правильные запросы, нахо-дить элементы даже при отсутствии уникальных идентификаторов и предсказы-вать, как изменится структура страницы после взаимодействия.

Мы освоили инструменты веб-разработчика (F12), которые стали нашим главным помощником при создании сценариев. С их помощью мы научились исследовать DOM-структуру страницы, проверять XPath-запросы, находить уникальные атрибуты элементов, отслеживать сетевые запросы и отлаживать сценарии. Эти инструменты позволяют «заглянуть под капот» любого сайта и точно определить, как роботу найти нужный элемент.

Наконец, мы подробно рассмотрели основные блоки веб-автоматизации, которые предоставляет Puzzle RPA. Мы научились управлять сессией браузера, переходить по ссылкам, работать с вкладками, кликать по элементам (одинарным, двойным, правым кликом), вводить текст и горячие клавиши, выбирать элементы из выпадающих списков, считывать свойства элементов и решать капчи через сервис ruCaptcha.

Особую ценность представляет Мастер UI – визуальный редактор XPath, который позволяет выбирать элементы на странице мышью, автоматически генерировать запросы и проверять их работоспособность без ручного написания кода.

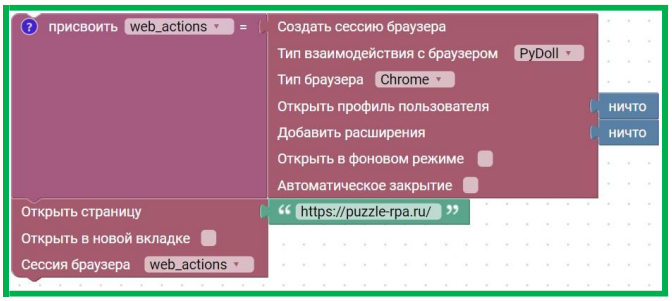
В ходе практической работы мы сформировали целостное понимание веб-автоматизации: от базовых принципов устройства веба до конкретной реализации сценариев в Puzzle RPA. Полученные знания позволяют нам самостоятельно разрабатывать роботов для автоматизации работы с веб-сайтами – регистрации, сбора данных, заполнения форм, мониторинга и других задач.

Главный результат – мы научились не просто использовать готовые блоки, но и понимать, что происходит «под капотом»: как браузер взаимодействует с сервером, как устроена DOM-модель, как правильно искать элементы и как делать сценарии устойчивыми к изменениям. Это фундамент, на котором можно строить сколь угодно сложные проекты веб-автоматизации.

ПРИМЕРЫ ЗАДАЧ

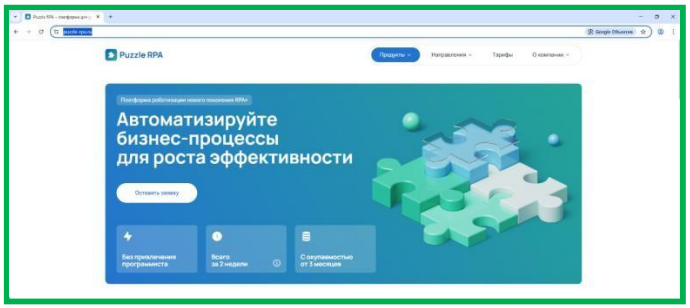
Пример 1 – Открыть страницу

В данном примере создается новая сессия браузера и открывается новая страница https://puzzle-rpa.ru/:



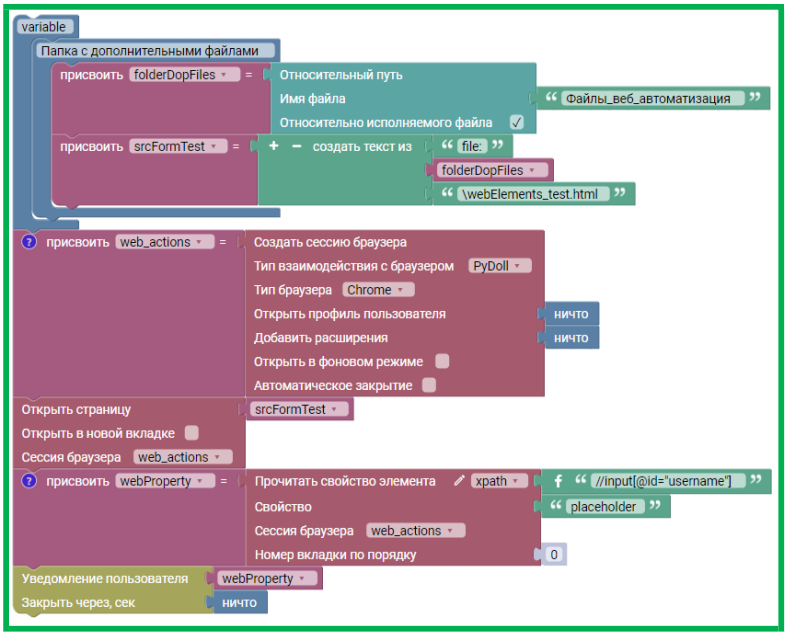
Результат

Открытый браузер Google Chrome:



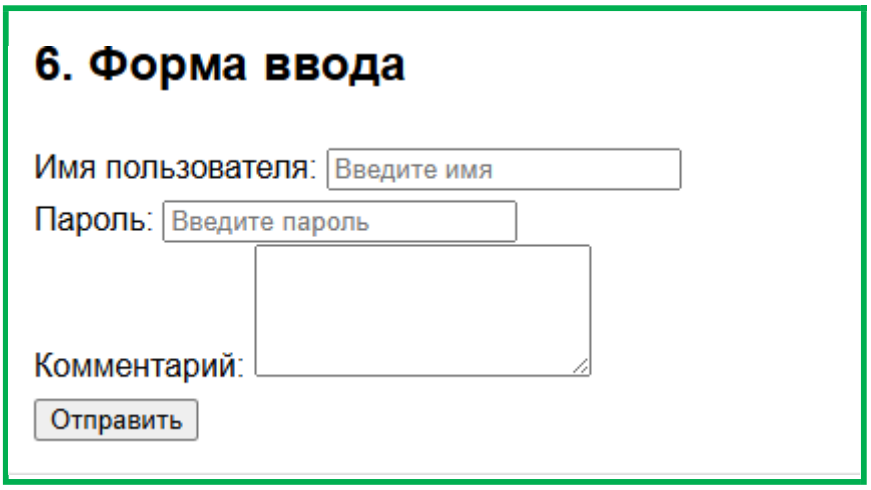
Пример 2 – Прочитать свойство элемента

В данном примере происходит считывание значения свойства на html странице:

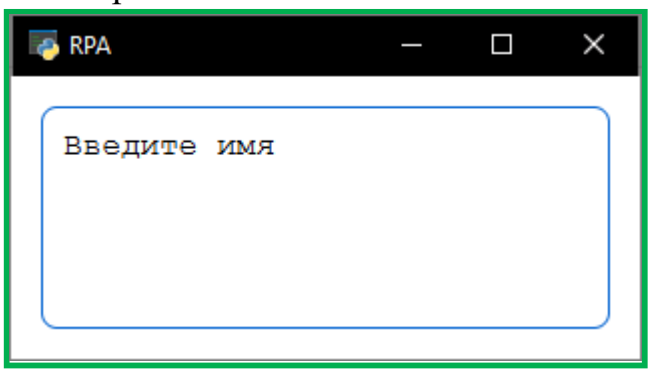


Алгоритм ожидания веб-элемента:

- Открывается html страница из папки **resources** проекта;
- Считывание значения свойства placeholder элемента //input[@id="username"];

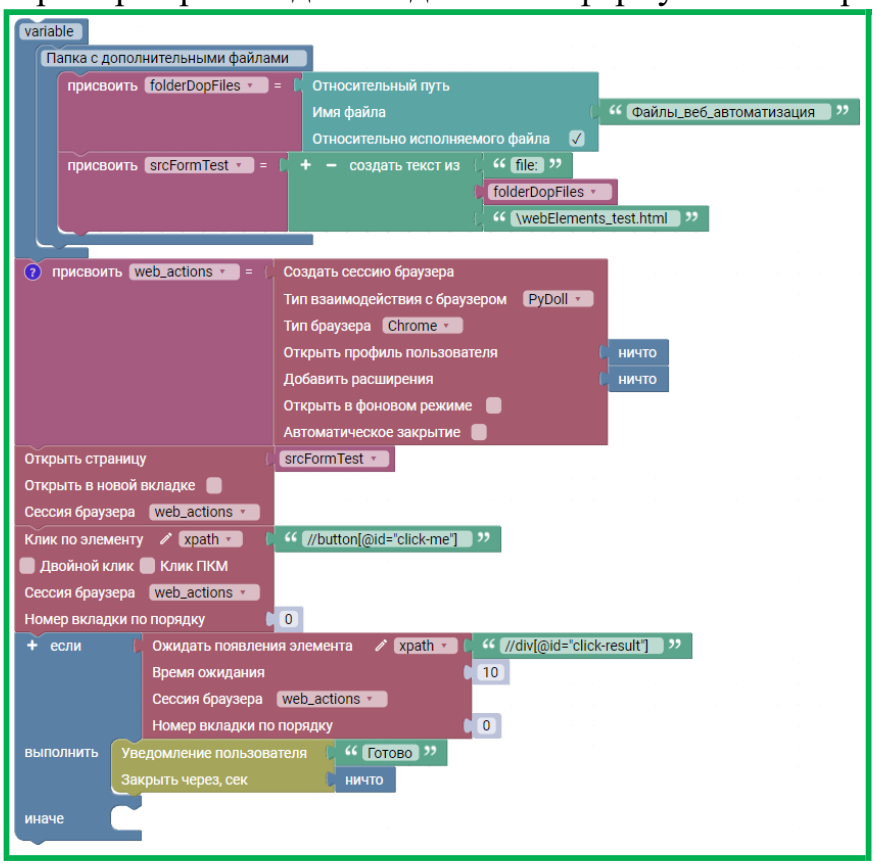


Вывод результата на экран:



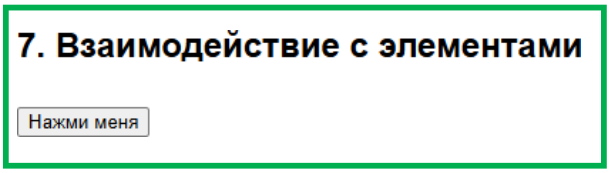
Пример 3 – Клик по элементу

В данном примере происходит ввод текста в форму на html странице:

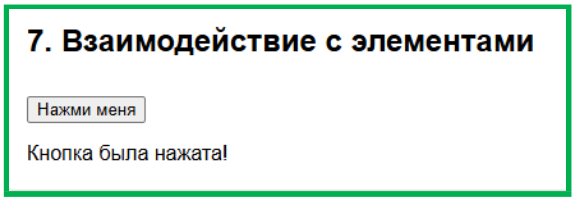


Алгоритм ожидания веб-элемента:

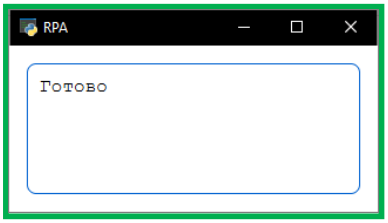
- Открывается html страница из папки **resources** проекта;



- Клик по кнопке **Нажми меня**;
- Выводится текстовое поле **Кнопка была нажата!**;

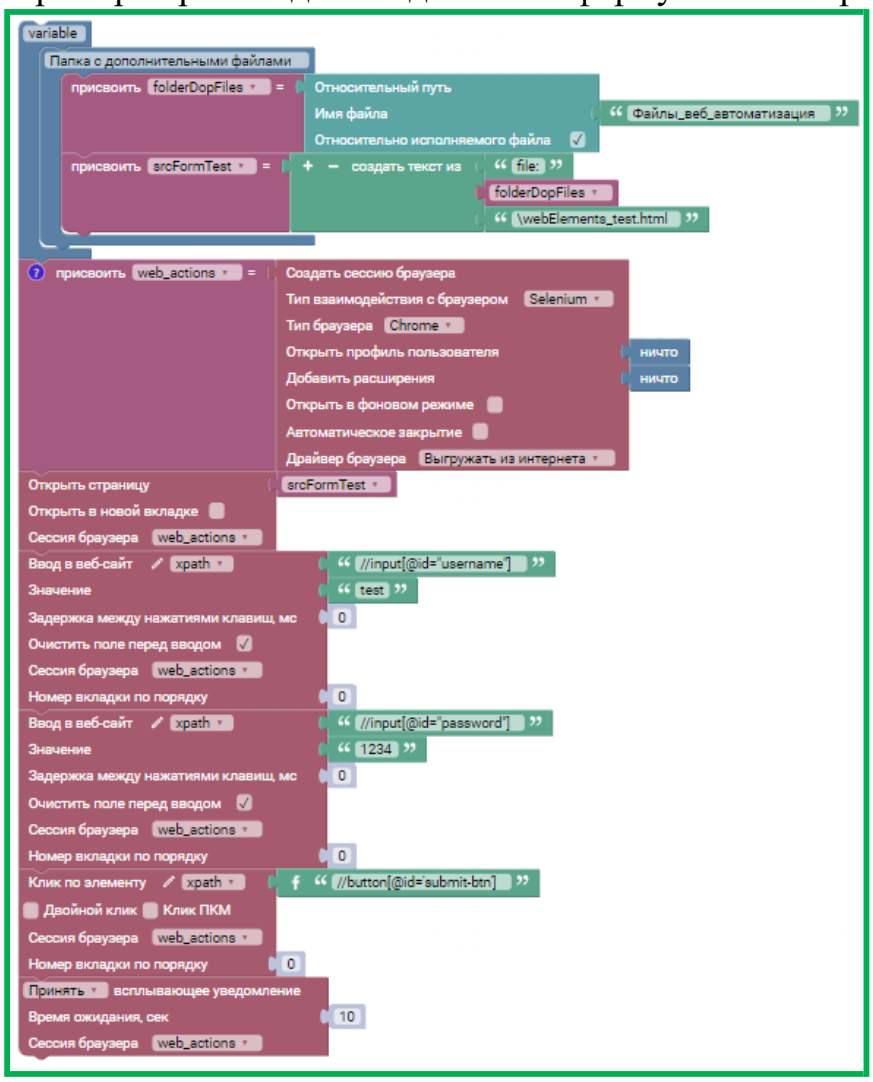


Вывод результата на экран:



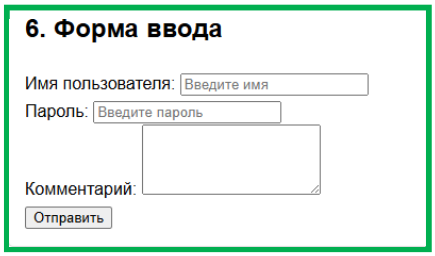
Пример 4 – Ввод в веб-сайт

В данном примере происходит ввод текста в форму на html странице:

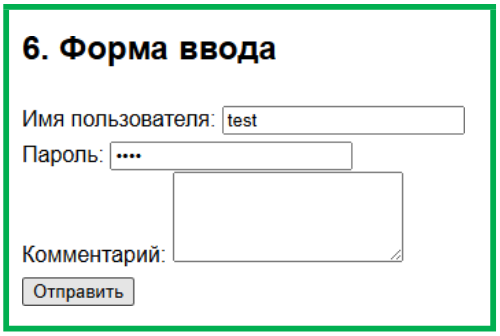


Алгоритм ожидания веб-элемента:

- Открывается html страница из папки **resources** проекта;

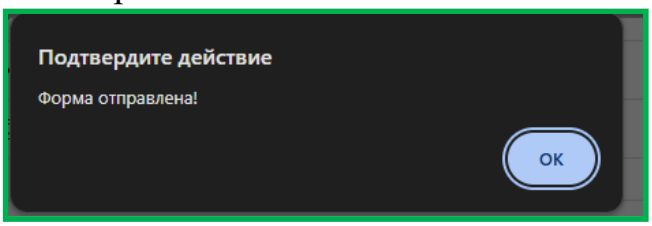


- Ввод текста test в поле ввода для логина;
- Ввод текста 1234 в поле ввода для пароля;



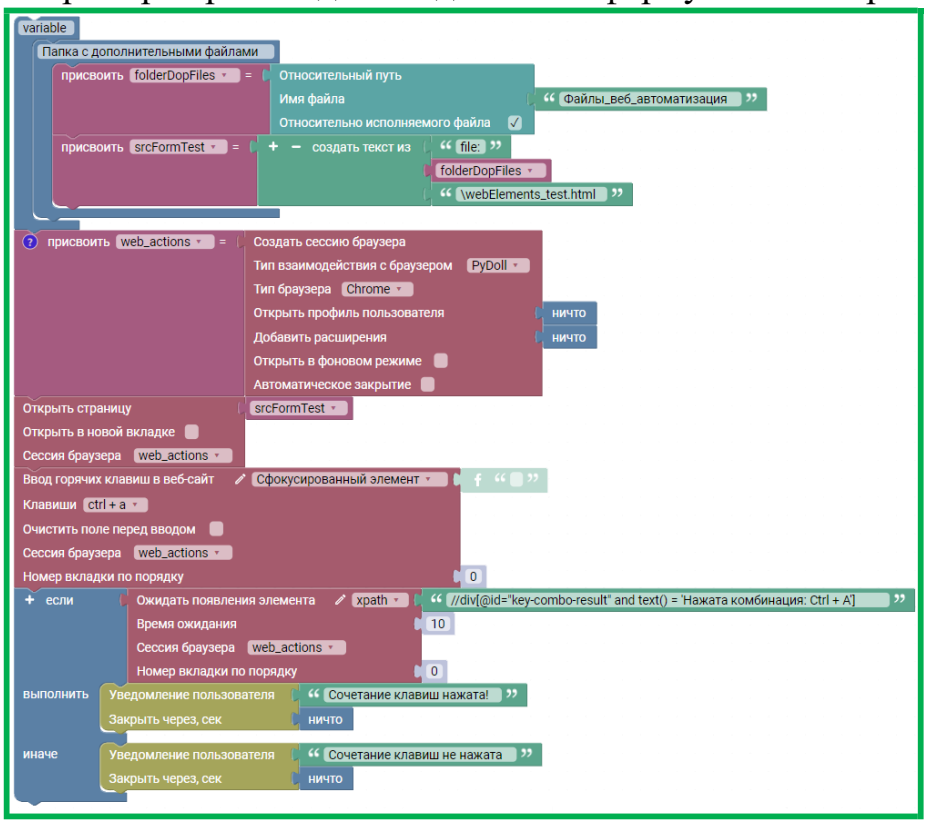
- Клик по кнопке **Отправить**, чтобы отправить форму

Вывод результата на экран:



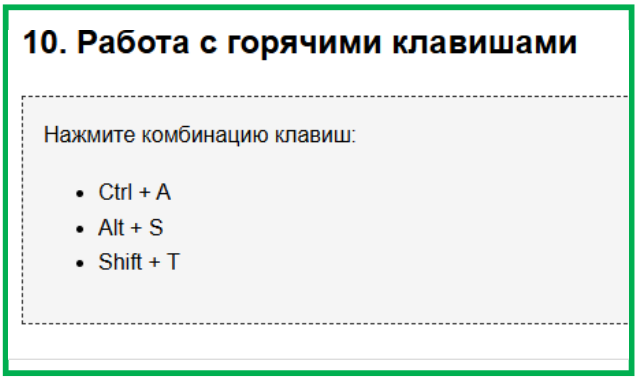
Пример 5 – Ввод горячих клавиш

В данном примере происходит ввод текста в форму на html странице:

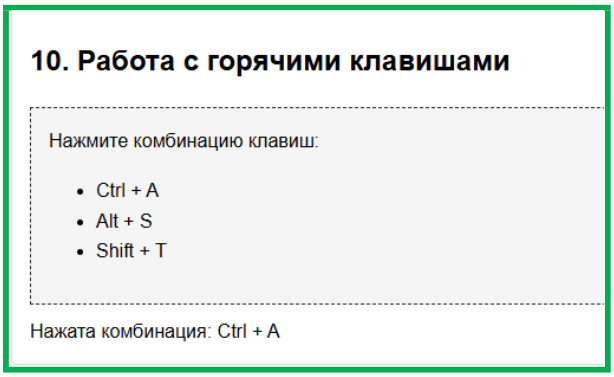


Алгоритм ожидания веб-элемента:

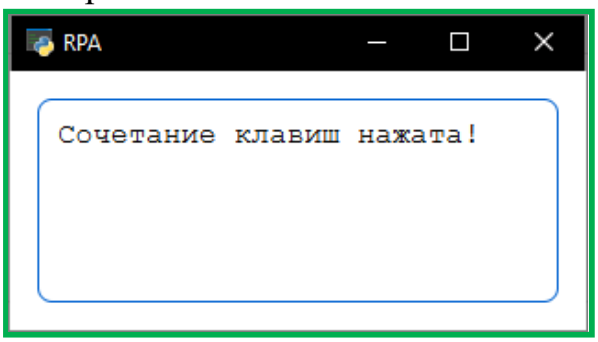
- Открывается html страница из папки **resources** проекта;



- Ввод сочетания клавиш в сфокусированный элемент на странице браузера;
- Вывод текста на странице браузера с нажатым сочетанием клавиш;



Вывод результата на экран:



Кейс 1. «Веб-портал управления заявками на участие в IT-мероприятиях»

Компания EventumTech занимается организацией IT-мероприятий: хакатонов, митапов, онлайн-курсов и конференций.

Для регистрации участников используется веб-форма, через которую пользователи подают заявки на участие. В периоды активных регистраций сотрудники вручную проверяют и обрабатывают сотни заявок, что приводит к:

- задержкам обработки;
- ошибкам при вводе данных;
- перегрузке персонала.

Для оптимизации процесса было принято решение:

- разработать учебный веб-портал регистрации заявок;
- протестировать возможность автоматизации обработки заявок с помощью RPA-робота (Puzzle RPA).

Задание 1: Разработка веб-приложения

Необходимо разработать веб-приложение с двумя страницами:

- Страница авторизации
- Страница подачи заявки (с таблицей заявок)

1.2. Требования к странице авторизации

Элемент	ID	Требования
Поле логина	`login`	Обязательное, мин. 4 символа (латиница + цифры)
Поле пароля	`password`	Обязательное, мин. 6 символов
Кнопка входа	`signInBtn`	Проверяет данные, при ошибке показывает сообщение

Ошибки валидации:

- Поля пустые → `"Please fill in all fields"`
- Данные неверные → `"Invalid credentials"`

Успешный вход: переход на страницу подачи заявки

1.3. Требования к странице подачи заявки

Поля формы:

Поле	ID	Требования
Participant Name	`participantName`	Только буквы, мин. 3 символа
Email	`email`	Формат email (user@example.com)
Event Name	`eventName`	Текст/цифры, мин. 3 символа
Participation Format	`participationFormat`	Выпадающий список: Online / Offline
Accept Terms	`acceptTerms`	Чекбокс, обязательный

Кнопка Submit Application:

- ID динамический: когда активна → submit_1234567890
- Состояние: disabled = true пока не заполнены ВСЕ поля
- Состояние: disabled = false когда все поля валидны

1.4. Дополнительные требования

- Асинхронность: задержка появления формы и таблицы (1-2 секунды)
- Динамические элементы: кнопка Submit Application меняет ID при активации

1.5. Тестовые данные для проверки

№	Participant Name	Email	Event Name	Format
1	Ivan Petrov	ivan@mail.com	AI Hackathon	Online
2	Anna Smirnova	anna@mail.com	Dev Meetup	Offline
3	Dmitry Orlov	dmitry@mail.com	Data Science Boot	Online
4	Olga Ivanova	olga@mail.com	QA Conference	Offline
5	Sergey Kuznetsov	sergey@mail.com	Cloud Summit	Online

Важное замечание: Запуск веб-приложения через локальный сервер

После того как мы разработали веб-приложение, главная ошибка это просто открывать HTML-файл двойным щелчком мыши в браузере. В адресной строке при этом появляется ссылка, начинающаяся с `file:///C:/Users/.` Такой способ открытия называется "протокол file" (файловый протокол).

Однако при работе с Puzzle RPA такой подход не сработает. Потому что робот, управляя браузером через Selenium WebDriver или pyDoll, не может коррект-

но взаимодействовать с веб-страницей, открытой через файловый протокол. Возникают проблемы с поиском элементов, выполнением JavaScript и обработкой динамического содержимого страницы.

Для корректной работы робота веб-приложение должно быть доступно по протоколу `http://` или `https://`. Для этого необходимо запустить локальный веб-сервер.

`localhost` – это специальное сетевое имя, которое указывает на ваш собственный компьютер. Оно всегда соответствует IP-адресу `127.0.0.1`. Когда робот обращается к `http://localhost:8000`, он отправляет запрос на ваш же компьютер, где запущен сервер. Сервер обрабатывает запрос и возвращает HTML-страницу. Благодаря этому все механизмы веб-автоматизации работают корректно.

Puzzle RPA использует технологии, которые полагаются на стандартные веб-протоколы. Протокол `http://` поддерживает все необходимые механизмы: поиск элементов по XPath и CSS-селекторам, выполнение JavaScript, обработку собы-тий, ожидание появления динамических элементов. Протокол `file://` эти возможности либо не поддерживает вовсе, либо поддерживает с серьёзными ограничениями.

Важное напоминание – сервер должен оставаться запущенным на всё время работы робота. Если закрыть терминал с запущенным сервером, робот потеряет связь с веб-приложением.

Таким образом, перед запуском робота всегда необходимо сначала запустить локальный веб-сервер и только затем выполнять сценарий в Puzzle RPA. Это обязательное условие для корректной работы автоматизации.

Задание 2: Автоматизация в Puzzle RPA

Разработать RPA-робота в Puzzle RPA, который автоматически:

- Авторизуется в веб-приложении
- Заполняет 5 тестовых заявок
- Отправляет их
- Проверяет результат в таблице
- Ведет лог выполнения

Рекомендуется реализовать структуру с использованием функций/подпроцессов

После работы робота в таблице должны быть 5 заявок с правильными данными и статусом "Pending".

Рекомендуется протестировать сценарий с некорректными данными:

Participant Name	Email	Event Name	Format
(пусто)	test	Event	Online

Ожидаемое поведение:

- кнопка Submit Application остаётся неактивной;
- робот обнаруживает проблему;
- корректно завершает сценарий.

Теперь перейдём к реализации RPA-сценария в среде Puzzle RPA.

В рамках практической работы был разработан RPA-робот в среде Puzzle RPA для автоматизации процесса подачи заявок на мероприятия. Робот выполня-ет следующие функции:

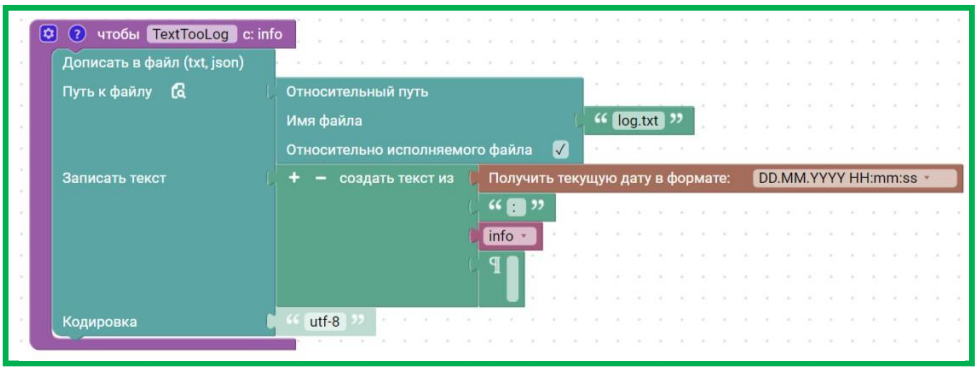
- Автоматический вход в веб-приложение
- Чтение данных из Excel-файла
- Массовое заполнение и отправка заявок
- Проверка корректности добавленных данных
- Логирование всех этапов выполнения

Функция `TextTooLog`

Шаг 1. Блок "Получить текущую дату" – форматируем время в формат "дд.мм.гггг чч:мм:сс".

Шаг 2. Блок "Собрать строку" – склеиваем полученную дату, текст информационного сообщения и перевод строки. На вход функция принимает параметр info, который содержит текстовое сообщение для записи в лог. Это сообщение передаётся при вызове функции, например `TextTooLog`('Окно для ввода загружено'), где строка 'Окно для ввода загружено' и есть переменная info.

Шаг 3. Блок "Дописать в файл" – сохраняем сформированную строку в файл log.txt, который располагается в папке с программой. Таким образом, каждый вызов функции добавляет новую строку в лог-файл с указанием времени и переданным сообщением.

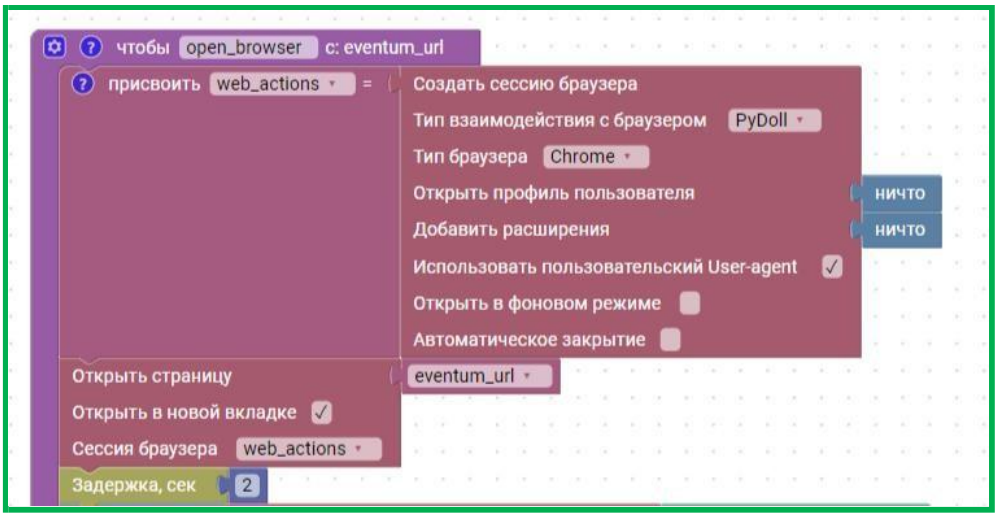


Функция `open_browser`

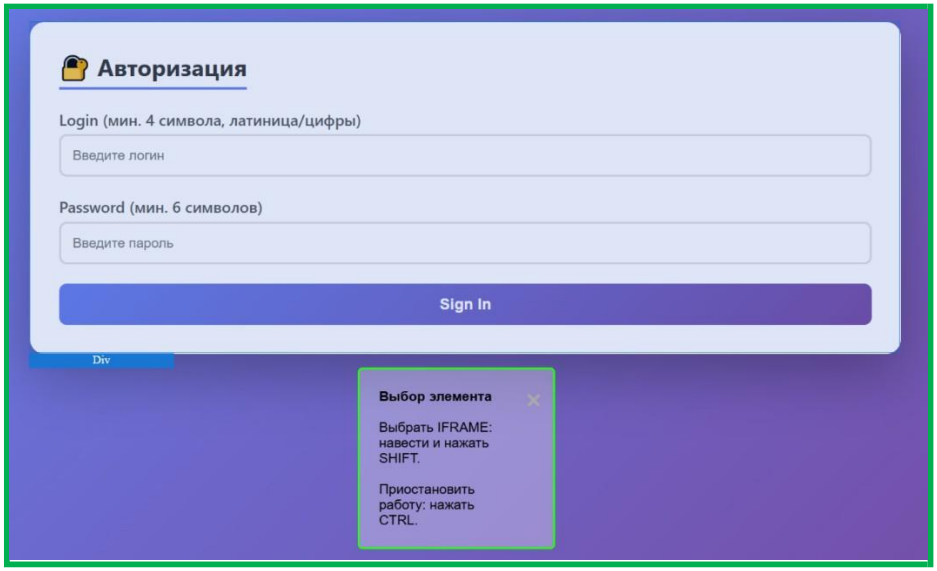
Шаг 1. Блок "Создать сессию браузера" – инициализируем браузер через функцию `init_web_automation_pydoll` с параметрами: тип браузера = `Chrome`. На вход функция принимает параметр `eventum_url`, который содержит адрес веб-страницы, которую необходимо открыть. Этот URL передаётся при вызове функции.

Шаг 2. Блок "Открыть страницу в браузере" – переходим по URL, полученному в переменной `eventum_url`, открывая страницу в новой вкладке браузера.

Шаг 3. Блок "Задержка" – устанавливаем паузу продолжительностью 2 секунды для полной загрузки страницы.



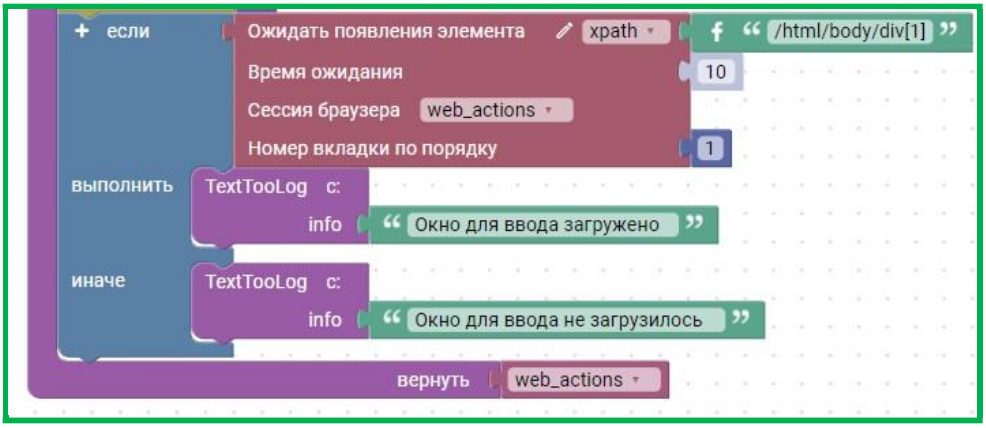
Шаг 4. Блок "Ожидать появление элемента" – ждём появления элемента с XPath `"/html/body/div[1]"` с таймаутом 10 секунд. Это позволяет убедиться, что главное окно приложения загрузилось.



Шаг 5. Если окно появилось – вызываем функцию `TextTooLog` с сообщением "Окно для ввода загружено".

Шаг 6. Если окно не появилось – вызываем функцию `TextTooLog` с сообщением "Окно для ввода не загрузилось".

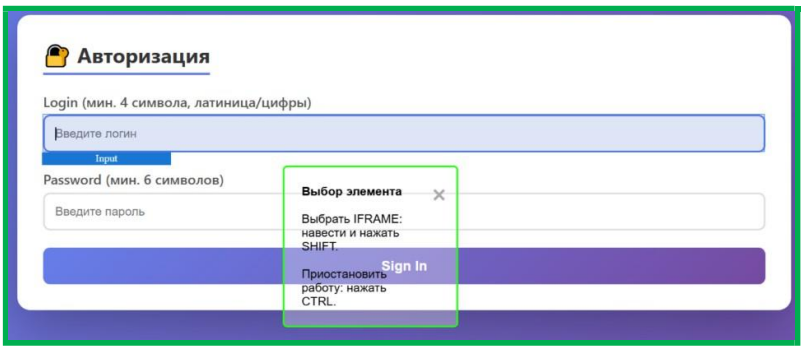
Шаг 7. Блок "Вернуть результат" – возвращаем переменную `web_actions` в основной процесс для дальнейшего управления браузером.



Функция `autorisation`

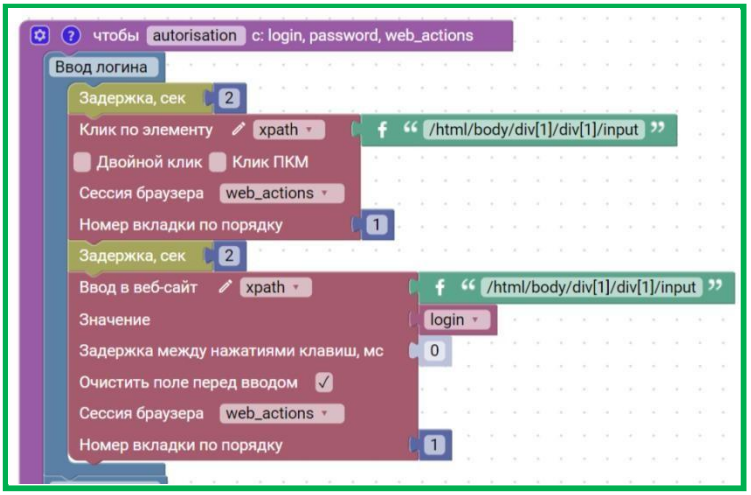
Шаг 1. Блок "Задержка" – пауза 2 секунды перед началом работы с формой авторизации. На вход функция принимает три параметра: `login` (логин пользователя), `password` (пароль пользователя) и `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей). Параметр `web_actions` передаётся из функции open_browser и содержит все методы для взаимодействия с браузером, такие как клики, ввод текста и проверка элементов.

Шаг 2. Блок "Клик по Web-элементу" – с помощью переданной сессии `web_actions` выполняем клик по полю ввода логина с XPath `"/html/body/div[1]/div[1]/input"`.



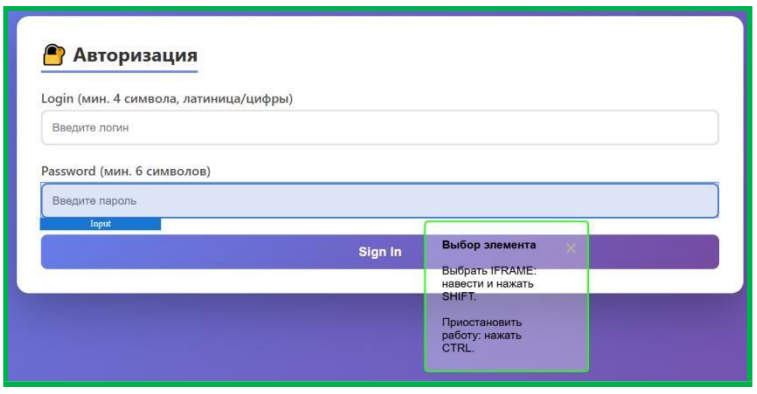
Шаг 3. Блок "Задержка" – пауза 2 секунды после клика.

Шаг 4. Блок "Ввод в веб-сайт" – используя сессию `web_actions`, вводим текст, полученный в параметре `login`, в поле ввода логина.



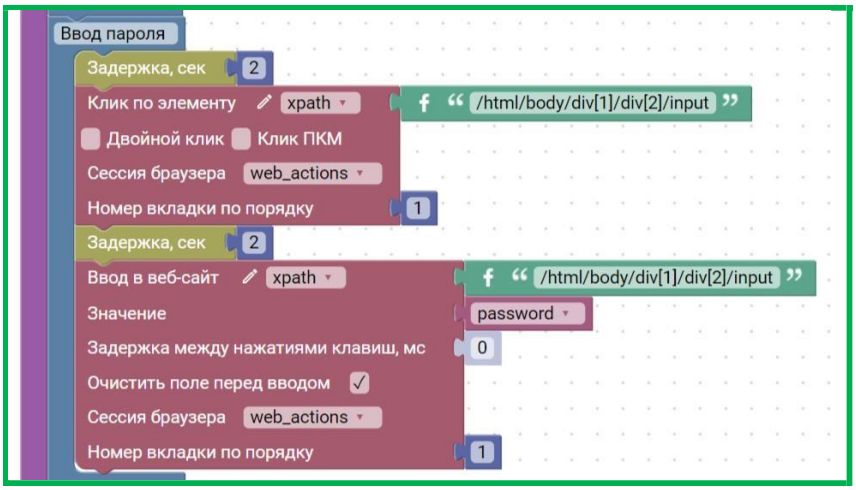
Шаг 5. Блок "Задержка" – пауза 2 секунды.

Шаг 6. Блок "Клик по Web-элементу" – с помощью сессии web_actions выполняем клик по полю ввода пароля с XPath `"/html/body/div[1]/div[2]/input"`.

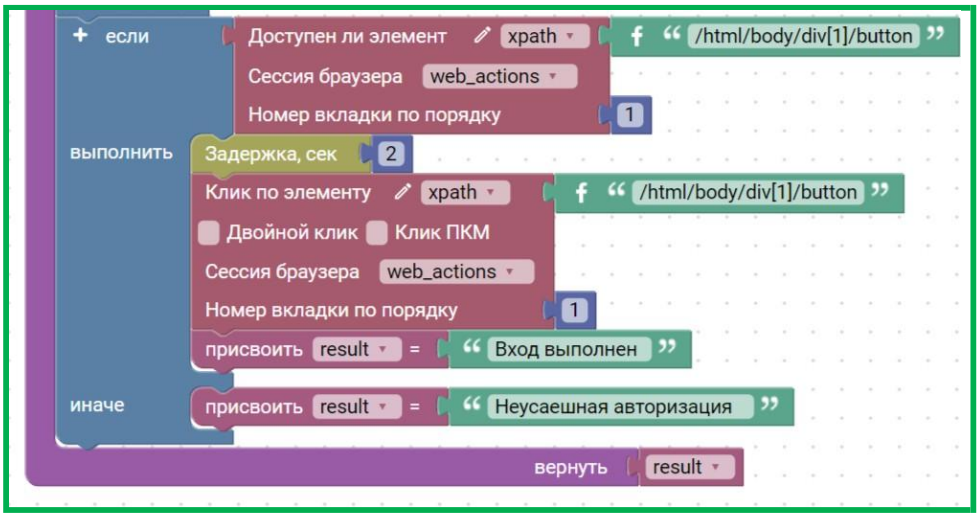


Шаг 7. Блок "Задержка" – пауза 2 секунды.

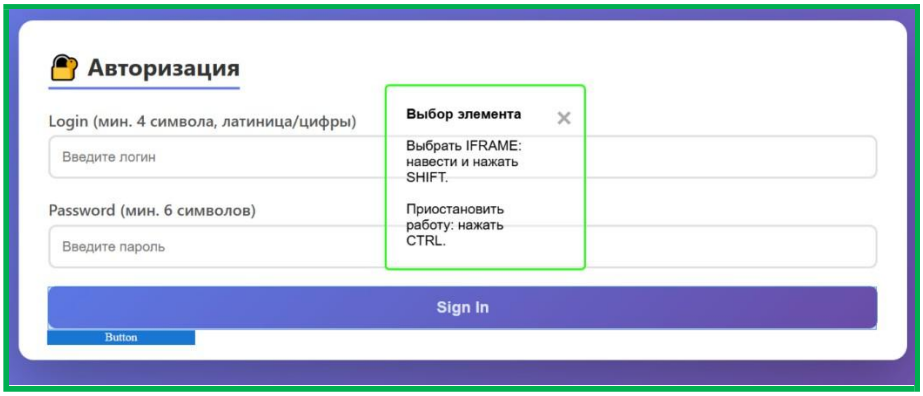
Шаг 8. Блок "Ввод в веб-сайт" – используя сессию `web_actions`, вводим текст, полученный в параметре `password`, в поле ввода пароля.



Шаг 9. Блок "Доступен ли элемент" – с помощью сессии `web_actions` проверяем, активна ли кнопка входа с XPath `"/html/body/div[1]/button"`.



Шаг 10. Если кнопка доступна – выполняем задержку 2 секунды, затем с помощью `web_actions` кликаем по кнопке и присваиваем переменной `result` значение "Вход выполнен".



Шаг 11. Если кнопка недоступна – присваиваем переменной result значение "Неуспешная авторизация".

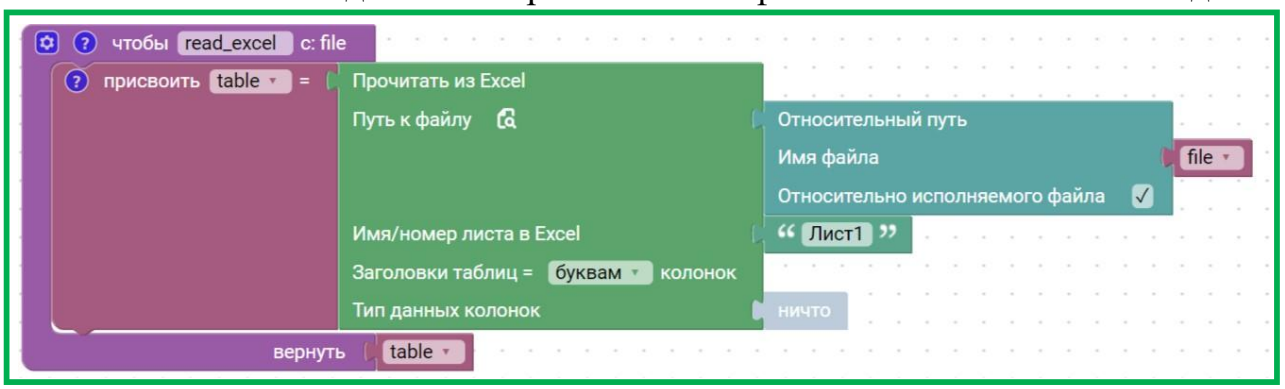
Шаг 12. Блок "Вернуть результат" – возвращаем переменную result в основной процесс, чтобы в дальнейшем можно было проверить, успешно ли прошла авторизация.

Функция `read_excel`

Шаг 1. Блок "Относительный путь" – формируем полный путь к файлу, имя которого передаётся в параметре `file`. На вход функция `read_excel` принимает параметр file, который содержит имя или относительный путь к Excel-файлу, 'requests.xlsx'. Этот параметр указывает, какой именно файл необходимо прочитать.

Шаг 2. Блок "Прочитать из Excel" – читаем данные из файла, путь к которому был сформирован на основе переданного параметра `file`, с листа "Лист1", сохраняя все данные в переменную `table`. Таким образом, функция может работать с любым Excel-файлом, имя которого будет передано при вызове.

Шаг 3. Блок "Вернуть результат" – возвращаем переменную `table` в основной процесс для дальнейшего использования при заполнении формы. Благодаря тому, что имя файла передаётся как параметр, функция является универсальной и может использоваться для чтения разных Excel-файлов без изменения её кода.



Функция `new_request`

Шаг 1. Блок "Цикл по (от до с шагом)" – организуем цикл для i от 1 до 5 включительно, то есть повторяем действия 5 раз для добавления пяти заявок. На вход функция принимает два параметра: `table` (таблица с данными из Excel, полученная от функции `read_excel`) и `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей). Параметр `table` содержит все данные из Excel-файла, где каждая строка соответствует отдельной заявке, а колонки содержат имя участника, email, название мероприятия и формат участия. Параметр web_actions передаётся из основной программы и содержит все методы для взаимодействия с браузером, такие как клики, ввод текста и ожидание элементов.

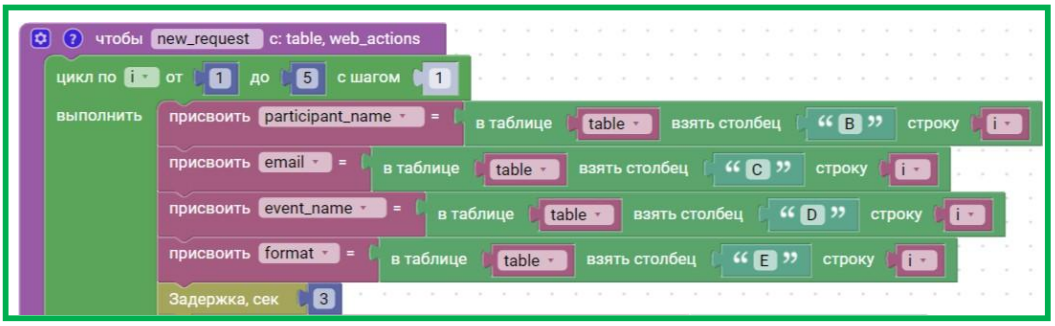
Шаг 2. Блок "Получить значение ячейки табличных данных" – из переменной table, полученной на вход, извлекаем значение из колонки B (Participant Name) по индексу i и сохраняем в переменную `participant_name`.

Шаг 3. Блок "Получить значение ячейки табличных данных" – из переменной `table` извлекаем значение из колонки C (`Email`) по индексу i и сохраняем в переменную `email`.

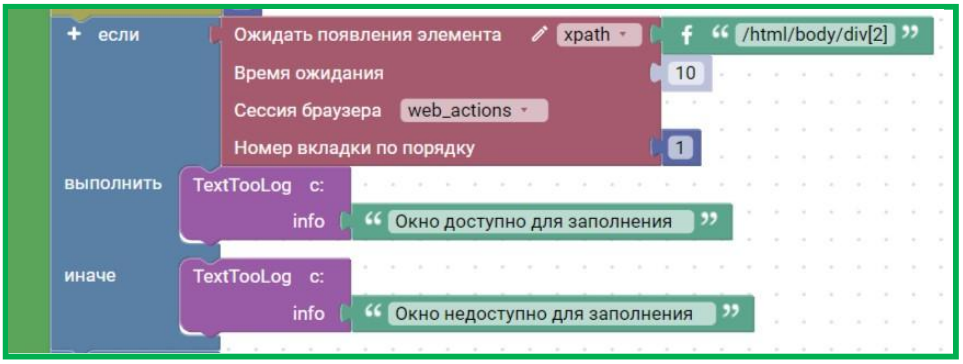
Шаг 4. Блок "Получить значение ячейки табличных данных" – из переменной table извлекаем значение из колонки D (Event Name) по индексу i и сохраняем в переменную `event_name`.

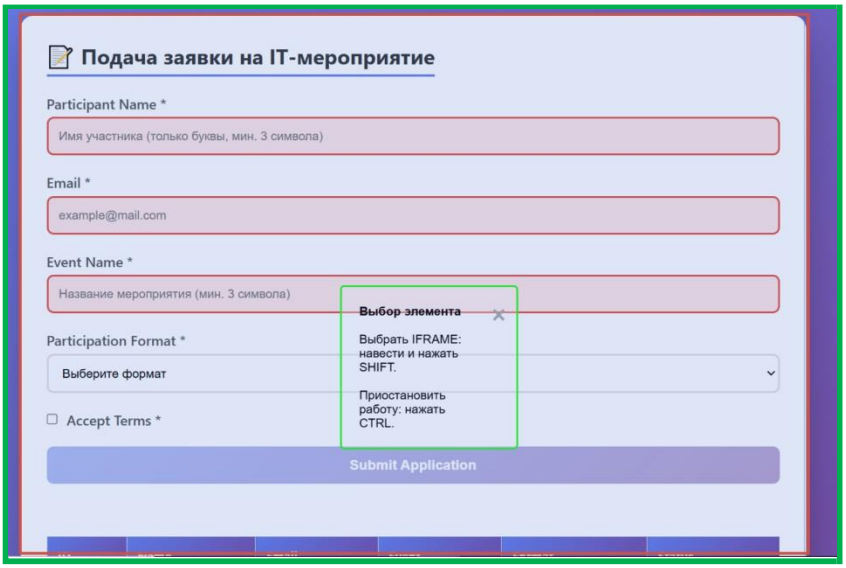
Шаг 5. Блок "Получить значение ячейки табличных данных" – из переменной table извлекаем значение из колонки E (Participation Format) по индексу i и сохраняем в переменную format2.

Шаг 6. Блок "Задержка" – пауза 3 секунды перед началом заполнения формы.



Шаг 7. Блок "Ожидать появление элемента" – с помощью переданной сессии `web_actions` проверяем, появилось ли окно формы заявки с XPath `"/html/body/div[2]"`, с таймаутом 10 секунд.



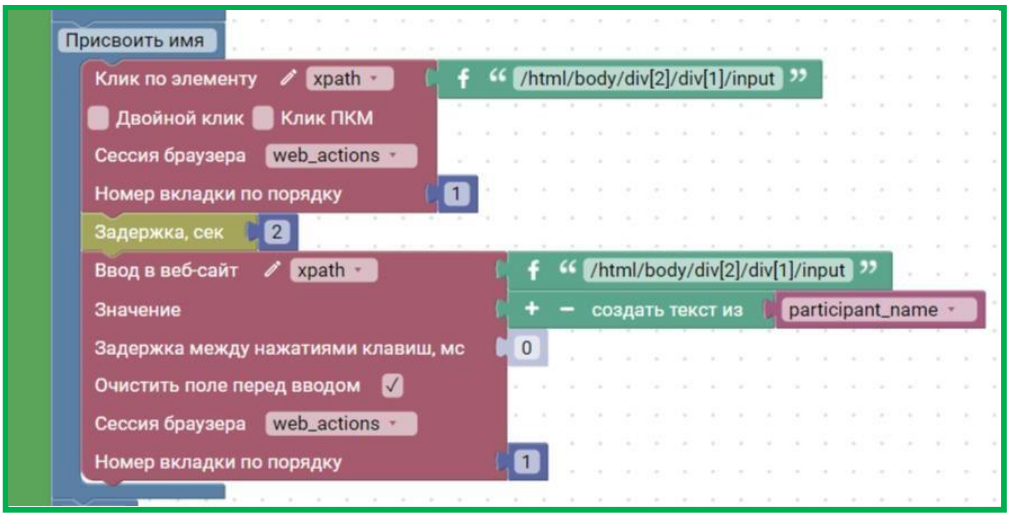


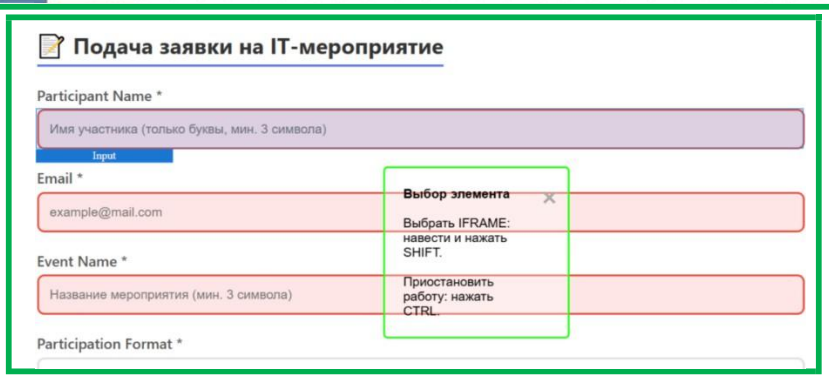
Шаг 8. Если окно доступно – вызываем функцию `TextTooLog` с сообщением "Окно доступно для заполнения". Если нет – вызываем `TextTooLog` с сообщением "Окно недоступно для заполнения".

Шаг 9. Блок "Клик по Web-элементу" – используя сессию web_actions, кликаем по полю Participant Name с XPath `"/html/body/div[2]/div[1]/input"`.

Шаг 10. Блок "Задержка" – пауза 2 секунды.

Шаг 11. Блок "Ввод в веб-сайт" – с помощью `web_actions` вводим значение переменной `participant_name` в поле Participant Name.

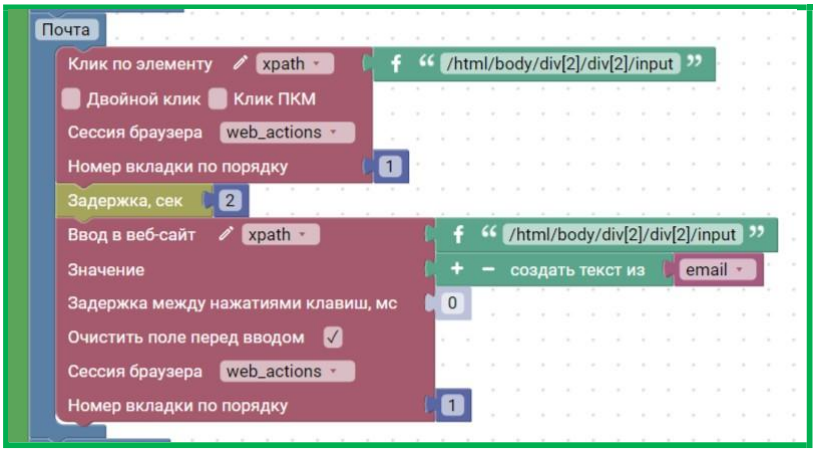


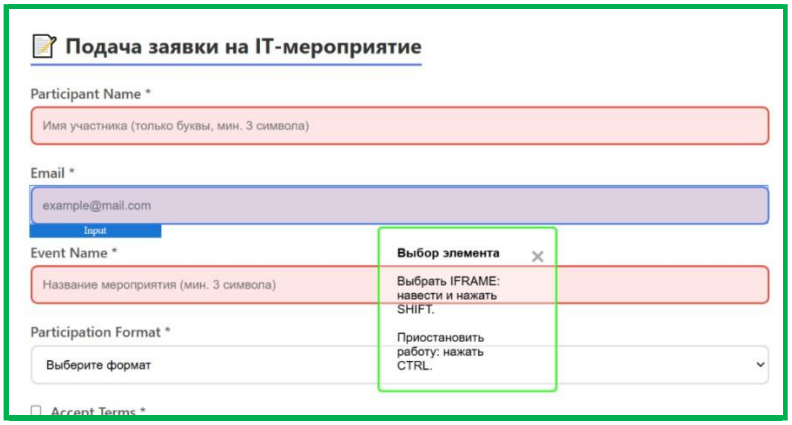


Шаг 12. Блок "Клик по Web-элементу" – используя web_actions, кликаем по полю Email с XPath `"/html/body/div[2]/div[2]/input"`.

Шаг 13. Блок "Задержка" – пауза 2 секунды.

Шаг 14. Блок "Ввод в веб-сайт" – с помощью `web_actions` вводим значение переменной `email` в поле Email.

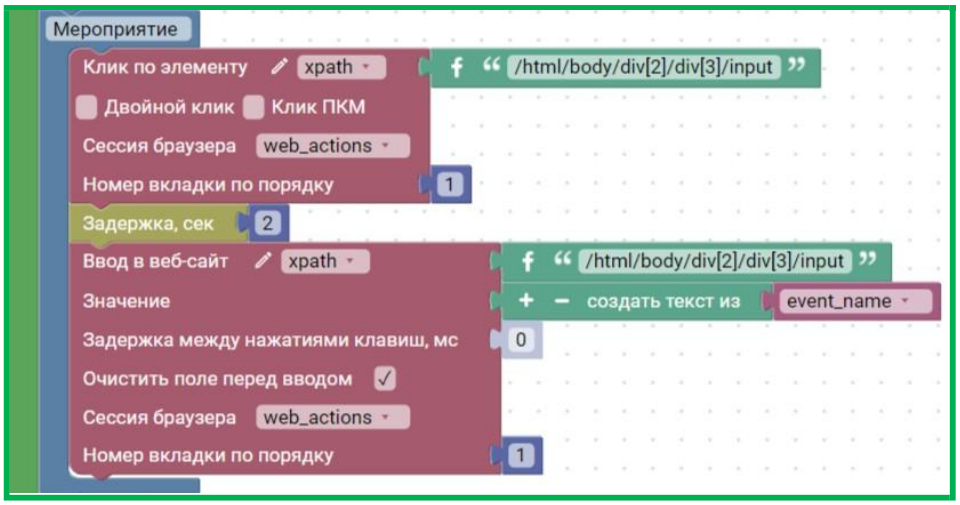


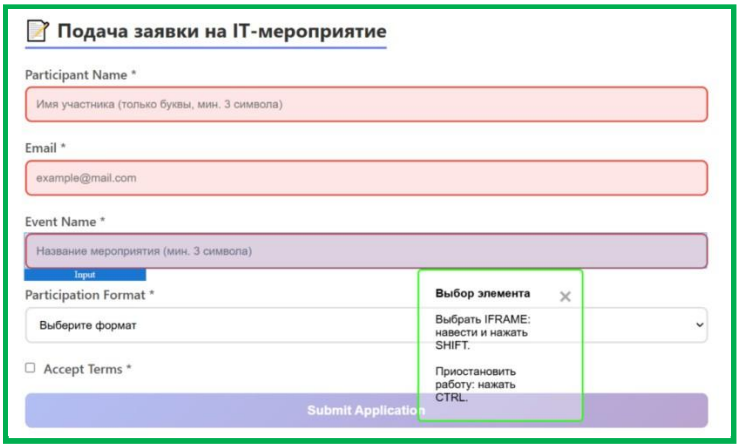


Шаг 15. Блок "Клик по Web-элементу" – используя `web_actions`, кликаем по полю Event Name с XPath `"/html/body/div[2]/div[3]/input"`.

Шаг 16. Блок "Задержка" – пауза 2 секунды.

Шаг 17. Блок "Ввод в веб-сайт" – с помощью `web_actions` вводим значение переменной `event_name` в поле `Event Name`.

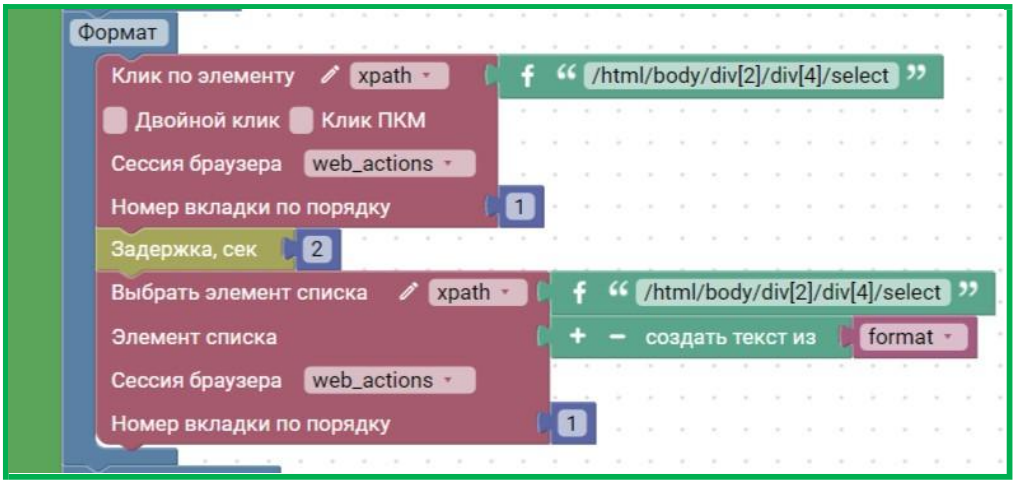


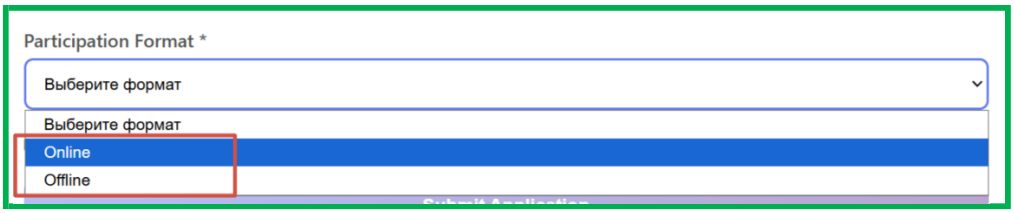


Шаг 18. Блок "Клик по Web-элементу" – используя `web_actions`, кликаем по выпадающему списку Participation Format с XPath `"/html/body/div[2]/div[4]/select"`.

Шаг 19. Блок "Задержка" – пауза 2 секунды для раскрытия списка.

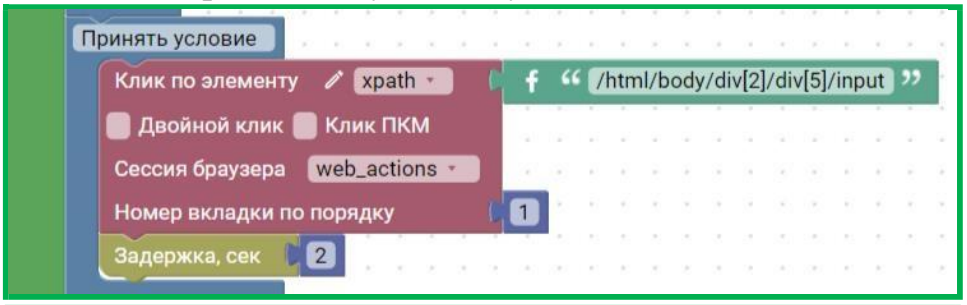
Шаг 20. Блок "Выбрать элемент списка" – с помощью `web_actions` выбираем значение из списка, соответствующее переменной `format` (Online или Offline).

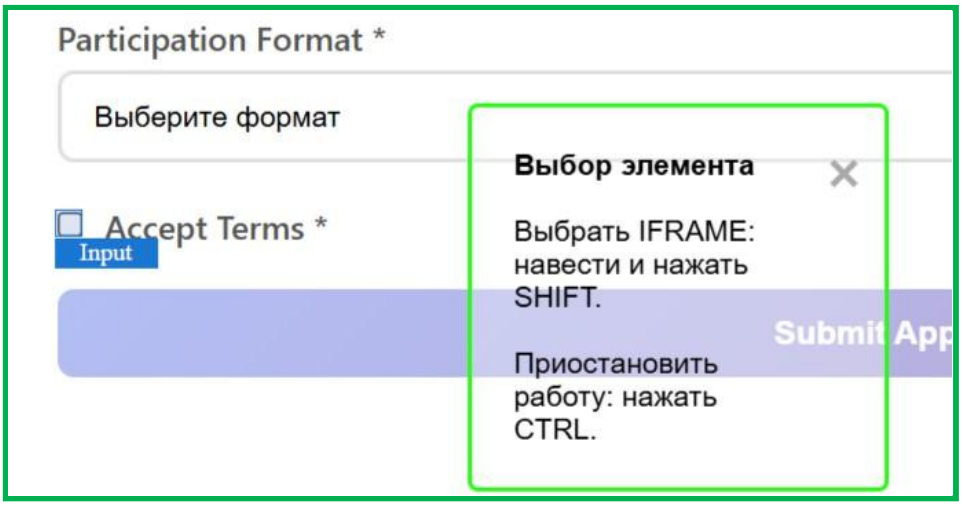




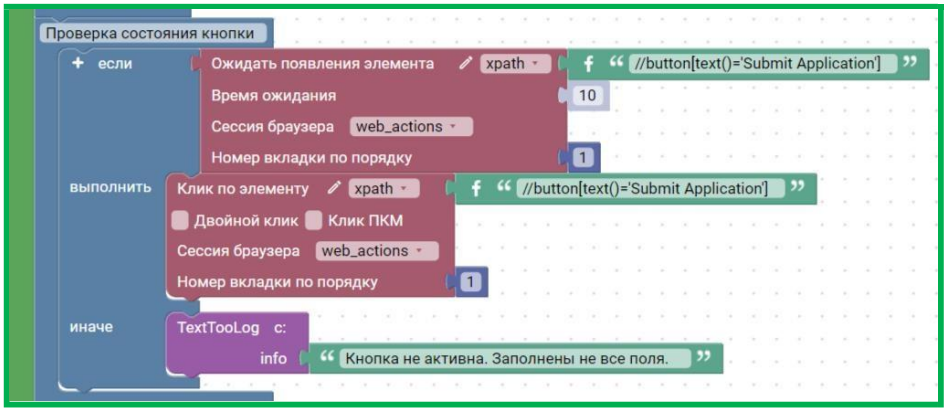
Шаг 21. Блок "Клик по Web-элементу" – используя `web_actions`, кликаем по чекбоксу Accept Terms с XPath `"/html/body/div[2]/div[5]/input"`.

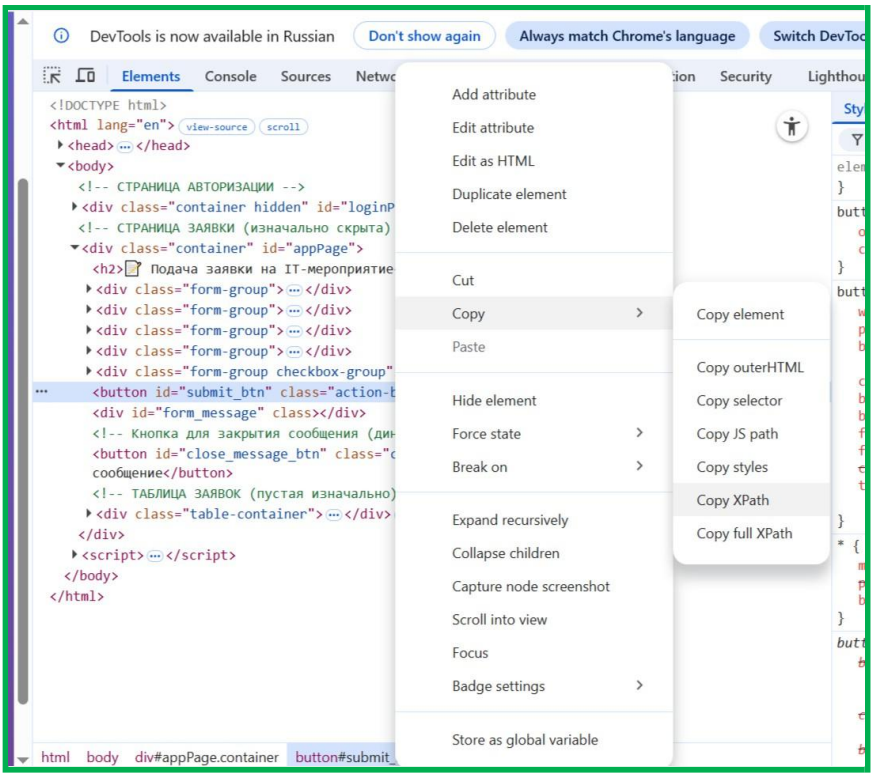
Шаг 22. Блок "Задержка" – пауза 2 секунды.





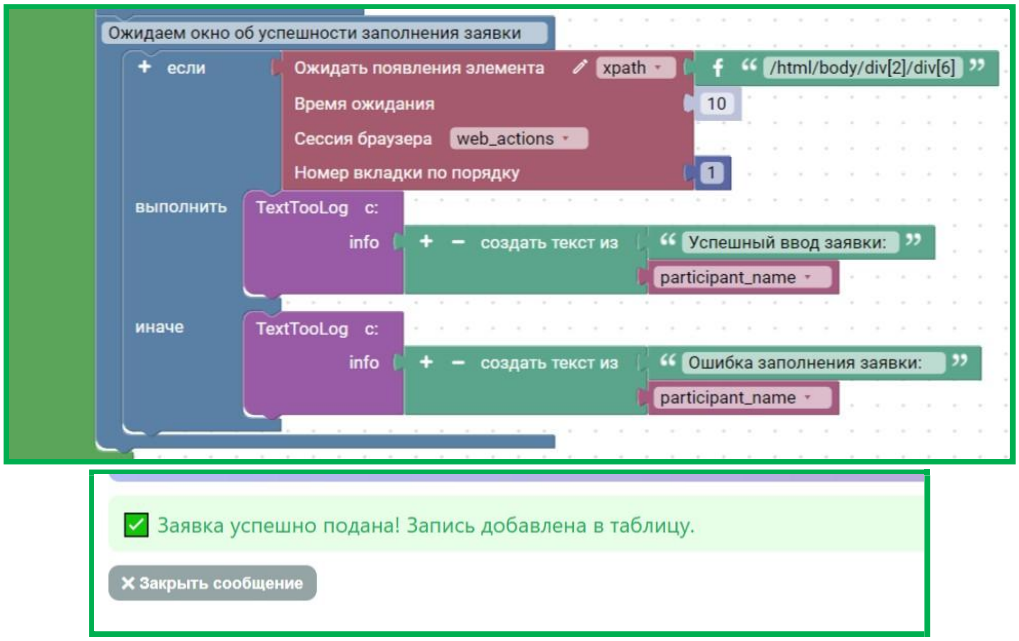
Шаг 23. Блок "Ожидать появление элемента" – с помощью web_actions ожидаем появления кнопки Submit Application с XPath `"//button[text()='Submit Application']"` с таймаутом 10 секунд. Чтобы выбрать XPath кнопки нам необходимо вызвать команду F12 и скопировать полный путь к элементу кнопки Submit Application.





Шаг 24. Если кнопка активна – выполняем клик по ней с помощью `web_actions`. Если нет – вызываем `TextTooLog` с сообщением "Кнопка не активна. Заполнены не все поля".

Шаг 25. Блок "Ожидать появление элемента" – с помощью `web_actions` ожидаем появления окна об успешной отправке заявки с XPath `"/html/body/div[2]/div[6]"` с таймаутом 10 секунд.

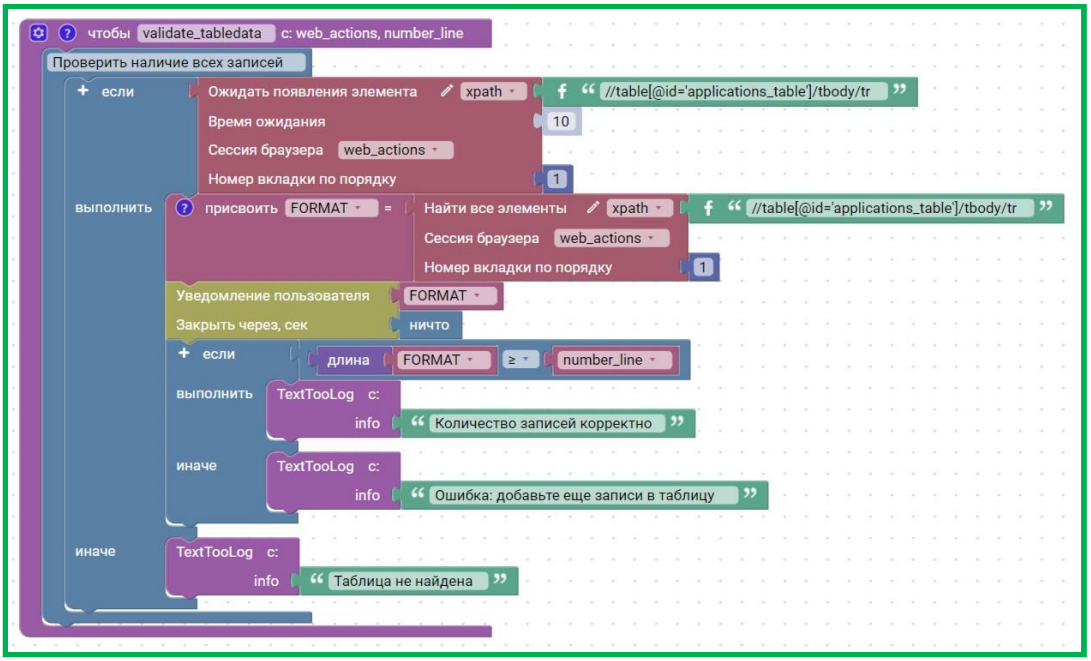


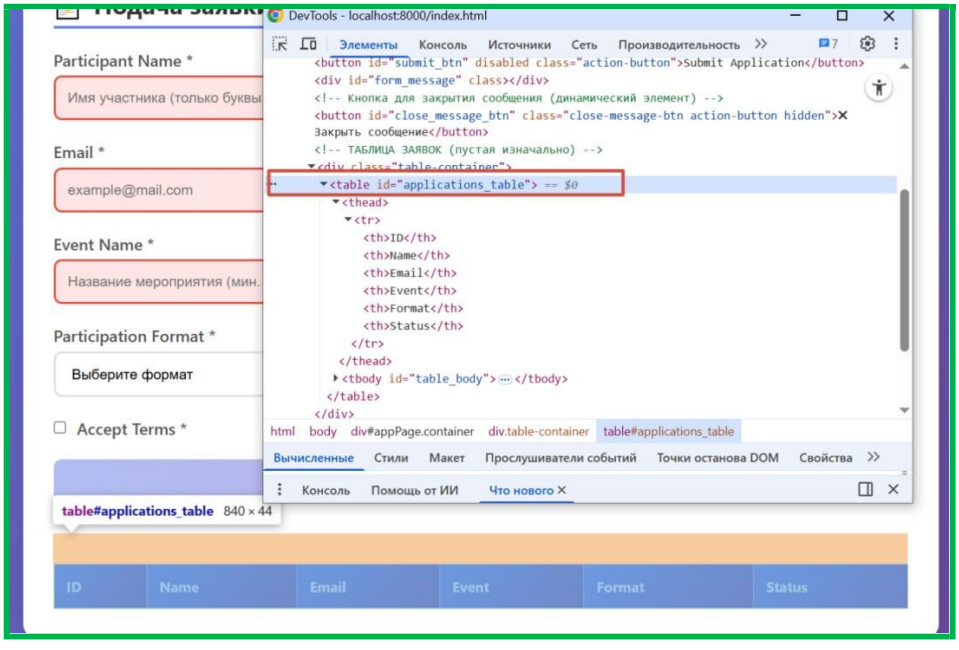
Шаг 26. Если окно появилось – вызываем `TextTooLog` с сообщением "Успешный ввод заявки: " + `participant_name`. Если нет – вызываем `TextTooLog` с сообщением "Ошибка заполнения заявки: " + `participant_name`.

Шаг 27. Блок "Задержка" – пауза 2 секунды перед переходом к следующей заявке. Таким образом, функция new_request использует переданные данные таблицы для последовательного заполнения пяти заявок и сессию браузера для автоматизации всех действий на веб-странице.

Функция `validate_tabledata`

Шаг 1. Блок "Ожидать появление элемента" – с помощью переданной сессии `web_actions` ожидаем появления строк таблицы с XPath `"//table[@id='applications_table']/tbody/tr"` с таймаутом 10 секунд. На вход функция принимает два параметра: `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей) и `number_line` (числовое значение, определяющее минимальное количество строк, которое должно присутствовать в таблице). Параметр `web_actions` передаётся из основной программы и содержит все методы для взаимодействия с браузером, такие как поиск элементов и ожидание их появления. Параметр `number_line` позволяет задать требуемое количество записей, например 5, и делает функцию универсальной для проверки любого порогового значения.





Шаг 2. Блок "Найти все элементы" – используя переданную сессию `web_actions`, находим все строки таблицы по тому же XPath и сохраняем их в переменную `FORMAT`.

Шаг 3. Блок "Уведомление пользователя" – отображаем пользователю количество найденных строк в виде всплывающего уведомления.

Шаг 4. Блок "Условный оператор" – проверяем, больше или равно количе-ство строк значению, переданному в параметре `number_line`. Таким образом, если при вызове функции передать `number_line = 5`, то условие проверит, что в таблице не менее 5 записей.

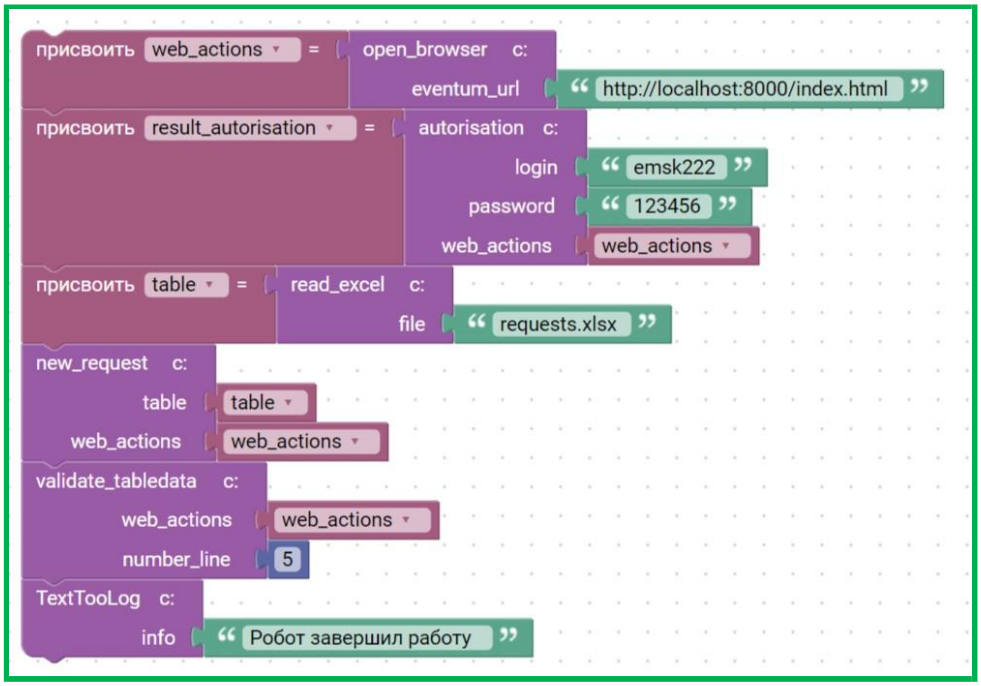
Шаг 5. Если количество строк корректно (то есть больше или равно `number_line`) – вызываем функцию `TextTooLog` с сообщением "Количество записей корректно".

Шаг 6. Если количество строк меньше требуемого значения `number_line` – вызываем `TextTooLog` с сообщением "Ошибка: добавьте еще записи в таблицу".

Шаг 7. Если таблица не найдена на странице – вызываем `TextTooLog` с сообщением "Таблица не найдена". Благодаря тому, что `number_line` передаётся как параметр, функция может использоваться для проверки разного количества записей без изменения её кода, а параметр `web_actions` обеспечивает возможность взаимодействия с браузером для поиска и анализа элементов таблицы.

Главный процесс `web_1`

Главный процесс является оркестратором всего RPA-сценария. Он последовательно вызывает функции, каждая из которых отвечает за отдельный этап автоматизации.



Шаг 1. Вызываем функцию `open_browser` с передачей URL "http://localhost:8000/index.html". Результат сохраняем в переменную `web_actions`.

Шаг 2. Вызываем функцию `autorisation` с передачей логина "emsk222", пароля "123456" и переменной `web_actions`. Результат сохраняем в переменную `result_autorisation`.

Шаг 3. Вызываем функцию `read_excel` с передачей имени файла `"requests.xlsx"`. Результат сохраняем в переменную `table`.

Шаг 4. Вызываем функцию `new_request` с передачей переменных `table` и `web_actions`. Функция добавляет 5 заявок на основе данных из Excel.

Шаг 5. Вызываем функцию `validate_tabledata` с передачей переменной `web_actions` и числа 5. Функция проверяет, что в таблице содержится не менее 5 записей.

Шаг 6. Вызываем функцию `TextTooLog` с сообщением "Робот завершил работу" для логирования успешного окончания выполнения сценария.

Практическая работа выполнена в полном объёме. Разработано веб-приложение для подачи заявок на мероприятия с таблицей, формой добавления и модальными окнами подтверждения. Создан модульный RPA-робот в среде Puzzle RPA, состоящий из функций запуска браузера, авторизации, чтения Excel, массового добавления заявок и проверки корректности добавленных записей. Реализована система логирования для контроля выполнения всех этапов автоматизации. Выполнена проверка корректности добавления заявок в таблицу.

Робот полностью заменяет ручной труд при вводе заявок, исключает ошибки, связанные с человеческим фактором, и значительно ускоряет процесс обработки данных.

Практическая часть

Задание 1. Разработка веб-приложения

Необходимо разработать веб-приложение с пользовательским интерфейсом.

Технология разработки выбирается самостоятельно (например: HTML/CSS/JavaScript, React, Vue и др.).

Каждый элемент интерфейса должен иметь уникальный id

🌐 Общая логика работы приложения

Приложение должно реализовывать следующий пользовательский сценарий:

1. Пользователь открывает страницу приложения
2. Заполняет форму регистрации инцидента
3. Отправляет заявку
4. Видит подтверждение в текстовой метке и добавленную запись в таблице

📝 Форма регистрации инцидента

📝 Поля ввода

Incident Title

Назначение: Краткое описание инцидента безопасности

Требования:

- обязательное поле
- минимальная длина — 5 символов
- допускается текст и цифры

Reporter
Назначение: Сотрудник, обнаруживший инцидент
Требования:

- обязательное поле
- только текстовые символы (без цифр и спецсимволов)
- минимальная длина — 3 символа

🎛 **Выпадающий список**
System
Назначение: Система, в которой произошёл инцидент
Значения:

- Authentication Server
- Database
- Web Portal
- Internal Network

Требования:

- обязательный выбор
- значение по умолчанию отсутствует

Severity
Назначение: Уровень критичности инцидента
Значения:

- Low
- Medium
- High
- Critical

Требования:

- обязательный выбор
- значение по умолчанию отсутствует

⚙ **Логика валидации формы**
Форма считается **валидной**, если:

- заполнено поле Incident Title
- Incident Title содержит минимум 5 символов
- заполнено поле Reporter
- Reporter содержит только текстовые символы
- выбран System
- выбран Severity

⭕ **Кнопка**
Report Incident

⚙️ Управление состоянием (disabled)

Кнопка должна:

- иметь состояние disabled = true, если:
  - не заполнено поле Incident Title
  - Incident Title < 5 символов
  - не заполнено поле Reporter
  - не выбран System
- иметь состояние disabled = false, если:
  - все поля заполнены корректно

⚠️ Поведение при ошибках

При попытке отправки (если реализовано):

- подсветка некорректных полей
- вывод сообщений в текстовой метке:

Примеры:

Incident Title must contain at least 5 characters

Reporter field must contain only letters

Please select a system

📩 Отправка формы

При нажатии Report Incident:

1. Выполняется финальная проверка
2. Создаётся новая запись
3. Очищается форма
4. Отображается сообщение в текстовой метке об отправке формы

⚙️ Требования к поведению

- появляется с задержкой (1–2 секунды)
- автоматически исчезает через 2–3 секунды **или**
- закрывается кнопкой

📊 Таблица инцидентов

📋 Структура

ID	Title	Reporter	System	Severity	Status

⚙️ Правила заполнения

ID

- автоинкремент
- уникальное значение

Title, Reporter, System, Severity

- берутся из формы

Status

{ "status": "Reported" }

- новая запись добавляется в таблицу
- таблица обновляется с задержкой

⚠️ Поведение таблицы

- новая запись добавляется в конец таблицы
- таблица обновляется с задержкой (имитация асинхронности)
- допускается частичная подгрузка данных

📩 Тестовые данные

№	Incident Title	Reporter	System	Severity
1	Unauthorized access attempt	Ivan Petrov	Authentication Server	Critical
2	Suspicious database query	Anna Smirnova	Database	High
3	Phishing email detected	Dmitry Orlov	Web Portal	Medium
4	Unusual network traffic	Olga Ivanova	Internal Network	High
5	Failed login attempts	Sergey Kuznetsov	Authentication Server	Critical

---

⚠️ Требования для имитации реального веб-приложения

Необходимо реализовать:

⏳ Асинхронность

- задержка появления:
  - формы
  - таблицы
  - сообщения об успехе

🔄 Динамические элементы

- хотя бы один элемент с изменяющимся id, например: id="btn_report_12345"

🪀 Повторяющиеся элементы

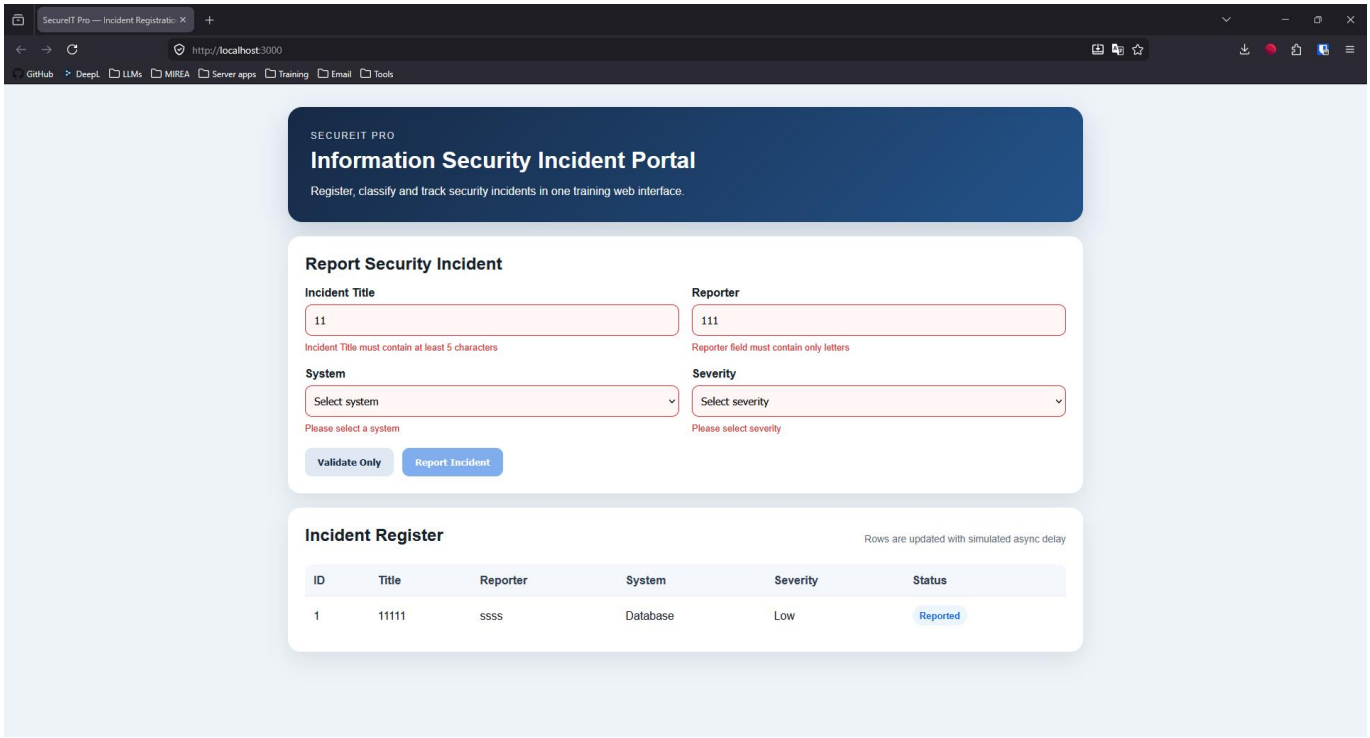
- минимум 2 одинаковых элемента:

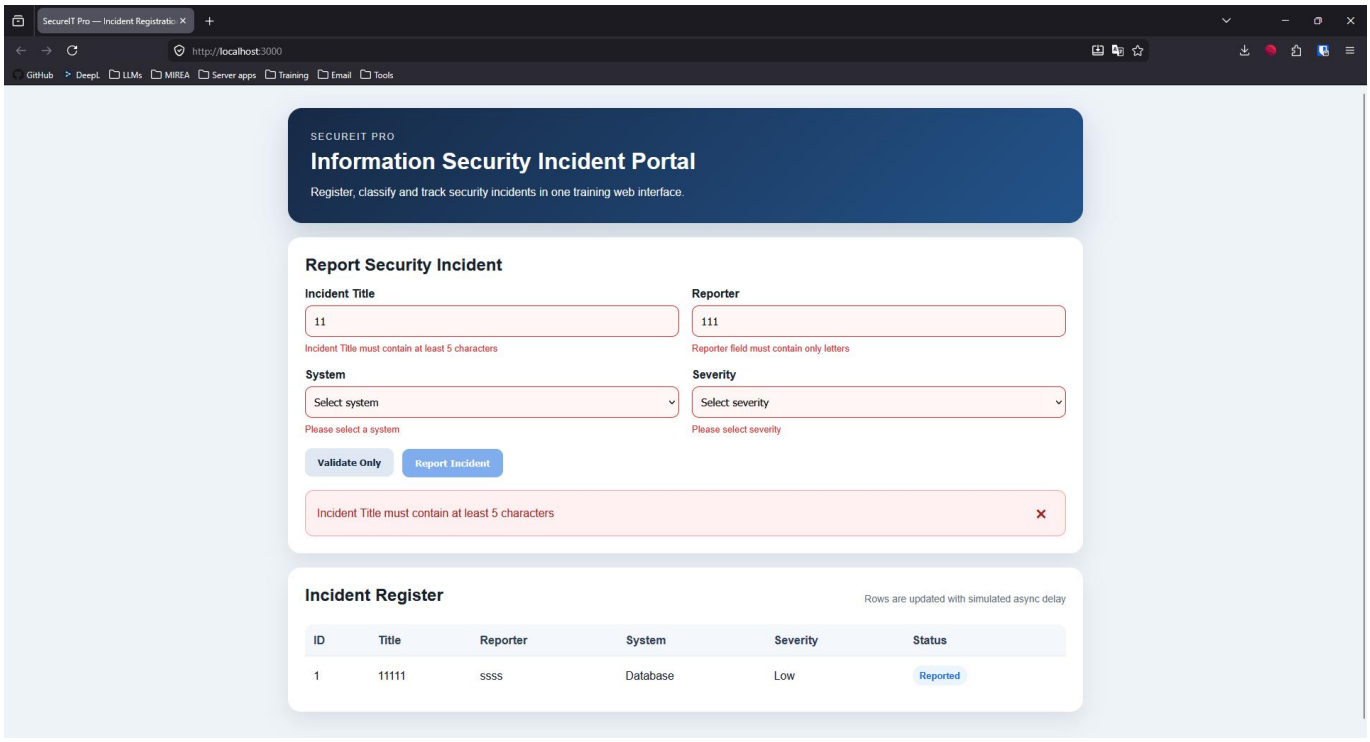
- кнопки
- строки таблицы

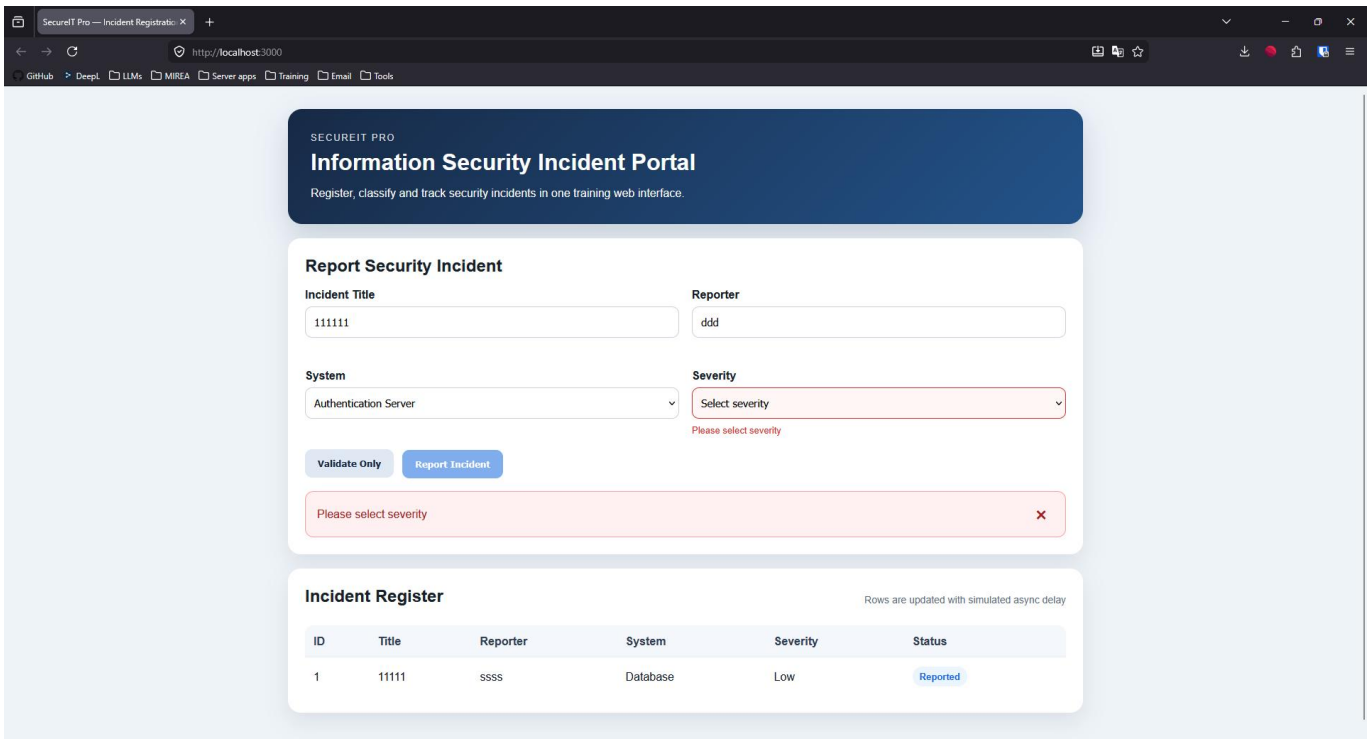
🎯 Цель

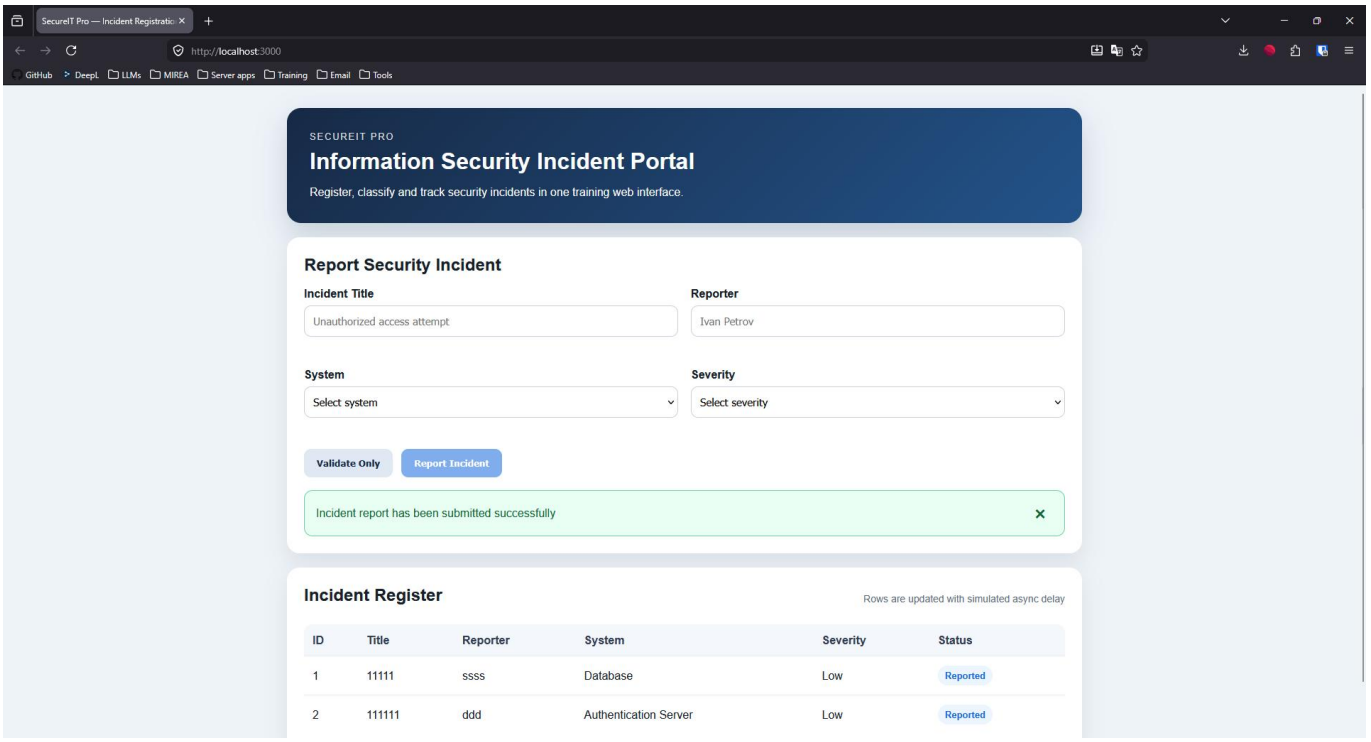
Создать веб-приложение, которое:

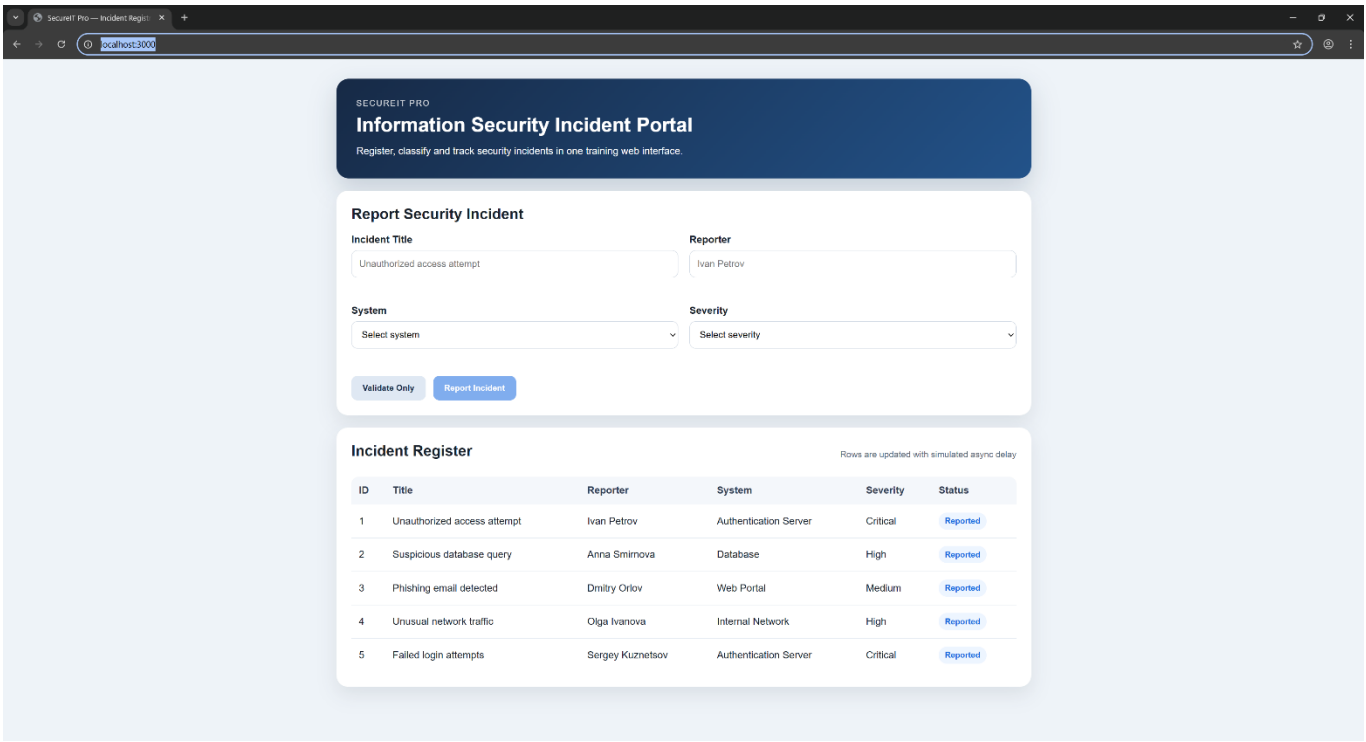
- имитирует реальный пользовательский интерфейс
- содержит типовые проблемы веб-автоматизации
- позволяет отработать:
  - XPath
  - ожидания
  - проверки

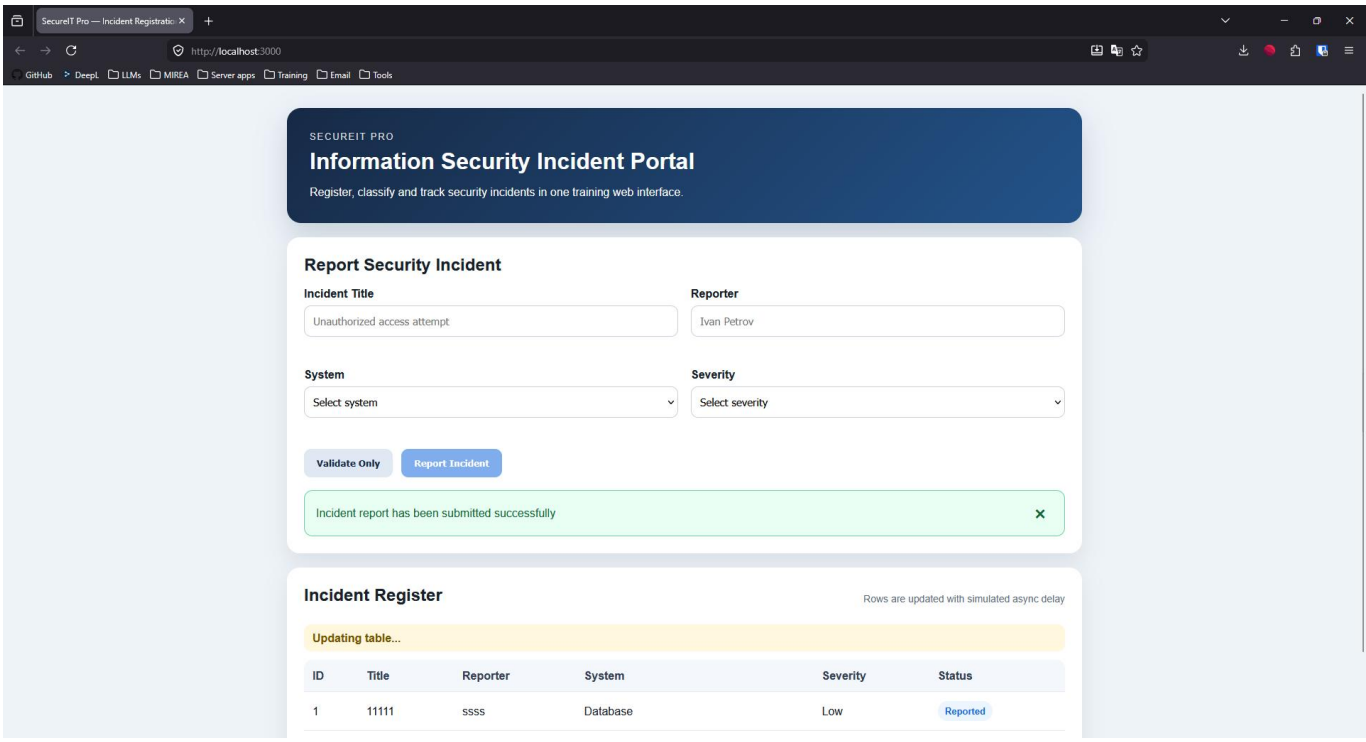



SecureIT Pro — Incident Registratic
http://localhost:3000
GitHub DeepL LLMs MIREA Server apps Training Email Tools
SECUREIT PRO
Information Security Incident Portal
Register, classify and track security incidents in one training web interface.
Report Security Incident
Incident Title
11
Incident Title must contain at least 5 characters
Reporter
111
Reporter field must contain only letters
System
Select system
Please select a system
Severity
Select severity
Please select severity
Validate Only
Report Incident
Incident Title must contain at least 5 characters
Incident Register
Rows are updated with simulated async delay
ID
Title
Reporter
System
Severity
Status
1
11111
ssss
Database
Low
Reported


SecureIT Pro — Incident Registratic
http://localhost:3000
GitHub DeepL LLMs MIREA Server apps Training Email Tools
SECUREIT PRO
Information Security Incident Portal
Register, classify and track security incidents in one training web interface.
Report Security Incident
Incident Title
111111
Reporter
ddd
System
Authentication Server
Severity
Select severity
Please select severity
Validate Only
Report Incident
Please select severity
Incident Register
Rows are updated with simulated async delay
ID
Title
Reporter
System
Severity
Status
1
11111
ssss
Database
Low
Reported


SECUREIT PRO
Information Security Incident Portal
Register, classify and track security incidents in one training web interface.
Report Security Incident
Incident Title
Unauthorized access attempt
Reporter
Ivan Petrov
System
Select system
Severity
Select severity
Validate Only
Report Incident
Incident report has been submitted successfully
Incident Register
Rows are updated with simulated async delay
ID
Title
Reporter
System
Severity
Status
1
11111
ssss
Database
Low
Reported
2
111111
ddd
Authentication Server
Low
Reported


SECUREIT PRO
Information Security Incident Portal
Register, classify and track security incidents in one training web interface.
Report Security Incident
Incident Title
Unauthorized access attempt
Reporter
Ivan Petrov
System
Select system
Severity
Select severity
Validate Only
Report Incident
Incident Register
Rows are updated with simulated async delay
ID
Title
Reporter
System
Severity
Status
1
Unauthorized access attempt
Ivan Petrov
Authentication Server
Critical
Reported
2
Suspicious database query
Anna Smirnova
Database
High
Reported
3
Phishing email detected
Dmitry Orlov
Web Portal
Medium
Reported
4
Unusual network traffic
Olga Ivanova
Internal Network
High
Reported
5
Failed login attempts
Sergey Kuznetsov
Authentication Server
Critical
Reported



Задание 2. Базовая автоматизация веб-приложения

Необходимо разработать процесс автоматизации в **Puzzle RPA**, имитирующий действия пользователя в браузере.

🤖 Алгоритм работы робота

1. Создать сессию браузера
2. Открыть приложение
3. Дождаться загрузки страницы

📝 Заполнение формы

Для каждой записи:

Ввод данных

- заполнить Incident Title
- заполнить Reporter
- выбрать System
- выбрать Severity

Проверка состояния кнопки

- проверить, что кнопка Report Incident активна;
- если кнопка неактивна:
    - определить причину;
    - заполнить или исправить данные;

Отправка формы

- нажать Report Incident
- дождаться появления сообщения об отправке в текстовой метке

📊 **Финальная проверка**

- проверить наличие всех записей в таблице (не менее 5)
- проверить корректность данных

📊 **Ожидаемый результат выполнения**

Таблица содержит все инциденты:

ID	Title	Reporter	System	Severity	Status
1	Unauthorized access attempt	Ivan Petrov	Authentication Server	Critical	Reported
2	Suspicious database query	Anna Smirnova	Database	High	Reported
3	Phishing email detected	Dmitry Orlov	Web Portal	Medium	Reported

---

🔎 **Требования к корректности выполнения**

✓ **Веб-приложение**

- корректно реализовано состояние disabled;
- форма не позволяет отправлять пустые данные;
- таблица корректно отображает записи;

✓ **Работа робота**

- робот корректно заполняет форму;
- учитывает состояние элементов;
- успешно отправляет заявки;

✓ **Данные**

- все записи добавлены;
- значения соответствуют входным данным;
- статус каждой записи — "Reported".

⚠️ **Дополнительная проверка**

Рекомендуется протестировать сценарий с некорректными данными:

Incident Title	Reporter	System	Severity
AB	User1	Database	Critical

Ожидаемое поведение:

- кнопка Report Incident остаётся неактивной;

- робот обнаруживает проблему;
- корректно завершает сценарий.

📜 **Логирование выполнения процесса**

Необходимо реализовать логирование.

Формат:

- текстовый файл или JSON

📌 **В журнале должны фиксироваться:**

- запуск процесса
- обработка каждой заявки
- отправка формы
- ошибки
- завершение процесса

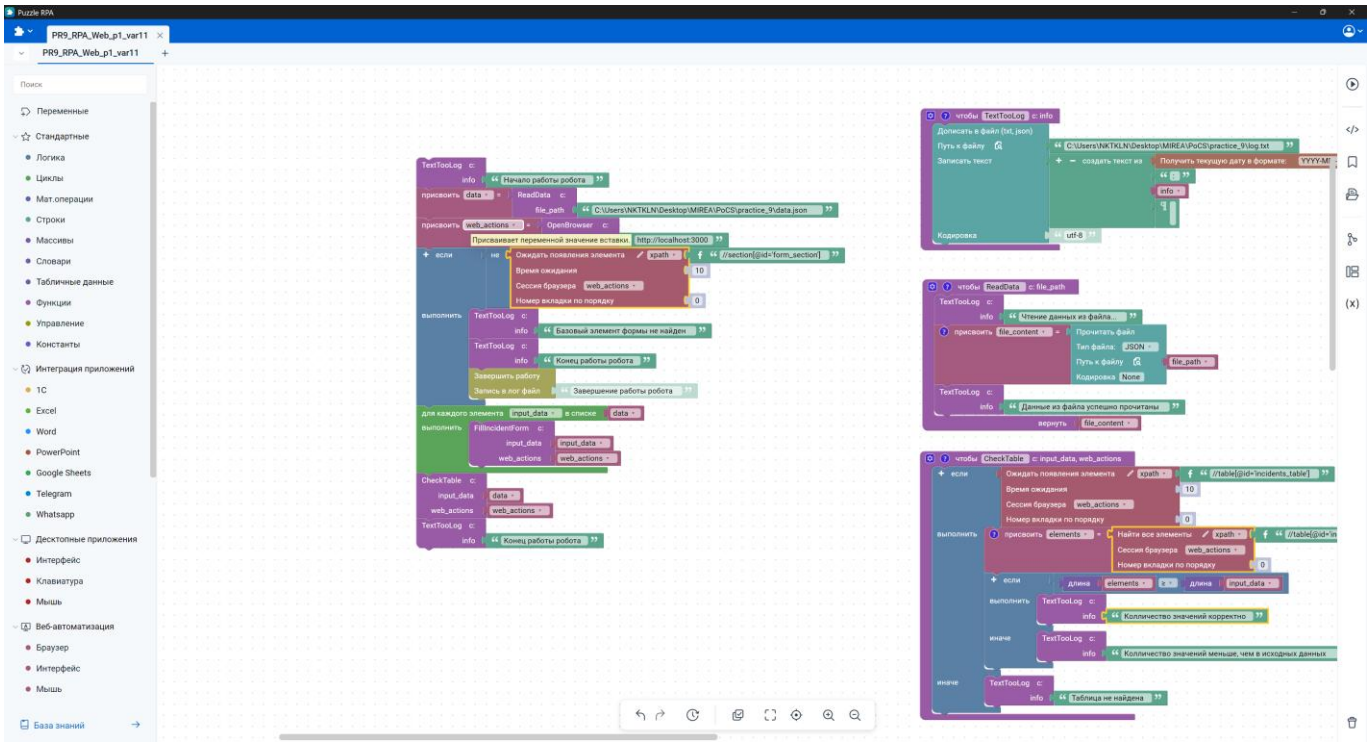
Пример записи

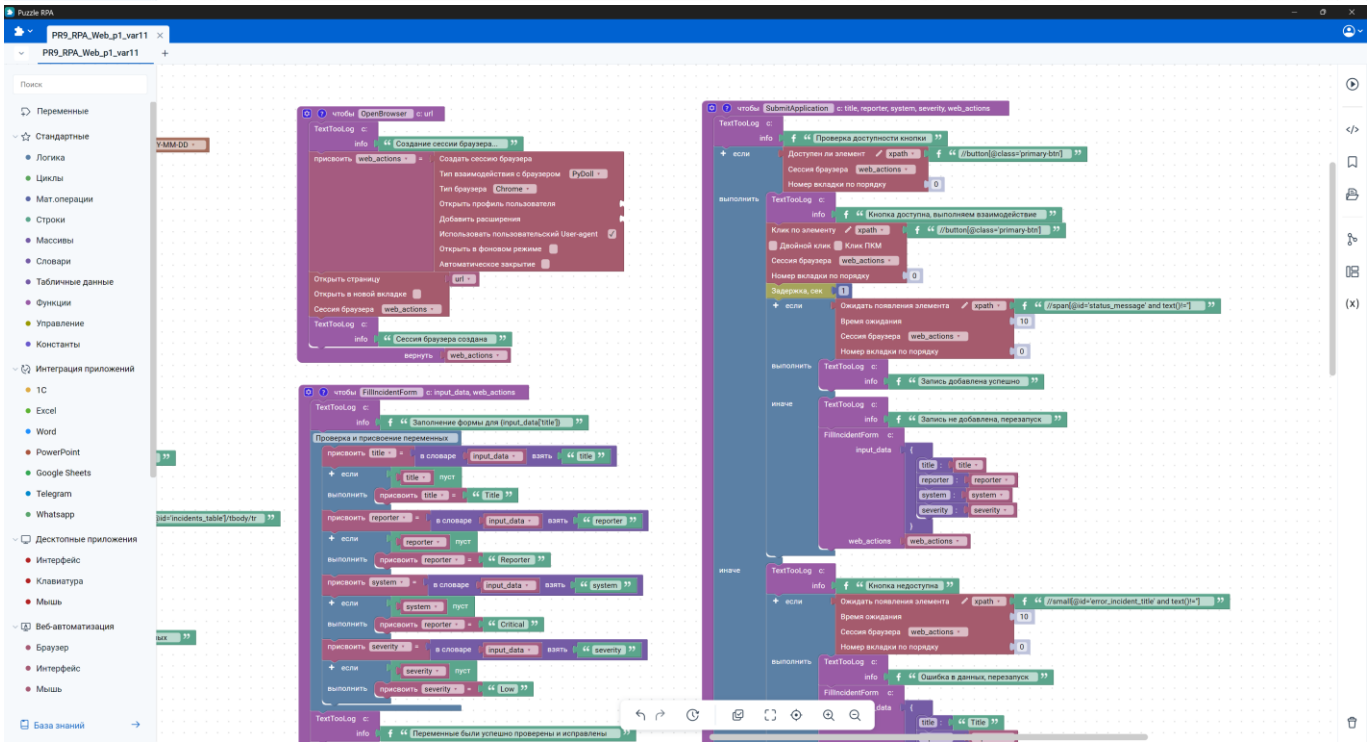
```
{
  "time": "2026-04-03 10:05:12",
  "action": "ReportIncident",
  "title": "Unauthorized access attempt",
  "severity": "Critical",
  "result": "success"
}
```

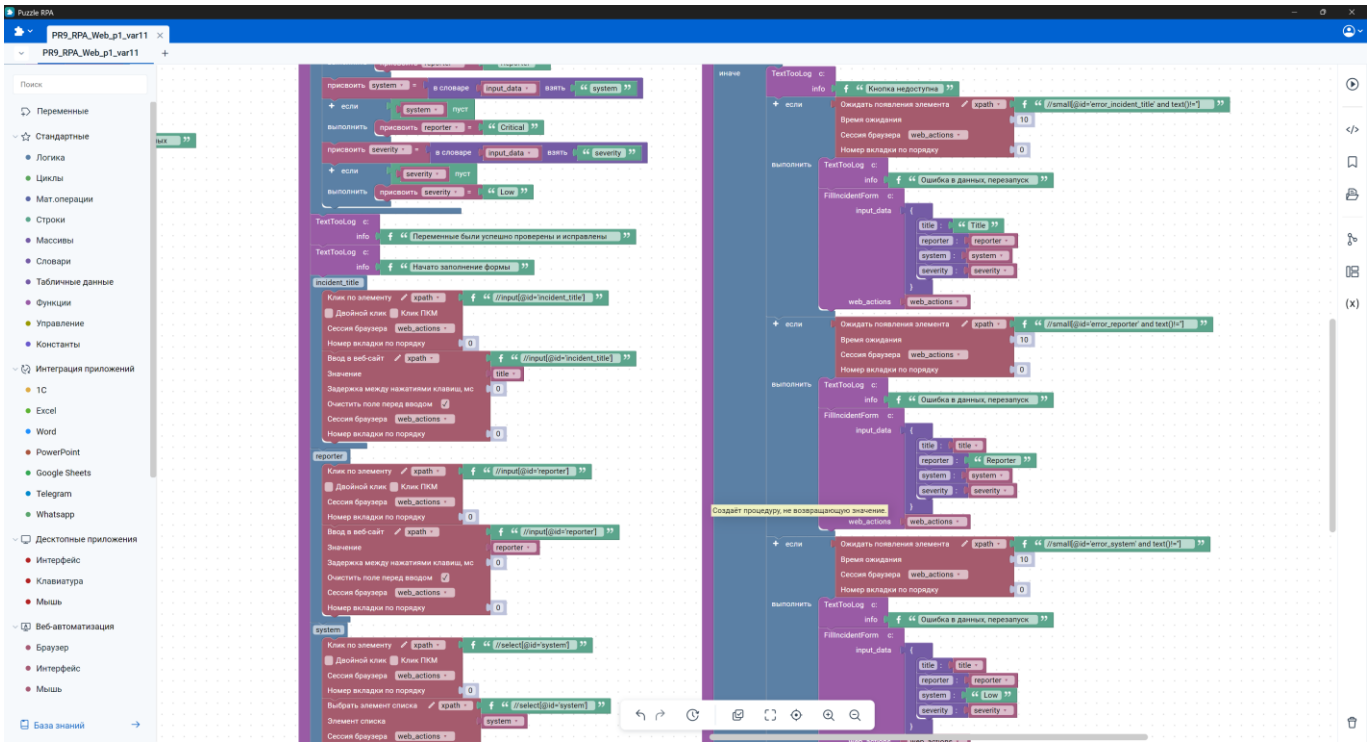
🧩 **Требования к модульности**

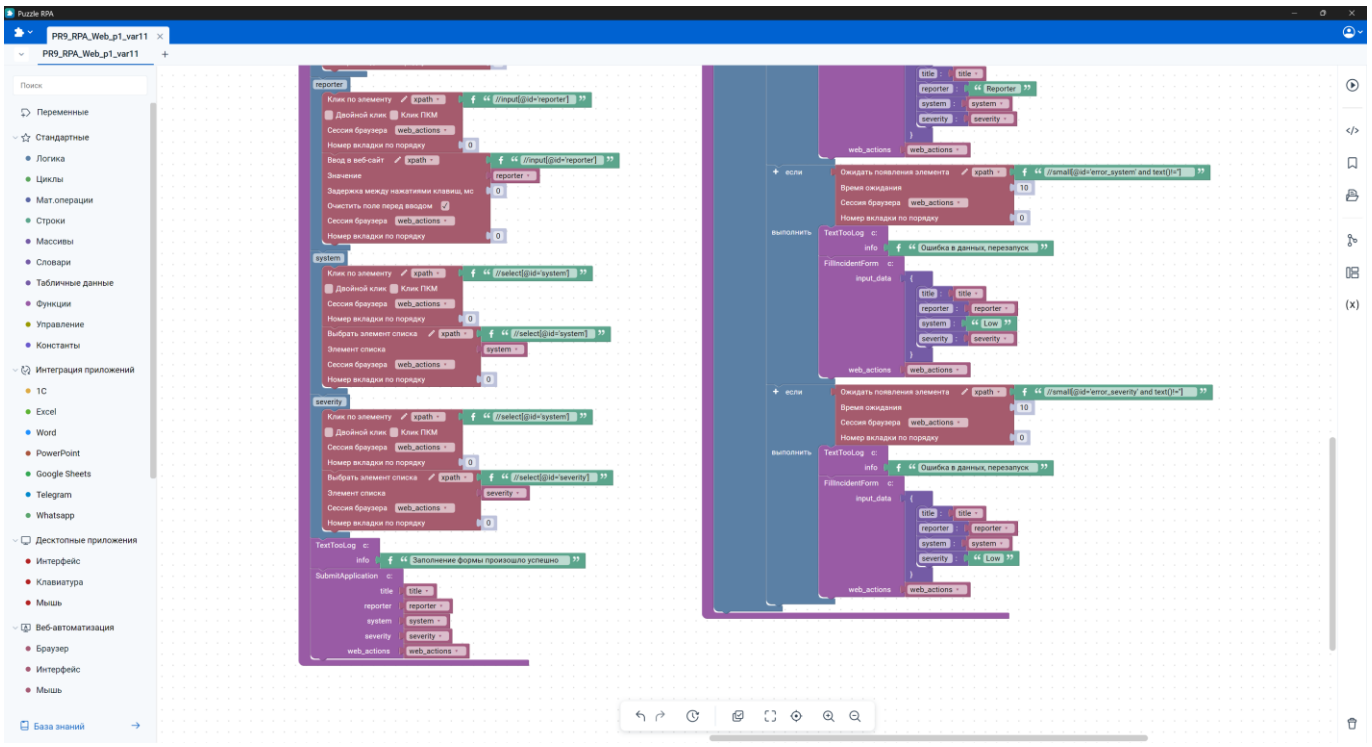
Рекомендуется реализовать структуру с использованием функций/подпроцессов:

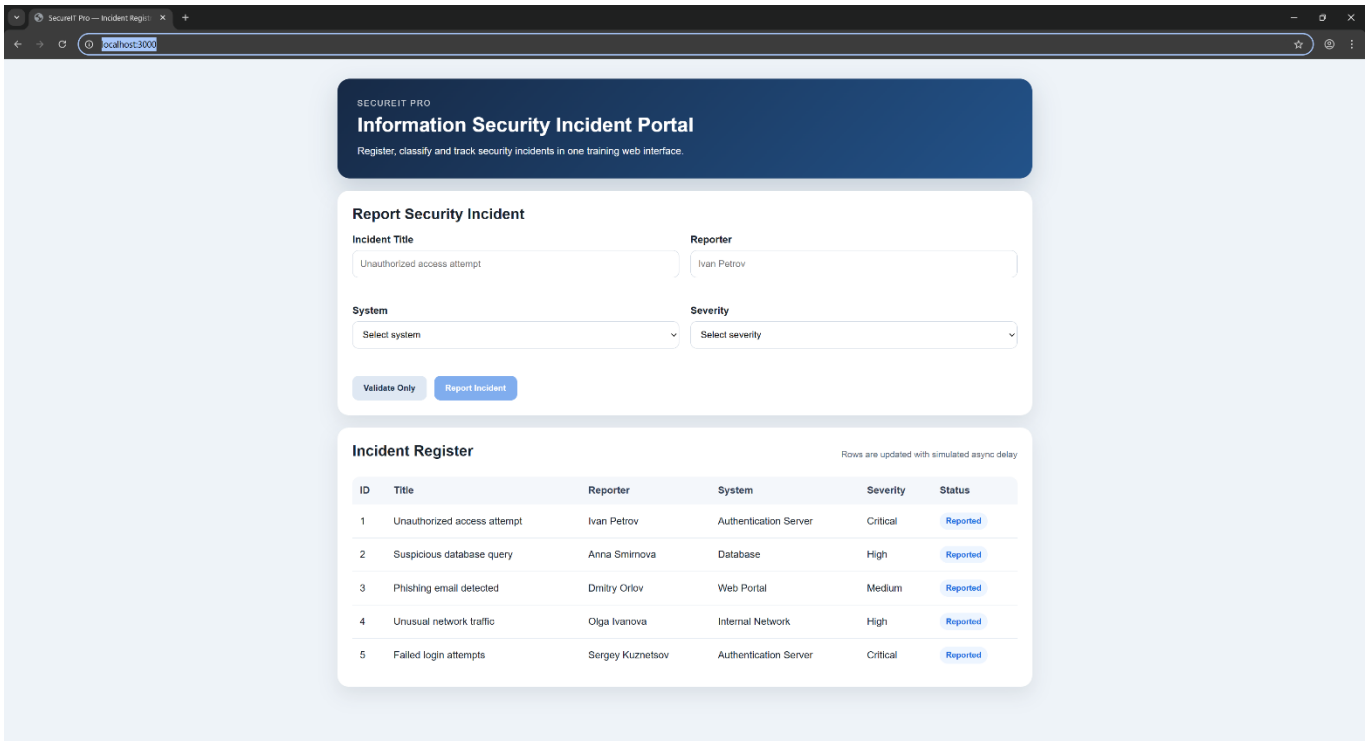
- OpenBrowser
- FillIncidentForm
- SubmitApplication
- CheckTable

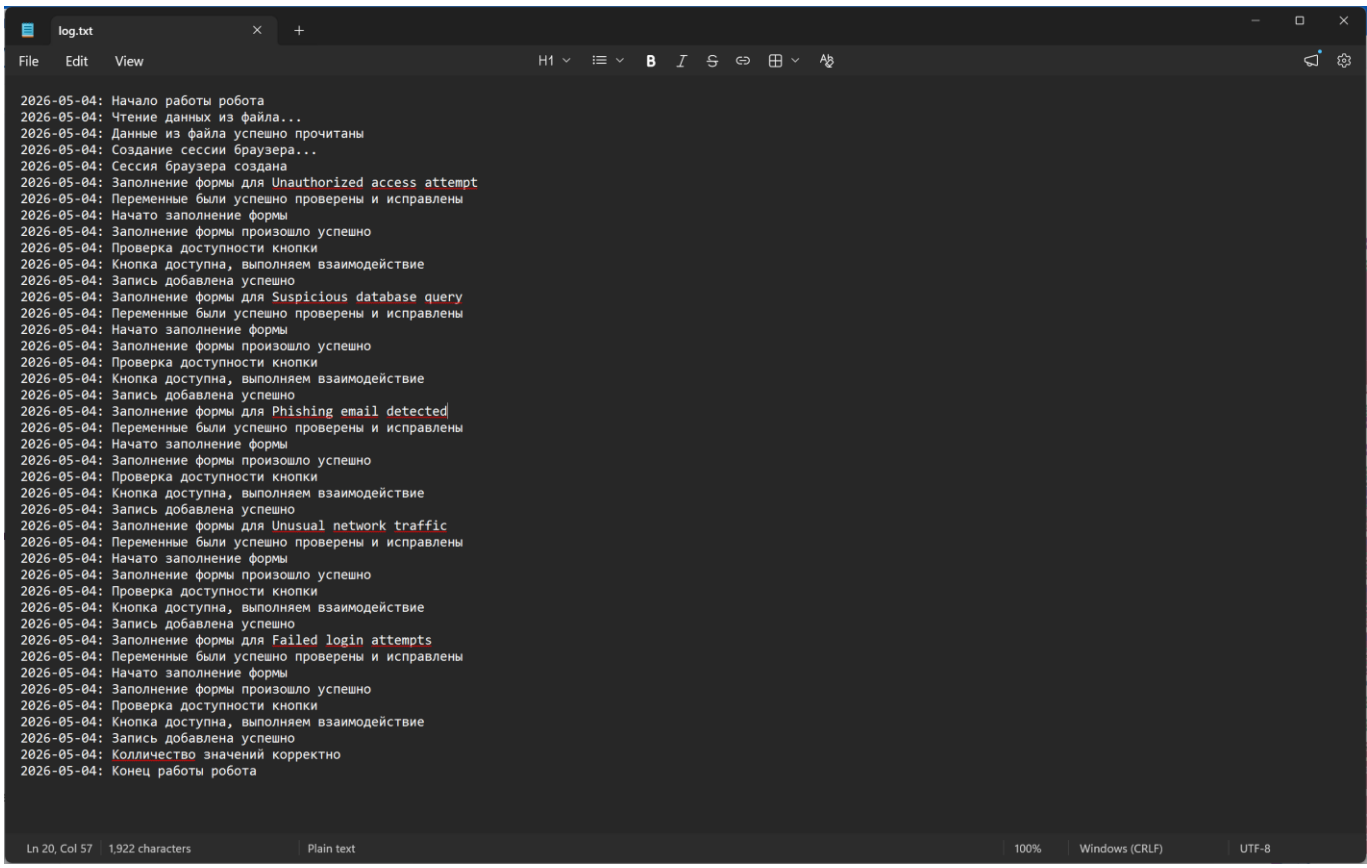












DeepSeek

https://chat.deepseek.com/share/c7s04b0zldk3uobelo

GitHub https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_9